DMI COLLEGE OF ENGINEERING

(Affiliated to Anna University, Chennai-600 025)

Palanchur, Chennai-600 123.

DEPARTMENT OF
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

AL3391

ARTIFICIAL INTELLIGENCE

for

BTech - AI&DS - III SEM

Dr. R. Balamurugan
Assistant Professor
DMICE

AL3391 ARTIFICIAL INTELLIGENCE L T P C
3 0 0 3

COURSE OBJECTIVES:

The main objectives of this course are to:

- Learn the basic AI approaches
- Develop problem solving agents
- Perform logical and probabilistic reasoning

UNIT I INTELLIGENT AGENTS 9

Introduction to AI – Agents and Environments – concept of rationality – nature of environments – structure of agents. Problem solving agents – search algorithms – uninformed search strategies.

UNIT II PROBLEM SOLVING 9

Heuristic search strategies – heuristic functions. Local search and optimization problems – local search in continuous space – search with non-deterministic actions – search in partially observable environments – online search agents and unknown environments

UNIT III GAME PLAYING AND CSP 9

Game theory – optimal decisions in games – alpha-beta search – monte-carlo tree search – stochastic games – partially observable games. Constraint satisfaction problems – constraint propagation – backtracking search for CSP – local search for CSP – structure of CSP.

UNIT IV LOGICAL REASONING 9

Knowledge-based agents – propositional logic – propositional theorem proving – propositional model checking – agents based on propositional logic. First-order logic – syntax and semantics – knowledge representation and engineering – inferences in first-order logic – forward chaining – backward chaining – resolution.

UNIT V PROBABILISTIC REASONING 9

Acting under uncertainty – Bayesian inference – naïve Bayes models. Probabilistic reasoning – Bayesian networks – exact inference in BN – approximate inference in BN – causal networks.

COURSE OUTCOMES:

At the end of this course, the students will be able to:

CO1: Explain intelligent agent frameworks
CO2: Apply problem solving techniques
CO3: Apply game playing and CSP techniques
CO4: Perform logical reasoning
CO5: Perform probabilistic reasoning under uncertainty

TOTAL:45 PERIODS

TEXT BOOKS:

1. Stuart Russell and Peter Norvig, “Artificial Intelligence – A Modern Approach”, Fourth Edition, Pearson Education, 2021.

REFERENCES

1. Dan W. Patterson, “Introduction to AI and ES”, Pearson Education,2007
2. Kevin Night, Elaine Rich, and Nair B., “Artificial Intelligence”, McGraw Hill, 2008
3. Patrick H. Winston, "Artificial Intelligence", Third Edition, Pearson Education, 2006
4. Deepak Khemani, “Artificial Intelligence”, Tata McGraw Hill Education, 2013.
5. http://nptel.ac.in/

UNIT I

INTELLIGENT AGENTS

SYLLABUS

Introduction to AI – Agents and Environments – concept of rationality – nature of environments – structure of agents. Problem solving agents – search algorithms – uninformed search strategies.

INTRODUCTION

- ✓ Artificial intelligence (AI) is the ability of machines to replicate or enhance human intellect, such as reasoning and learning from experience.
- ✓ Artificial intelligence has been used in computer programs for years, but it is now applied to many other products and services.
- ✓ For example, some digital cameras can determine what objects are present in an image using artificial intelligence software.
- ✓ In addition, experts predict many more innovative uses for artificial intelligence in the future, including smart electric grids.
- ✓ AI uses techniques from probability theory, economics, and algorithm design to solve practical problems.
- ✓ In addition, the AI field draws upon computer science, mathematics, psychology, and linguistics.
- ✓ Computer science provides tools for designing and building algorithms, while mathematics offers tools for modeling and solving the resulting optimization problems.
- ✓ To understand the idea behind AI, you should think about what distinguishes human intelligence from that of other creatures – our ability to learn from experiences and apply these lessons to new situations.
- ✓ We can do this because of our advanced brainpower; we have more neurons than any animal species.
- ✓ Research in the field is concerned with producing machines to automate tasks requiring intelligent behavior.
- ✓ Examples include control, planning and scheduling, the ability to answer diagnostic and consumer questions, handwriting, natural language processing and perception, speech recognition, and the ability to move and manipulate objects.

Topic: 1 :	ARTIFICIAL INTELLIGENCE

Definition

"It is a branch of computer science by which we can create intelligent machines which can behave like a human, think like humans, and able to make decisions."

Why Artificial Intelligence?

Before Learning about Artificial Intelligence, we should know that what is the importance of AI and why should we learn it. Following are some main reasons to learn about AI:

- With the help of AI, you can create such software or devices which can solve real-world problems very easily and with accuracy such as health issues, marketing, traffic issues, etc.
- With the help of AI, you can create your personal virtual Assistant, such as Cortana, Google Assistant, Siri, etc.
- With the help of AI, you can build such Robots which can work in an environment where survival of humans can be at risk.
- AI opens a path for other new technologies, new devices, and new Opportunities.

Goals of Artificial Intelligence

Following are the main goals of Artificial Intelligence:

1. Replicate human intelligence
2. Solve Knowledge-intensive tasks
3. An intelligent connection of perception and action
4. Building a machine which can perform tasks that requires human intelligence such as:
    - Proving a theorem
    - Playing chess
    - Plan some surgical operation
    - Driving a car in traffic
5. Creating some system which can exhibit intelligent behavior, learn new things by itself, demonstrate, explain, and can advise to its user.

Subfields of Artificial Intelligence

Here, are some important subfields of Artificial Intelligence:

Machine Learning: Machine learning is the art of studying algorithms that learn from examples and experiences. Machine learning is based on the idea that some patterns in the data were identified and used for future predictions. The difference from hardcoding rules is that the machine learns to find such rules.

Deep Learning: Deep learning is a sub-field of machine learning. Deep learning does not mean the machine learns more in-depth knowledge; it uses different layers to learn from the data. The depth of the model is represented by the number of layers in the model. For instance, the Google LeNet model for image recognition counts 22 layers.

Natural Language Processing: A neural network is a group of connected I/O units where each connection has a weight associated with its computer programs. It helps you to build predictive models from large databases. This model builds upon the human nervous system. You can use this model to conduct image understanding, human learning, computer speech, etc.

Expert Systems: An expert system is an interactive and reliable computer-based decision-making system that uses facts and heuristics to solve complex decision-making problems. It is also considered at the highest level of human intelligence. The main goal of an expert system is to solve the most complex issues in a specific domain.

Fuzzy Logic: Fuzzy Logic is defined as a many-valued logic form that may have truth values of variables in any real number between 0 and 1. It is the handle concept of partial truth. In real life, we may encounter a situation where we can't decide whether the statement is true or false.

Advantages of Artificial Intelligence

Following are some main advantages of Artificial Intelligence:

- **High Accuracy with less errors:** AI machines or systems are prone to less errors and high accuracy as it takes decisions as per pre-experience or information.
- **High-Speed:** AI systems can be of very high-speed and fast-decision making, because of that AI systems can beat a chess champion in the Chess game.
- **High reliability:** AI machines are highly reliable and can perform the same action multiple times with high accuracy.
- **Useful for risky areas:** AI machines can be helpful in situations such as defusing a bomb, exploring the ocean floor, where to employ a human can be risky.
- **Digital Assistant:** AI can be very useful to provide digital assistant to the users such as AI technology is currently used by various E-commerce websites to show the products as per customer requirement.
- **Useful as a public utility:** AI can be very useful for public utilities such as a self-driving car which can make our journey safer and hassle-free, facial recognition for security purpose, Natural language processing to communicate with the human in human-language, etc.

Disadvantages of Artificial Intelligence

Following are the disadvantages of AI:

- **High Cost:** The hardware and software requirement of AI is very costly as it requires lots of maintenance to meet current world requirements.
- **Can't think out of the box:** Even we are making smarter machines with AI, but still they cannot work out of the box, as the robot will only do that work for which they are trained, or programmed.
- **No feelings and emotions:** AI machines can be an outstanding performer, but still it does not have the feeling so it cannot make any kind of emotional attachment with human, and may sometime be harmful for users if the proper care is not taken.
- **Increase dependency on machines:** With the increment of technology, people are getting more dependent on devices and hence they are losing their mental capabilities.

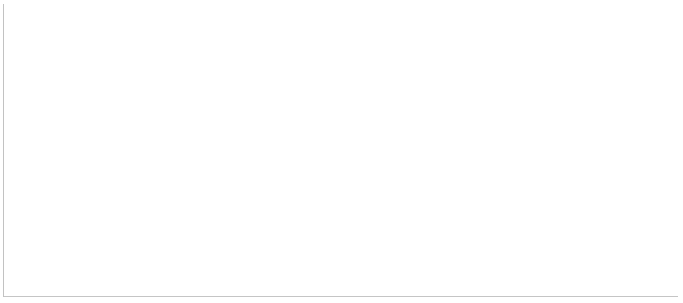
- **No Original Creativity:** As humans are so creative and can imagine some new ideas but still AI machines cannot beat this power of human intelligence and cannot be creative and imaginative.

Topic: 2 :	AGENTS AND ENVIRONMENTS

What are Agent and Environment?

An agent is anything that can perceive its environment through sensors and acts upon that environment through effectors.

- A human agent has sensory organs such as eyes, ears, nose, tongue and skin parallel to the sensors, and other organs such as hands, legs, mouth, for effectors.
- A robotic agent replaces cameras and infrared range finders for the sensors, and various motors and actuators for effectors.
- A software agent has encoded bit strings as its programs and actions.



Agent Terminology

- **Performance Measure of Agent** − It is the criteria, which determines how successful an agent is.
- **Behavior of Agent** − It is the action that agent performs after any given sequence of percepts.
- **Percept** − It is agent's perceptual inputs at a given instance.
- **Percept Sequence** − It is the history of all that an agent has perceived till date.
- **Agent Function** − It is a map from the precept sequence to an action.

Rationality

- ✓ Rationality is nothing but status of being reasonable, sensible, and having good sense of judgment.
- ✓ Rationality is concerned with expected actions and results depending upon what the agent has perceived.
- ✓ Performing actions with the aim of obtaining useful information is an important part of rationality.

What is Ideal Rational Agent?

An ideal rational agent is the one, which is capable of doing expected actions to maximize its performance measure, on the basis of

- Its percept sequence
- Its built-in knowledge base

Rationality of an agent depends on the following

- The **performance measures**, which determine the degree of success.
- Agent's **Percept Sequence** till now.
- The agent's **prior knowledge about the environment**.
- The **actions** that the agent can carry out.

A rational agent always performs right action, where the right action means the action that causes the agent to be most successful in the given percept sequence. The problem the agent solves is characterized by Performance Measure, Environment, Actuators, and Sensors (PEAS).

The Structure of Intelligent Agents

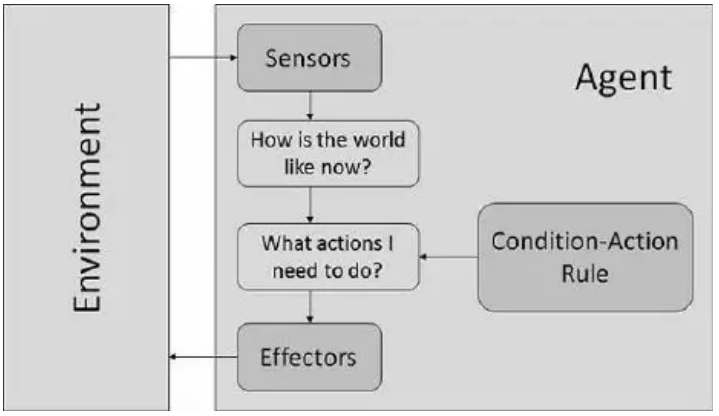
Agent's structure can be viewed as −

- Agent = Architecture + Agent Program
- Architecture = the machinery that an agent executes on.
- Agent Program = an implementation of an agent function.

Simple Reflex Agents

- They choose actions only based on the current percept.
- They are rational only if a correct decision is made only on the basis of current precept.
- Their environment is completely observable.

Condition-Action Rule − It is a rule that maps a state (condition) to an action.



Agent Environment in AI

- ✓ An environment is everything in the world which surrounds the agent, but it is not a part of an agent itself.
- ✓ An environment can be described as a situation in which an agent is present.
- ✓ The environment is where agent lives, operate and provide the agent with something to sense and act upon it. An environment is mostly said to be non-feministic.

Features of Environment

- ✓ As per Russell and Norvig, an environment can have various features from the point of view of an agent:
  1. Fully observable vs Partially Observable
  2. Static vs Dynamic
  3. Discrete vs Continuous
  4. Deterministic vs Stochastic
  5. Single-agent vs Multi-agent
  6. Episodic vs sequential
  7. Known vs Unknown
  8. Accessible vs Inaccessible

1. Fully observable vs Partially Observable:
   - If an agent sensor can sense or access the complete state of an environment at each point of time then it is **a fully observable** environment, else it is **partially observable**.
   - A fully observable environment is easy as there is no need to maintain the internal state to keep track history of the world.
   - An agent with no sensors in all environments then such an environment is called as **unobservable**.
2. Deterministic vs Stochastic:
   - If an agent's current state and selected action can completely determine the next state of the environment, then such environment is called a deterministic environment.
   - A stochastic environment is random in nature and cannot be determined completely by an agent.
   - In a deterministic, fully observable environment, agent does not need to worry about uncertainty.
3. Episodic vs Sequential:
   - In an episodic environment, there is a series of one-shot actions, and only the current percept is required for the action.
   - However, in Sequential environment, an agent requires memory of past actions to determine the next best actions.
4. Single-agent vs Multi-agent
   - If only one agent is involved in an environment, and operating by itself then such an environment is called single agent environment.
   - However, if multiple agents are operating in an environment, then such an environment is called a multi-agent environment.
   - The agent design problems in the multi-agent environment are different from single agent environment.
5. Static vs Dynamic:
   - If the environment can change itself while an agent is deliberating then such environment is called a dynamic environment else it is called a static environment.
   - Static environments are easy to deal because an agent does not need to continue looking at the world while deciding for an action.
   - However for dynamic environment, agents need to keep looking at the world at each action.

- Taxi driving is an example of a dynamic environment whereas Crossword puzzles are an example of a static environment.

6. Discrete vs Continuous:
   - If in an environment there are a finite number of percepts and actions that can be performed within it, then such an environment is called a discrete environment else it is called continuous environment.
   - A chess gamecomes under discrete environment as there is a finite number of moves that can be performed.
   - A self-driving car is an example of a continuous environment.
7. Known vs Unknown
   - Known and unknown are not actually a feature of an environment, but it is an agent's state of knowledge to perform an action.
   - In a known environment, the results for all actions are known to the agent. While in unknown environment, agent needs to learn how it works in order to perform an action.
   - It is quite possible that a known environment to be partially observable and an Unknown environment to be fully observable.
8. Accessible vs Inaccessible
   - If an agent can obtain complete and accurate information about the state's environment, then such an environment is called an Accessible environment else it is called inaccessible.
   - An empty room whose state can be defined by its temperature is an example of an accessible environment.
   - Information about an event on earth is an example of Inaccessible environment.

Topic: 3 : CONCEPT OF RATIONALITY

Rational Agent:

- ✓ A rational agent is an agent which has clear preference, models uncertainty, and acts in a way to maximize its performance measure with all possible actions.
- ✓ A rational agent is said to perform the right things. AI is about creating rational agents to use for game theory and decision theory for various real-world scenarios.

✓ For an AI agent, the rational action is most important because in AI reinforcement learning algorithm, for each best possible action, agent gets the positive reward and for each wrong action, an agent gets a negative reward.

Rationality:

✓ The rationality of an agent is measured by its performance measure.

✓ Rationality can be judged on the basis of following points:

- Performance measure which defines the success criterion.
- Agent prior knowledge of its environment.
- Best possible actions that an agent can perform.
- The sequence of percepts.

PEAS Representation

✓ PEAS is a type of model on which an AI agent works upon.

✓ When we define an AI agent or rational agent, then we can group its properties under PEAS representation model.

✓ It is made up of four words:

- **P:** Performance measure
- **E:** Environment
- **A:** Actuators
- **S:** Sensors

✓ Here performance measure is the objective for the success of an agent's behavior.

PEAS for self-driving cars:

✓ Let's suppose a self-driving car then PEAS representation will be:

Performance: Safety, time, legal drive, comfort

Environment: Roads, other vehicles, road signs, pedestrian

Actuators : Steering, accelerator, brake, signal, horn

Sensors : Camera, GPS, speedometer, odometer, accelerometer, sonar.

✓ Figure 2.4 summarizes the PEAS description for the taxi's task environment.

✓ We discuss each element in more detail in the following paragraphs.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

Figure 2.4 PEAS description of the task environment for an automated taxi.

Properties of task environments

- ✓ The range of task environments that might arise in AI is obviously vast.
- ✓ We can, however, identify a fairly small number of dimensions along which task environments can be categorized.
- ✓ These dimensions determine, to a large extent, the appropriate agent design and the applicability of each of the principal families of techniques for agent implementation. First,

Topic: 4 :	NATURE OF ENVIRONMENTS

The Nature of Environments

- ✓ Some programs operate in the entirely artificial environment confined to keyboard input, database, computer file systems and character output on a screen.
- ✓ In contrast, some software agents (software robots or softbots) exist in rich, unlimited softbots domains.
- ✓ The simulator has a very detailed, complex environment. The software agent needs to choose from a long array of actions in real time.
- ✓ A softbot designed to scan the online preferences of the customer and show interesting items to the customer works in the real as well as an artificial environment.
- ✓ The most famous artificial environment is the Turing Test environment, in which one real and other artificial agents are tested on equal ground.
- ✓ This is a very challenging environment as it is highly difficult for a software agent to perform as well as a human.

Turing Test

- ✓ The success of an intelligent behavior of a system can be measured with Turing Test.
- ✓ Two persons and a machine to be evaluated participate in the test.
- ✓ Out of the two persons, one plays the role of the tester.
- ✓ Each of them sits in different rooms.
- ✓ The tester is unaware of who is machine and who is a human.
- ✓ He interrogates the questions by typing and sending them to both intelligences, to which he receives typed responses.
- ✓ This test aims at fooling the tester. If the tester fails to determine machine's response from the human response, then the machine is said to be intelligent.

Properties of Environment

- ✓ The environment has multifold properties −
  - **Discrete / Continuous** − If there are a limited number of distinct, clearly defined, states of the environment, the environment is discrete (For example, chess); otherwise it is continuous (For example, driving).

- **Observable / Partially Observable** − If it is possible to determine the complete state of the environment at each time point from the percepts it is observable; otherwise it is only partially observable.
- **Static / Dynamic** − If the environment does not change while an agent is acting, then it is static; otherwise it is dynamic.
- **Single agent / Multiple agents** − The environment may contain other agents which may be of the same or different kind as that of the agent.
- **Accessible / Inaccessible** − If the agent's sensory apparatus can have access to the complete state of the environment, then the environment is accessible to that agent.
- **Deterministic / Non-deterministic** − If the next state of the environment is completely determined by the current state and the actions of the agent, then the environment is deterministic; otherwise it is non-deterministic.
- **Episodic / Non-episodic** − In an episodic environment, each episode consists of the agent perceiving and then acting. The quality of its action depends just on the episode itself. Subsequent episodes do not depend on the actions in the previous episodes. Episodic environments are much simpler because the agent does not need to think ahead.

Topic: 5 : STRUCTURE OF AGENTS

The Structure of Intelligent Agents

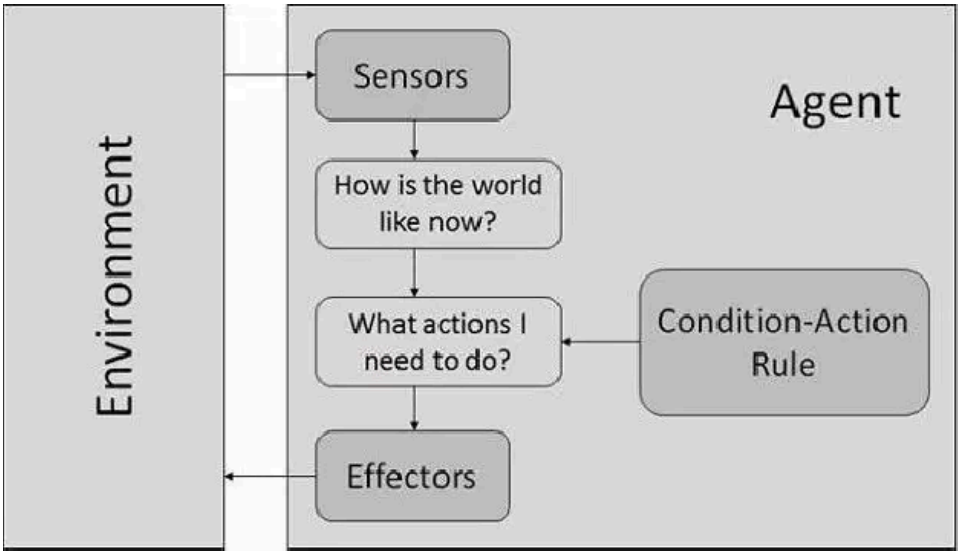
Agent's structure can be viewed as −

- Agent = Architecture + Agent Program
- Architecture = the machinery that an agent executes on.
- Agent Program = an implementation of an agent function.

Simple Reflex Agents

- They choose actions only based on the current percept.
- They are rational only if a correct decision is made only on the basis of current precept.
- Their environment is completely observable.

Condition-Action Rule − It is a rule that maps a state (condition) to an action.



Model Based Reflex Agents

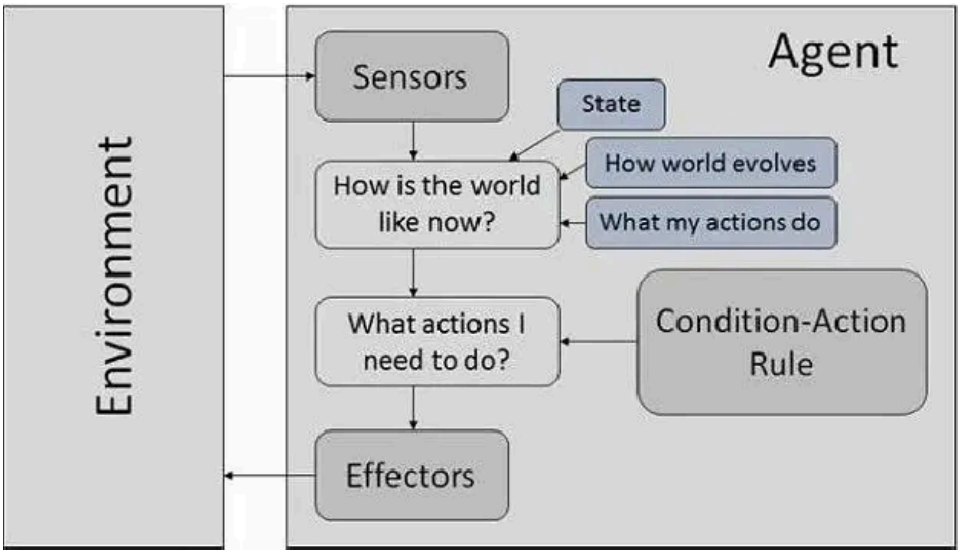
✓ They use a model of the world to choose their actions. They maintain an internal state.

Model − knowledge about "how the things happen in the world".

Internal State − It is a representation of unobserved aspects of current state depending on percept history.

Updating the state requires the information about −

- How the world evolves.
- How the agent's actions affect the world.



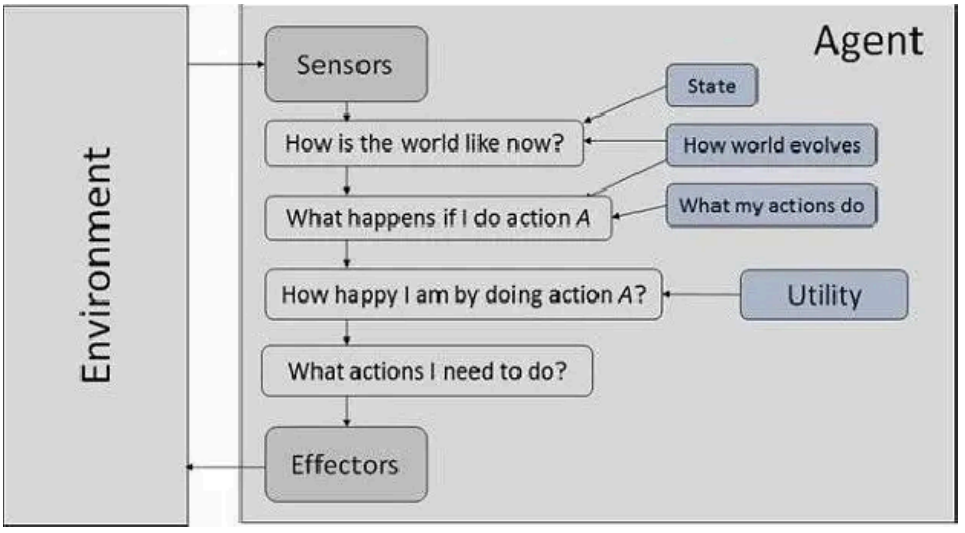
Goal Based Agents

✓ They choose their actions in order to achieve goals. Goal-based approach is more flexible than reflex agent since the knowledge supporting a decision is explicitly modeled, thereby allowing for modifications.

Goal − It is the description of desirable situations.

Utility Based Agents

- ✓ They choose actions based on a preference (utility) for each state.
- ✓ Goals are inadequate when −
  - There are conflicting goals, out of which only few can be achieved.
  - Goals have some uncertainty of being achieved and you need to weigh likelihood of success against the importance of a goal.



Topic: 6 : PROBLEM SOLVING AGENTS

Problem solving agents – search algorithms

- ✓ Search algorithms are one of the most important areas of Artificial Intelligence.
- ✓ This topic will explain all about the search algorithms in AI.

Problem-solving agents:

- ✓ In Artificial Intelligence, Search techniques are universal problem-solving methods.
- ✓ **Rational agents** or **Problem-solving agents** in AI mostly used these search strategies or algorithms to solve a specific problem and provide the best result.
- ✓ Problem-solving agents are the goal-based agents and use atomic representation. In this topic, we will learn various problem-solving search algorithms.

Search Algorithm Terminologies:

- **Search:** Searchingis a step by step procedure to solve a search-problem in a given search space. A search problem can have three main factors:
    a. **Search Space:** Search space represents a set of possible solutions, which a system may have.
    b. **Start State:** It is a state from where agent begins **the search**.
    c. **Goal test:** It is a function which observe the current state and returns whether the goal state is achieved or not.
- **Search tree:** A tree representation of search problem is called Search tree. The root of the search tree is the root node which is corresponding to the initial state.
- **Actions:** It gives the description of all the available actions to the agent.
- **Transition model:** A description of what each action do, can be represented as a transition model.
- **Path Cost:** It is a function which assigns a numeric cost to each path.
- **Solution:** It is an action sequence which leads from the start node to the goal node.
- **Optimal Solution:** If a solution has the lowest cost among all solutions.

Properties of Search Algorithms:

Following are the four essential properties of search algorithms to compare the efficiency of these algorithms:

Completeness: A search algorithm is said to be complete if it guarantees to return a solution if at least any solution exists for any random input.

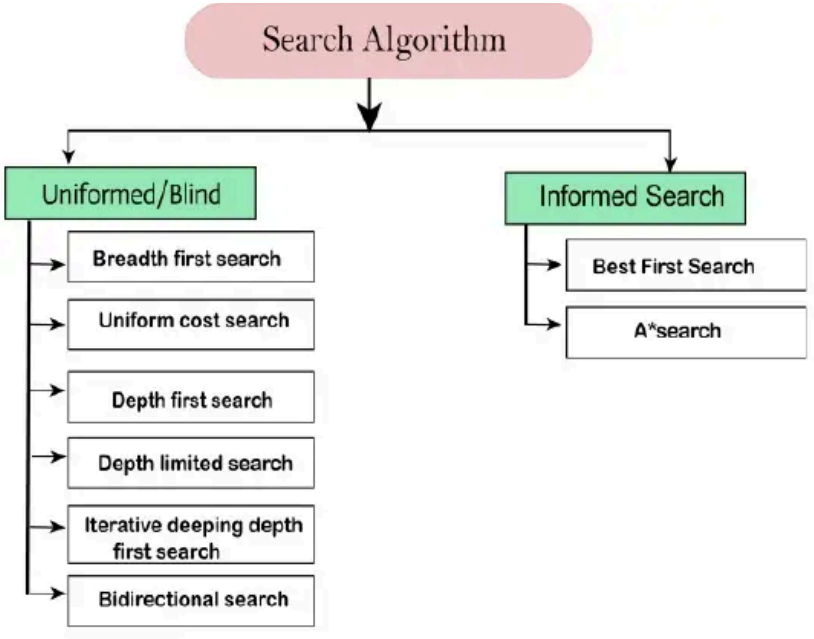
Optimality: If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution for is said to be an optimal solution.

Time Complexity: Time complexity is a measure of time for an algorithm to complete its task.

Space Complexity: It is the maximum storage space required at any point during the search, as the complexity of the problem.

Types of search algorithms

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



Topic: 7 :	UNINFORMED SEARCH STRATEGIES

Uninformed/Blind Search:

- ✓ The uninformed search does not contain any domain knowledge such as closeness, the location of the goal.

- ✓ It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes.
- ✓ Uninformed search applies a way in which search tree is searched without any information about the search space like initial state operators and test for the goal, so it is also called blind search.
- ✓ It examines each node of the tree until it achieves the goal node.

It can be divided into five main types:

- o Breadth-first search
- o Uniform cost search
- o Depth-first search
- o Iterative deepening depth-first search
- o Bidirectional Search

Informed Search

- ✓ Informed search algorithms use domain knowledge.
- ✓ In an informed search, problem information is available which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy.
- ✓ Informed search is also called a Heuristic search.
- ✓ A heuristic is a way which might not always be guaranteed for best solutions but guaranteed to find a good solution in reasonable time.
- ✓ Informed search can solve much complex problem which could not be solved in another way.
- ✓ An example of informed search algorithms is a traveling salesman problem.
    1. Greedy Search
    2. A* Search

Breadth-first Search:

- o Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.
- o BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- o The breadth-first search algorithm is an example of a general-graph search algorithm.

- Breadth-first search implemented using FIFO queue data structure.

Advantages:

- BFS will provide a solution if any solution exists.
- If there are more than one solutions for a given problem, then BFS will provide the minimal solution which requires the least number of steps.

Disadvantages:

- It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
- BFS needs lots of time if the solution is far away from the root node.

Example:

✓ In the below tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K.

✓ BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

2. Depth-first Search

✓ Depth-first search isa recursive algorithm for traversing a tree or graph data structure.

✓ It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.

✓ DFS uses a stack data structure for its implementation.

✓ The process of the DFS algorithm is similar to the BFS algorithm.

Advantage:

- DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
- It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).

Disadvantage:

- There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
- DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

Example:

✓ In the below search tree, we have shown the flow of depth-first search, and it will follow the order as:

✓ Root node--->Left node --- > right node.

✓ It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found.

✓ After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.

3. Depth-Limited Search Algorithm:

- ✓ A depth-limited search algorithm is similar to depth-first search with a predetermined limit.
- ✓ Depth-limited search can solve the drawback of the infinite path in the Depth-first search. In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

Depth-limited search can be terminated with two Conditions of failure:

- ✓ Standard failure value: It indicates that problem does not have any solution.
- ✓ Cutoff failure value: It defines no solution for the problem within a given depth limit.

Advantages:

Depth-limited search is Memory efficient.

Disadvantages:

- Depth-limited search also has a disadvantage of incompleteness.
- It may not be optimal if the problem has more than one solution.

Example:

Bidirectional Search

- ✓ It searches forward from initial state and backward from goal state till both meet to identify a common state.
- ✓ The path from initial state is concatenated with the inverse path from the goal state. Each search is done only up to half of the total path.

Uniform Cost Search

- ✓ Sorting is done in increasing cost of the path to a node. It always expands the least cost node. It is identical to Breadth First search if each transition has the same cost.
- ✓ It explores paths in the increasing order of cost.

Disadvantage

- There can be multiple long paths with the cost ≤ C*. Uniform Cost search must explore them all.
- Iterative Deepening Depth-First Search
- It performs depth-first search to level 1, starts over, executes a complete depth-first search to level 2, and continues in such way till the solution is found.

UNIT II
PROBLEM SOLVING

SYLLABUS

Heuristic search strategies – heuristic functions. Local search and optimization problems – local search in continuous space – search with non-deterministic actions – search in partially observable environments – online search agents and unknown environments

INTRODUCTION

- ✓ The reflex agent of AI directly maps states into action.
- ✓ Whenever these agents fail to operate in an environment where the state of mapping is too large and not easily performed by the agent, then the stated problem dissolves and sent to a problem-solving domain which breaks the large stored problem into the smaller storage area and resolves one by one.
- ✓ The final integrated action will be the desired outcomes.
- ✓ On the basis of the problem and their working domain, different types of problem-solving agent defined and use at an atomic level without any internal state visible with a problem-solving algorithm.
- ✓ The problem-solving agent performs precisely by defining problems and several solutions.
- ✓ So we can say that problem solving is a part of artificial intelligence that encompasses a number of techniques such as a tree, B-tree, heuristic algorithms to solve a problem.
- ✓ We can also say that a problem-solving agent is a result-driven agent and always focuses on satisfying the goals.

These are the following steps which require to solve a problem :

- **Goal Formulation:** This one is the first and simple step in problem-solving. It organizes finite steps to formulate a target/goals which require some action to achieve the goal. Today the formulation of the goal is based on AI agents.

- **Problem formulation:** It is one of the core steps of problem-solving which decides what action should be taken to achieve the formulated goal. In AI this core part is dependent upon software agent which consisted of the following components to formulate the associated problem.

Components to formulate the associated problem:

- **Initial State:** This state requires an initial state for the problem which starts the AI agent towards a specified goal. In this state new methods also initialize problem domain solving by a specific class.

- **Action:** This stage of problem formulation works with function with a specific class taken from the initial state and all possible actions done in this stage

- **Transition:** This stage of problem formulation integrates the actual action done by the previous action stage and collects the final stage to forward it to their next stage.

- **Goal test:** This stage determines that the specified goal achieved by the integrated transition model or not, whenever the goal achieves stop the action and forward into the next stage to determines the cost to achieve the goal.

- **Path costing:** This component of problem-solving numerical assigned what will be the cost to achieve the goal. It requires all hardware software and human working cost.

Topic: 1 : HEURISTIC SEARCH STRATEGIES

- ✓ So far we have talked about the uninformed search algorithms which looked through search space for all possible solutions of the problem without having any additional knowledge about search space.
- ✓ But informed search algorithm contains an array of knowledge such as how far we are from the goal, path cost, how to reach to goal node, etc.
- ✓ This knowledge help agents to explore less to the search space and find more efficiently the goal node.
- ✓ The informed search algorithm is more useful for large search space.
- ✓ Informed search algorithm uses the idea of heuristic, so it is also called Heuristic search.

- ✓ Heuristics function: Heuristic is a function which is used in Informed Search, and it finds the most promising path.
- ✓ It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal.
- ✓ The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time.
- ✓ Heuristic function estimates how close a state is to the goal.
- ✓ It is represented by h(n), and it calculates the cost of an optimal path between the pair of states. The value of the heuristic function is always positive.
- ✓ Admissibility of the heuristic function is given as:

  $h(n) <= h^*(n)$
- ✓ Here h(n) is heuristic cost, and h*(n) is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.

Pure Heuristic Search:

- ✓ Pure heuristic search is the simplest form of heuristic search algorithms.
- ✓ It expands nodes based on their heuristic value h(n). It maintains two lists, OPEN and CLOSED list. In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.
- ✓ On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list.
- ✓ The algorithm continues unit a goal state is found.
- ✓ In the informed search we will discuss two main algorithms which are given below:
  - Best First Search Algorithm(Greedy search)
  - A* Search Algorithm

1.) Best-first Search Algorithm (Greedy Search):

- ✓ Greedy best-first search algorithm always selects the path which appears best at that moment.
- ✓ It is the combination of depth-first search and breadth-first search algorithms.
- ✓ It uses the heuristic function and search.
- ✓ Best-first search allows us to take the advantages of both algorithms.
- ✓ With the help of best-first search, at each step, we can choose the most promising node. In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e.

f(n)= g(n).

Where, h(n)= estimated cost from node n to the goal.

The greedy best first algorithm is implemented by the priority queue.

Best first search algorithm:

- Step 1: Place the starting node into the OPEN list.
- Step 2: If the OPEN list is empty, Stop and return failure.
- Step 3: Remove the node n, from the OPEN list which has the lowest value of h(n), and places it in the CLOSED list.
- Step 4: Expand the node n, and generate the successors of node n.
- Step 5: Check each successor of node n, and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
- Step 6: For each successor node, algorithm checks for evaluation function f(n), and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
- Step 7: Return to Step 2.

Advantages:

- Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
- This algorithm is more efficient than BFS and DFS algorithms.

Disadvantages:

- It can behave as an unguided depth-first search in the worst case scenario.
- It can get stuck in a loop as DFS.
- This algorithm is not optimal.

Example:

Consider the below search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function f(n)=h(n) , which is given in the below table.

In this search example, we are using two lists which are OPEN and CLOSED Lists. Following are the iteration for traversing the above example.

Iteration 3 : Open [I, G, E, A], Closed [S, B, F]

: Open [I, E, A], Closed [S, B, F, G]

Hence the final solution path will be: S----> B-----> F ---> G

Expand the nodes of S and put in the CLOSED list

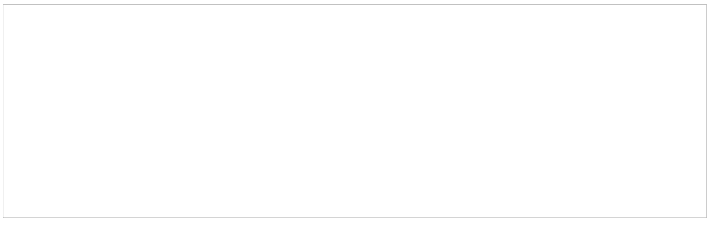
Initialization: Open [A, B], Closed [S]

Iteration 1: Open [A], Closed [S, B]

Iteration 2: Open [E, F, A], Closed [S, B]
: Open [E, A], Closed [S, B, F]

2.) A* Search Algorithm:

- ✓ A* search is the most commonly known form of best-first search.
- ✓ It uses heuristic function h(n), and cost to reach the node n from the start state g(n).
- ✓ It has combined features of UCS and greedy best-first search, by which it solve the problem efficiently.
- ✓ A* search algorithm finds the shortest path through the search space using the heuristic function.
- ✓ This search algorithm expands less search tree and provides optimal result faster. A* algorithm is similar to UCS except that it uses g(n)+h(n) instead of g(n).
- ✓ In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence we can combine both costs as following, and this sum is called as a fitness number.



Algorithm of A* search:

Step1: Place the starting node in the OPEN list.

Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

Step 3: Select the node from the OPEN list which has the smallest value of evaluation function (g+h), if node n is goal node then return success and stop, otherwise

Step 4: Expand node n and generate all of its successors, and put n into the closed list. For each successor n', check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.

Step 5: Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest g(n') value.

Step 6: Return to Step 2.

Advantages:

- A* search algorithm is the best algorithm than other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

Disadvantages:

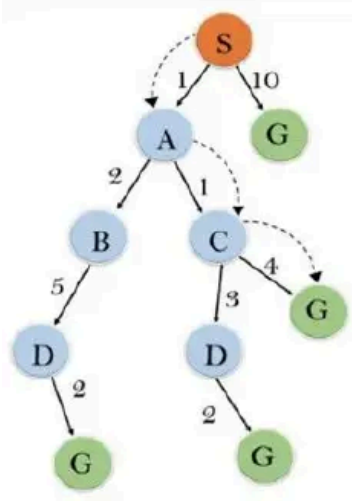
- It does not always produce the shortest path as it mostly based on heuristics and approximation.
- A* search algorithm has some complexity issues.
- The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.

Example:

In this example, we will traverse the given graph using the A* algorithm. The heuristic value of all states is given in the below table so we will calculate the f(n) of each state using the formula f(n)= g(n) + h(n), where g(n) is the cost to reach any node from start state.

Here we will use OPEN and CLOSED list.

Solution:



Initialization: {(S, 5)}

Iteration1: {(S--> A, 4), (S-->G, 10)}

Iteration2: {(S--> A-->C, 4), (S--> A-->B, 7), (S-->G, 10)}

Iteration3: {(S--> A-->C--->G, 6), (S--> A-->C--->D, 11), (S--> A-->B, 7), (S-->G, 10)}

Iteration 4 will give the final result, as S--->A--->C--->G it provides the optimal path with cost 6.

Points to remember:

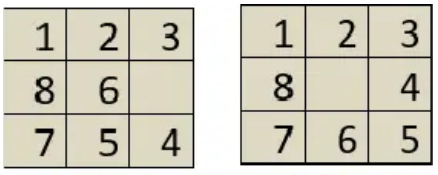
- A* algorithm returns the path which occurred first, and it does not search for all remaining paths.
- The efficiency of A* algorithm depends on the quality of heuristic.
- A* algorithm expands all nodes which satisfy the condition f(n)

Topic: 2 : HEURISTIC FUNCTIONS

Heuristic Functions in AI: As we have already seen that an informed search make use of heuristic functions in order to reach the goal node in a more prominent way. Therefore, there are several pathways in a search tree to reach the goal node from the current node. The selection of a good heuristic function matters certainly. *A good heuristic function is determined by its efficiency. More is the information about the problem, more is the processing time.*

Some toy problems, such as 8-puzzle, 8-queen, tic-tac-toe, etc., can be solved more efficiently with the help of a heuristic function. Let's see how:

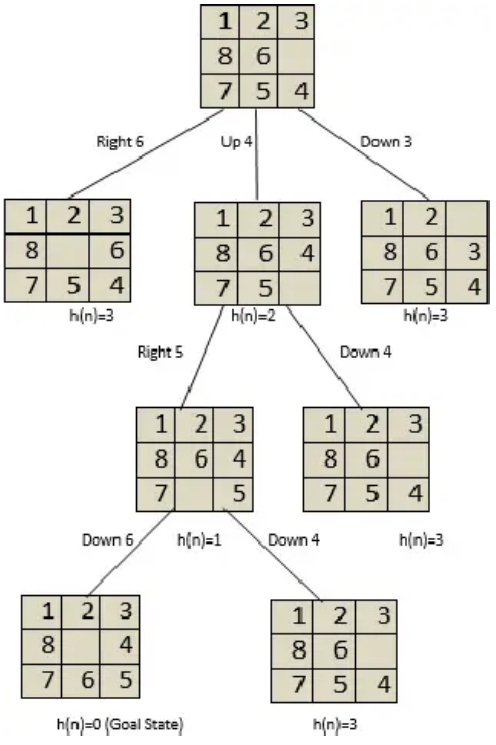
- ✓ Consider the following 8-puzzle problem where we have a start state and a goal state.
- ✓ Our task is to slide the tiles of the current/start state and place it in an order followed in the goal state.
- ✓ There can be four moves either **left, right, up, or down**.
- ✓ There can be several ways to convert the current/start state to the goal state, but, we can use a heuristic function h(n) to solve the problem more efficiently.



A heuristic function for the 8-puzzle problem is defined below:

h(n)=Number of tiles out of position.

- ✓ So, there is total of three tiles out of position i.e., 6,5 and 4. Do not count the empty tile present in the goal state). i.e. h(n)=3.
- ✓ Now, we require to minimize the value of **h(n) =0.**
- ✓ We can construct a state-space tree to minimize the h(n) value to 0, as shown below:



- ✓ It is seen from the above state space tree that the goal state is minimized from h(n)=3 to h(n)=0.
- ✓ However, we can create and use several heuristic functions as per the requirement.
- ✓ It is also clear from the above example that a heuristic function h(n) can be defined as the information required to solve a given problem more efficiently.
- ✓ The information can be related to the **nature of the state, cost of transforming from one state to another, goal node characterstics,** etc., which is expressed as a heuristic function.

Properties of a Heuristic search Algorithm

Use of heuristic function in a heuristic search algorithm leads to following properties of a heuristic search algorithm:

- **Admissible Condition:** An algorithm is said to be admissible, if it returns an optimal solution.
- **Completeness:** An algorithm is said to be complete, if it terminates with a solution (if the solution exists).

- **Dominance Property:** If there are two admissible heuristic algorithms **A1** and **A2** having **h1** and **h2** heuristic functions, then **A1** is said to dominate **A2** if **h1** is better than **h2** for all the values of node **n.**
- **Optimality Property:** If an algorithm is **complete, admissible**, and **dominating** other algorithms, it will be the best one and will definitely give an optimal solution.

Topic: 3 :	LOCAL SEARCH AND OPTIMIZATION PROBLEMS

- ✓ The search algorithms that we have seen so far are designed to explore search spaces systematically.
- ✓ This systematicity is achieved by keeping one or more paths in memory and by recording which alternatives have been explored at each point along the path.
- ✓ When a goal is found, the *path* to that goal also constitutes a *solution* to the problem. In many problems, however, the path to the goal is irrelevant. For example, in the 8-queens problem.
- ✓ what matters is the final configuration of queens, not the order in which they are added. The same general property holds for many important applications such as integrated-circuit design, factory-floor layout, job-shop scheduling, automatic programming, telecommunications network optimization, vehicle routing, and portfolio management.

- ✓ If the path to the goal does not matter, we might consider a different class of algorithms, ones that do not worry about paths at all.
- ✓ **Local search** algorithms operate using a single **current node** (rather than multiple paths) and generally move only to neighbors of that node.
- ✓ Typically, the paths followed by the search are not retained. Although local search algorithms are not systematic, they have two key advantages:

  (1) they use very little memory—usually a constant amount; and

  (2) they can often find reasonable solutions in large or infinite (continuous) state spaces for which systematic algorithms are unsuitable.

Below are some different types of local searches:

- Hill-climbing Search
- Simulated Annealing
- Local Beam Search

Hill Climbing Algorithm

- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem.
- It terminates when it reaches a peak value where no neighbor has a higher value.
- Hill climbing algorithm is a technique which is used for optimizing the mathematical problems. One of the widely discussed examples of Hill climbing algorithm is Traveling-salesman Problem in which we need to minimize the distance traveled by the salesman.
- It is also called greedy local search as it only looks to its good immediate neighbor state and not beyond that.
- A node of hill climbing algorithm has two components which are state and value.
- Hill Climbing is mostly used when a good heuristic is available.
- In this algorithm, we don't need to maintain and handle the search tree or graph as it only keeps a single current state.

Features of Hill Climbing:

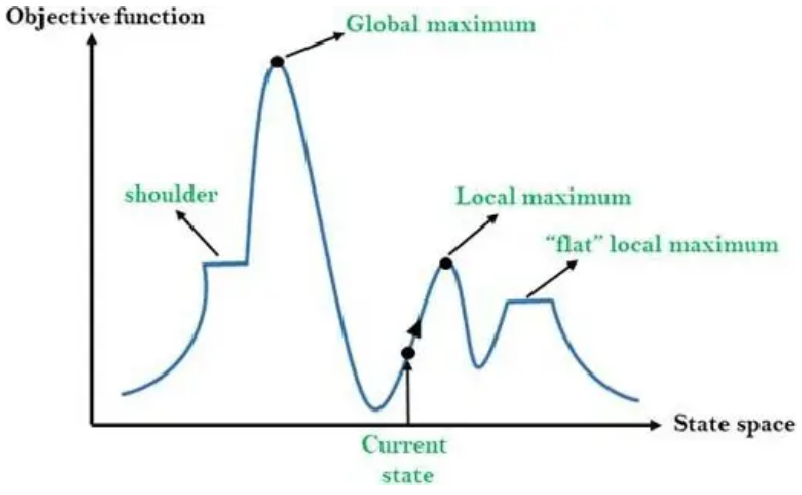
Following are some main features of Hill Climbing Algorithm:

- **Generate and Test variant:** Hill Climbing is the variant of Generate and Test method. The Generate and Test method produce feedback which helps to decide which direction to move in the search space.
- **Greedy approach:** Hill-climbing algorithm search moves in the direction which optimizes the cost.
- **No backtracking:** It does not backtrack the search space, as it does not remember the previous states.

State-space Diagram for Hill Climbing:

The state-space landscape is a graphical representation of the hill-climbing algorithm which is showing a graph between various states of algorithm and Objective function/Cost.

On Y-axis we have taken the function which can be an objective function or cost function, and state-space on the x-axis. If the function on Y-axis is cost then, the goal of search is to find the global minimum and local minimum. If the function of Y-axis is Objective function, then the goal of the search is to find the global maximum and local maximum.



Different regions in the state space landscape:

Local Maximum: Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.

Global Maximum: Global maximum is the best possible state of state space landscape. It has the highest value of objective function.

Current state: It is a state in a landscape diagram where an agent is currently present.

Flat local maximum: It is a flat space in the landscape where all the neighbor states of current states have the same value.

Shoulder: It is a plateau region which has an uphill edge.

Types of Hill Climbing Algorithm:

- Simple hill Climbing:
- Steepest-Ascent hill-climbing:
- Stochastic hill Climbing:

1. Simple Hill Climbing:

Simple hill climbing is the simplest way to implement a hill climbing algorithm. **It only evaluates the neighbor node state at a time and selects the first one which optimizes current cost and set it as a current state**. It only checks it's one successor state, and if it finds better than the current state, then move else be in the same state. This algorithm has the following features:

- Less time consuming
- Less optimal solution and the solution is not guaranteed

Algorithm for Simple Hill Climbing:

- **Step 1:** Evaluate the initial state, if it is goal state then return success and Stop.
- **Step 2:** Loop Until a solution is found or there is no new operator left to apply.
- **Step 3:** Select and apply an operator to the current state.
- **Step 4:** Check new state:
    a. If it is goal state, then return success and quit.
    b. Else if it is better than the current state then assign new state as a current state.
    c. Else if not better than the current state, then return to step2.
- **Step 5:** Exit.

2. Steepest-Ascent hill climbing:

The steepest-Ascent algorithm is a variation of simple hill climbing algorithm. This algorithm examines all the neighboring nodes of the current state and selects one neighbor node which is closest to the goal state. This algorithm consumes more time as it searches for multiple neighbors

Algorithm for Steepest-Ascent hill climbing:

- **Step 1:** Evaluate the initial state, if it is goal state then return success and stop, else make current state as initial state.
- **Step 2:** Loop until a solution is found or the current state does not change.
    a. Let SUCC be a state such that any successor of the current state will be better than it.
    b. For each operator that applies to the current state:
        a. Apply the new operator and generate a new state.
        b. Evaluate the new state.

c. If it is goal state, then return it and quit, else compare it to the SUCC.
d. If it is better than SUCC, then set new state as SUCC.
e. If the SUCC is better than the current state, then set current state to SUCC.

- **Step 5:** Exit.

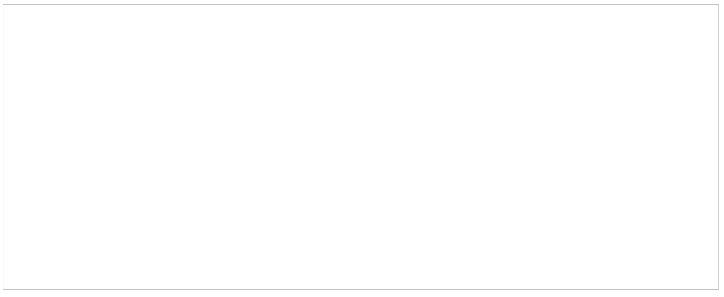
3. Stochastic hill climbing:

Stochastic hill climbing does not examine for all its neighbor before moving. Rather, this search algorithm selects one neighbor node at random and decides whether to choose it as a current state or examine another state.

Problems in Hill Climbing Algorithm:

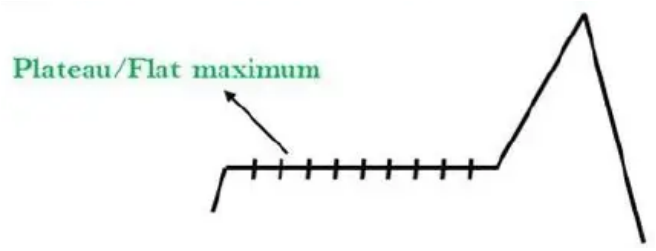
1. Local Maximum: A local maximum is a peak state in the landscape which is better than each of its neighboring states, but there is another state also present which is higher than the local maximum.

Solution: Backtracking technique can be a solution of the local maximum in state space landscape. Create a list of the promising path so that the algorithm can backtrack the search space and explore other paths as well.



2. Plateau: A plateau is the flat area of the search space in which all the neighbor states of the current state contains the same value, because of this algorithm does not find any best direction to move. A hill-climbing search might be lost in the plateau area.

Solution: The solution for the plateau is to take big steps or very little steps while searching, to solve the problem. Randomly select a state which is far away from the current state so it is possible that the algorithm could find non-plateau region.



3. Ridges: A ridge is a special form of the local maximum. It has an area which is higher than its surrounding areas, but itself has a slope, and cannot be reached in a single move.

Solution: With the use of bidirectional search, or by moving in different directions, we can improve this problem.

Simulated Annealing:

A hill-climbing algorithm which never makes a move towards a lower value guaranteed to be incomplete because it can get stuck on a local maximum. And if algorithm applies a random walk, by moving a successor, then it may complete but not efficient. **Simulated Annealing** is an algorithm which yields both efficiency and completeness.

In mechanical term **Annealing** is a process of hardening a metal or glass to a high temperature then cooling gradually, so this allows the metal to reach a low-energy crystalline state. The same process is used in simulated annealing in which the algorithm picks a random move, instead of picking the best move. If the random move improves the state, then it follows the same path. Otherwise, the algorithm follows the

path which has a probability of less than 1 or it moves downhill and chooses another path.

Topic: 4:	LOCAL SEARCH IN CONTINUOUS SPACE

- ✓ In previous sections, we explained the distinction between discrete and continuous environments, pointing out that most real-world environments are continuous.
- ✓ Yet none of the algorithms we have described (except for first-choice hill climbing and simulated annealing) can handle continuous state and action spaces, because they have infinite branching factors.
- ✓ This section provides a *very brief* introduction to some local search techniques for finding optimal solutions in continuous spaces.
- ✓ The literature on this topic is vast; many of the basic techniques originated in the 17th century, after the development of calculus by Newton and Leibniz.

Many methods attempt to use the **gradient** of the landscape to find a maximum. The gradient of the objective function is a vector ∇f that gives the magnitude and direction of the steepest slope. For our problem, we have

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial y_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial y_3} \right) .$$

In some cases, we can find a maximum by solving the equation $\nabla f = 0$. (This could be done, for example, if we were placing just one airport; the solution is the arithmetic mean of all the cities' coordinates.) In many cases, however, this equation cannot be solved in closed form. For example, with three airports, the expression for the gradient depends on what cities are closest to each airport in the current state. This means we can compute the gradient *locally* (but not *globally*); for example,

$$\frac{\partial f}{\partial x_1} = 2 \sum_{c \in C_1} (x_i - x_c) . \tag{4.2}$$

Given a locally correct expression for the gradient, we can perform steepest-ascent hill climb-

Topic: 5:	SEARCHING WITH NONDETERMINISTIC ACTIONS

- ✓ In previous sections, we assumed that the environment is fully observable and deterministic and that the agent knows what the effects of each action are.
- ✓ Therefore, the agent can calculate exactly which state results from any sequence of actions and always knows which state it is in.
- ✓ Its percepts provide no new information after each action, although of course they tell the agent the initial state.
- ✓ When the environment is either partially observable or nondeterministic (or both), percepts become useful.
- ✓ In a partially observable environment, every percept helps narrow down the set of possible states the agent might be in, thus making it easier for the agent to achieve its goals.
- ✓ When the environment is nondeterministic, percepts tell the agent which of the possible outcomes of its actions has actually occurred.
- ✓ In both cases, the future percepts cannot be determined in advance and the agent's future actions will depend on those future percepts.
- ✓ So the solution to a problem is not a sequence but a **contingency plan** (also known as a **strategy**) that specifies what to do depending on what percepts are received. In this section, we examine the case of non determinism, deferring partial observability.

The erratic vacuum world

- ✓ Let us take an example of the vacuum world as a search problem.
- ✓ Recall that the state space has eight states, as shown in Figure 4.9.
- ✓ There are three actions—*Left*, *Right*,and*Suck*—and the goal is to clean up all the dirt (states 7 and 8).
- ✓ If the environment is observable, deterministic, and completely known, then the problem is trivially solvable by any of the algorithms, and the solution is an action sequence.
- ✓ For example, if the initial state is 1, then the action sequence [*Suck*,*Right*,*Suck*] will reach a goal state, 8.

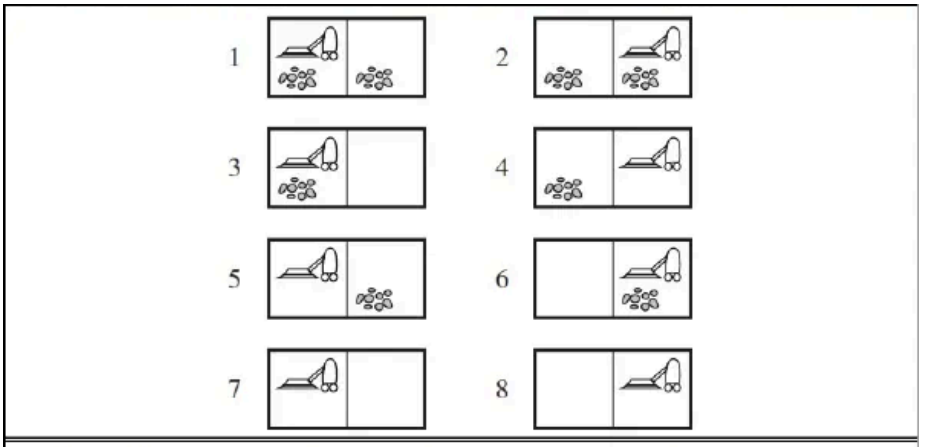


Figure 4.9 The eight possible states of the vacuum world; states 7 and 8 are goal states.

- ✓ Now suppose that we introduce nondeterminism in the form of a powerful but erratic vacuum cleaner. In the **erratic vacuum world**,the*Suck* action works as follows:
- ✓ When applied to a dirty square the action cleans the square and sometimes cleans up dirt in an adjacent square, too.
- When applied to a clean square the action sometimes deposits dirt on the carpet.
  - ✓

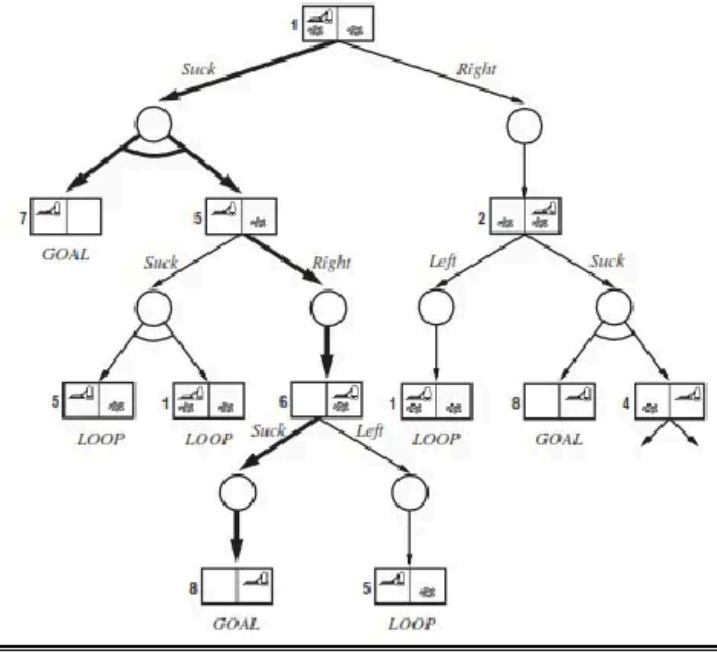


Figure 4.10 The first two levels of the search tree for the erratic vacuum world. State nodes are OR nodes where some action must be chosen. At the AND nodes, shown as circles, every outcome must be handled, as indicated by the arc linking the outgoing branches. The solution found is shown in bold lines.

Topic: 6:	SEARCH IN PARTIALLY OBSERVABLE ENVIRONMENTS

We now turn to the problem of partial observability, where the agent's percepts do not suffice to pin down the exact state. As noted at the beginning of the previous section, if the agent is in one of several possible states, then an action may lead to one of several possible outcomes—*even if the environment is deterministic*. The key concept required for solving partially observable problems is the **belief state**, representing the agent's current belief about the possible physical states it might be in, given the sequence of actions and percepts up to that point. We begin with the simplest scenario for studying belief states, which is when the agent has no sensors at all; then we add in partial sensing as well as nondeterministic actions.

4.4.1 Searching with no observation

When the agent's percepts provide *no information at all*, we have what is called a **sensorless** problem or sometimes a **conformant** problem. At first, one might think the sensorless agent has no hope of solving a problem if it has no idea what state it's in; in fact, sensorless problems are quite often solvable. Moreover, sensorless agents can be surprisingly useful, primarily because they *don't* rely on sensors working properly. In manufacturing systems, for example, many ingenious methods have been developed for orienting parts correctly from an unknown initial position by using a sequence of actions with no sensing at all. The high cost of sensing is another reason to avoid it: for example, doctors often prescribe a broad-spectrum antibiotic rather than using the contingent plan of doing an expensive blood test, then waiting for the results to come back, and then prescribing a more specific antibiotic and perhaps hospitalization because the infection has progressed too far.

We can make a sensorless version of the vacuum world. Assume that the agent knows the geography of its world, but doesn't know its location or the distribution of dirt. In that case, its initial state could be any element of the set $\{1, 2, 3, 4, 5, 6, 7, 8\}$. Now, consider what happens if it tries the action *Right*. This will cause it to be in one of the states $\{2, 4, 6, 8\}$—the agent now has more information! Furthermore, the action sequence [*Right*,*Suck*] will always end up in one of the states $\{4, 8\}$. Finally, the sequence [*Right*,*Suck*,*Left*,*Suck*] is guaranteed to reach the goal state 7 no matter what the start state. We say that the agent can **coerce** the world into state 7.

To solve sensorless problems, we search in the space of belief states rather than physical states.[10] Notice that in belief-state space, the problem is *fully observable* because the agent

- **Belief states**: The entire belief-state space contains every possible set of physical states. If P has N states, then the sensorless problem has up to 2^N states, although many may be unreachable from the initial state.
- **Initial state:** Typically the set of all states in P, although in some cases the agent will have more knowledge than this.
- **Actions**: This is slightly tricky. Suppose the agent is in belief state $b = \{s_1, s_2\}$, but $\text{ACTIONS}_P(s_1) \neq \text{ACTIONS}_P(s_2)$; then the agent is unsure of which actions are legal. If we assume that illegal actions have no effect on the environment, then it is safe to take the *union* of all the actions in any of the physical states in the current belief state b:

$$\text{ACTIONS}(b) = \bigcup_{s \in b} \text{ACTIONS}_P(s) .$$

  On the other hand, if an illegal action might be the end of the world, it is safer to allow only the *intersection*, that is, the set of actions legal in *all* the states. For the vacuum world, every state has the same legal actions, so both methods give the same result.
- **Transition model**: The agent doesn't know which state in the belief state is the right one; so as far as it knows, it might get to any of the states resulting from applying the action to one of the physical states in the belief state. For deterministic actions, the set of states that might be reached is

$$b' = \text{RESULT}(b, a) = \{s' : s' = \text{RESULT}_P(s, a) \text{ and } s \in b\} . \quad (4.4)$$

  With deterministic actions, b' is never larger than b. With nondeterminism, we have

$$\begin{aligned} b' = \text{RESULT}(b, a) &= \{s' : s' \in \text{RESULTS}_P(s, a) \text{ and } s \in b\} \\ &= \bigcup_{s \in b} \text{RESULTS}_P(s, a) , \end{aligned}$$

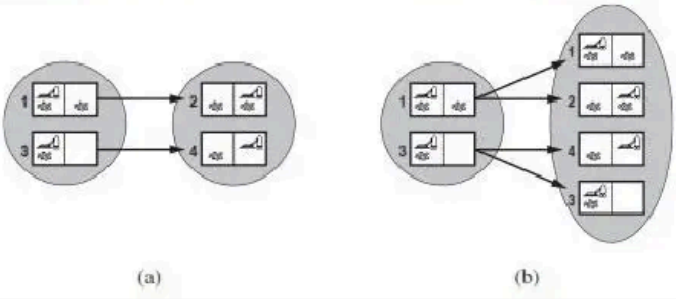


Figure 4.13 (a) Predicting the next belief state for the sensorless vacuum world with a deterministic action, *Right*. (b) Prediction for the same belief state and action in the slippery version of the sensorless vacuum world.

Figure 4.14 shows the reachable belief-state space for the deterministic, sensorless vacuum world. There are only 12 reachable belief states out of $2^8 = 256$ possible belief states.

The preceding definitions enable the automatic construction of the belief-state problem formulation from the definition of the underlying physical problem. Once this is done, we can apply any of the search algorithms of Chapter 3. In fact, we can do a little bit more than that. In "ordinary" graph search, newly generated states are tested to see if they are identical to existing states. This works for belief states, too; for example, in Figure 4.14, the action sequence [*Suck*,*Left*,*Suck*] starting at the initial state reaches the same belief state as [*Right*,*Left*,*Suck*], namely, $\{5, 7\}$. Now, consider the belief state reached by [*Left*], namely, $\{1, 3, 5, 7\}$. Obviously, this is not identical to $\{5, 7\}$, but it is a *superset*. It is easy to prove (Exercise 4.8) that if an action sequence is a solution for a belief state b, it is also a solution for any subset of b. Hence, we can discard a path reaching $\{1, 3, 5, 7\}$ if $\{5, 7\}$ has already been generated. Conversely, if $\{1, 3, 5, 7\}$ has already been generated and found to be solvable, then any *subset*, such as $\{5, 7\}$, is guaranteed to be solvable. This extra level of pruning may dramatically improve the efficiency of sensorless problem solving.

Topic: 6:	ONLINE SEARCH AGENTS AND UNKNOWN ENVIRONMENTS

- ✓ So far we have concentrated on agents that use offline search algorithms.
- ✓ They compute a complete solution before setting foot in the real world and then execute the solution.
- ✓ In contrast, an online search13 agent interleaves computation and action: first it takes an action, then it observes the environment and computes the next action.
- ✓ Online search is a good idea in dynamic or semi dynamic domains—domains where there is a penalty for sitting around and computing too long.
- ✓ Online search is also helpful in nondeterministic domains because it allows the agent to focus its computational efforts on the contingencies that actually arise rather than those that might happen but probably won't.
- ✓ Of course, there is a tradeoff: the more an agent plans ahead, the less often it will find itself up the creek without a paddle.
- ✓ Online search is a necessary idea for unknown environments, where the agent does not know what states exist or what its actions do. In this state of ignorance, the agent faces an exploration problem and must use its actions as experiments in order to learn enough to make deliberation worthwhile.
- ✓ The canonical example of online search is a robot that is placed in a new building and must explore it to build a map that it can use for getting from A to B.
- ✓ Methods for escaping from labyrinths—required knowledge for aspiring heroes of antiquity—are also examples of online search algorithms.
- ✓ Spatial exploration is not the only form of exploration, however
- ✓ .Consider a newborn baby: it has many possible actions but knows the outcomes of none of them, and it has experienced only a few of the possible states that it can reach.
- ✓ The baby's gradual discovery of how the world works is, in part, an online search process.

Online search problems

- ✓ An online search problem must be solved by an agent executing actions, rather than by pure computation.
- ✓ We assume a deterministic and fully observable environment , but we stipulate that the agent knows only the following:
- ✓ • ACTIONS(s), which returns a list of actions allowed in state s;

- ✓ • The step-cost function c(s, a, s)—note that this cannot be used until the agent knows that s is the outcome; and
- ✓ • GOAL-TEST(s). Note in particular that the agent cannot determine RESULT(s, a) except by actually being in s and doing a.
- ✓ For example, in the maze problem shown in Figure 4.19, the agent does not know that going Up from (1,1) leads to (1,2); nor, having done that, does it know that going Down will take it back to (1,1).
- ✓ This degree of ignorance can be reduced in some applications—for example, a robot explorer might know how its movement actions work and be ignorant only of the locations of obstacles.

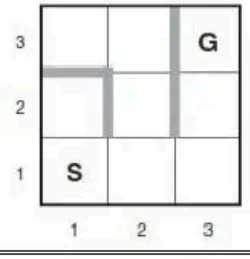


Figure 4.19 A simple maze problem. The agent starts at *S* and must reach *G* but knows nothing of the environment.

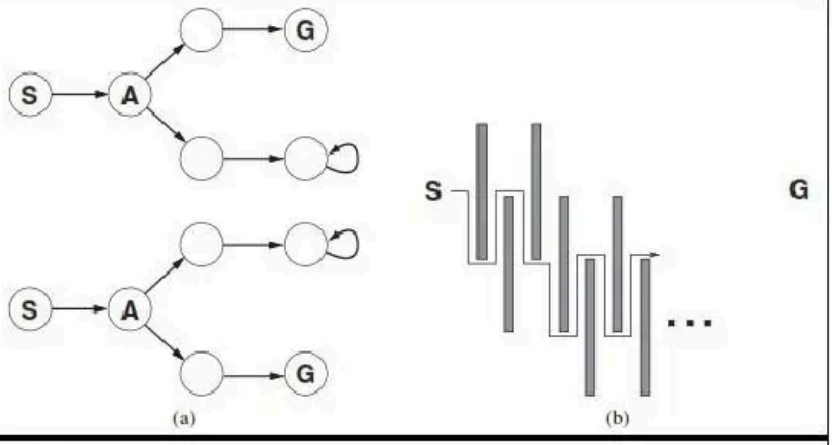


Figure 4.20 (a) Two state spaces that might lead an online search agent into a dead end. Any given agent will fail in at least one of these spaces. (b) A two-dimensional environment that can cause an online search agent to follow an arbitrarily inefficient route to the goal. Whichever choice the agent makes, the adversary blocks that route with another long, thin wall, so that the path followed is much longer than the best possible path.

- ✓ Finally, the agent might have access to an admissible heuristic function h(s) that estimates the distance from the current state to a goal state.
- ✓ For example, in Figure 4.19, the agent might know the location of the goal and be able to use the Manhattan-distance heuristic.
- ✓ Typically, the agent's objective is to reach a goal state while minimizing cost. (Another possible objective is simply to explore the entire environment.)
- ✓ The cost is the total path cost of the path that the agent actually travels. It is common to compare this cost with the path cost of the path the agent would follow *if it knew the search space in advance*—that is, the actual shortest path (or shortest complete exploration).
- ✓ In the language of online algorithms, this is called the **competitive ratio**; we would like it to be as small as possible.

Online Search Agent

- ✓ After each action, an online agent receives a percept telling it what state it has reached; from this information, it can augment its map of the environment.
- ✓ The current map is used to decide where to go next.
- ✓ This interleaving of planning and action means that online search algorithms are quite different from the offline search algorithms we have seen previously.
- ✓ For example, offline algorithms such as A can expand a node in one part of the space and then immediately expand a node in another part of the space, because node expansion involves simulated rather than real actions.
- ✓ An online algorithm, on the other hand, can discover successors only for a node that it physically occupies.
- ✓ To avoid traveling all the way across the tree to expand the next node, it seems better to expand nodes in a *local* order.
- ✓ Depth-first search has exactly this property because (except when backtracking) the next node expanded is a child of the previous node expanded.

Online local search

- ✓ Like depth-first search, **hill-climbing search** has the property of locality in its node expansions.
- ✓ In fact, because it keeps just one current state in memory, hill-climbing search is *already* an online search algorithm! Unfortunately, it is not very useful in its simplest form because it leaves the agent sitting at local maxima with nowhere to go.

- ✓ Moreover, random restarts cannot be used, because the agent cannot transport itself to a new state. Instead of random restarts, one might consider using a **random walk** to explore the environment.

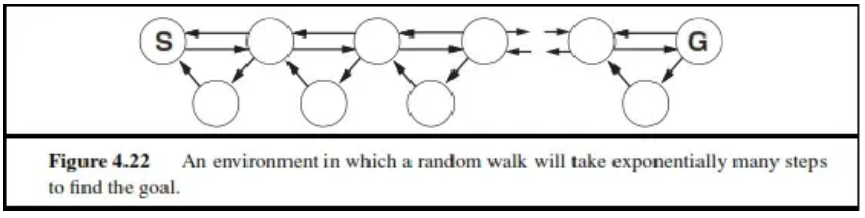


Figure 4.22 An environment in which a random walk will take exponentially many steps to find the goal.

- ✓ A random walk simply selects at random one of the available actions from the current state; preference can be given to actions that have not yet been tried. It is easy to prove that a random walk will *eventually* find a goal or complete its exploration, provided that the space is finite. On the other hand, the process can be very slow. Figure 4.22 shows an environment in which a random walk will take exponentially many steps to find the goal because, at each step, backward progress is twice as likely as forward progress. The example is contrived, of course, but there are many real-world state spaces whose topology causes these kinds of "traps" for random walks.

UNIT III

GAME PLAYING AND CSP

SYLLABUS

Game theory – optimal decisions in games – alpha-beta search – monte-carlo tree search – stochastic games – partially observable games. Constraint satisfaction problems – constraint propagation – backtracking search for CSP – local search for CSP – structure of CSP.

INTRODUCTION

Adversarial Search & Games

Definition

Adversarial search is basically a kind of search in which one can trace the movement of an enemy or opponent.

or

Adversarial search is a search, where we examine the problem which arises when we try to plan ahead of the world and other agents are planning against us.

- ✓ In artificial intelligence, deep learning, machine learning, and computer vision, adversarial search is basically a kind of search.
- ✓ In which one can trace the movement of an enemy or opponent.
- ✓ The step which arises the problem for the user or the agent doesn't want that specific step task to be carried out.
- ✓ Such searches are important in chess, business strategy tools, trading kind of websites, and war-based games where there are AI agents.
- ✓ In an adversarial search, the user can change the current state but has no control over the next stage.
- ✓ The next state control is in the hands of the opponent and is unpredictable.

- In previous topics, we have studied the search strategies which are only associated with a single agent that aims to find the solution which often expressed in the form of a sequence of actions.
- But, there might be some situations where more than one agent is searching for the solution in the same search space, and this situation usually occurs in game playing.

- The environment with more than one agent is termed as **multi-agent environment**, in which each agent is an opponent of other agent and playing against each other. Each agent needs to consider the action of other agent and effect of that action on their performance.
- So, Searches in which two or more players with conflicting goals are trying to explore the same search space for the solution, are called **adversarial searches**, often known as **Games**.
- Games are modeled as a Search problem and heuristic evaluation function, and these are the two main factors which help to model and solve games in AI.

Topic: 1 :	**GAME THEORY**

Game : Any set of circumstances that has a result dependent on the actions of two or more decision-makers (players)

- **Players** : A strategic decision-maker within the context of the game
- **Strategy**: A complete plan of action a player will take given the set of circumstances that might arise within the game
- **Payoff** : The payout a player receives from arriving at a particular outcome (The payout can be in any quantifiable form, from dollars to utility.)

✓ We first consider games with two players, whom we call MAX and MIN for reasons that will soon become obvious.
✓ MAX moves first, and then they take turns moving until the game is over.
✓ At the end of the game, points are awarded to the winning player and penalties are given to the loser.

Types of Games in AI:

- **Perfect information:** A game with the perfect information is that in which agents can look into the complete board. Agents have all the information about the game, and they can see each other moves also. Examples are Chess, Checkers, Go, etc.

- **Imperfect information:** If in a game agents do not have all information about the game and not aware with what's going on, such type of games are called the game with imperfect information, such as tic-tac-toe, Battleship, blind, Bridge, etc.

- **Deterministic games:** Deterministic games are those games which follow a strict pattern and set of rules for the games, and there is no randomness associated with them. Examples are chess, Checkers, Go, tic-tac-toe, etc.

- **Non-deterministic games:** Non-deterministic are those games which have various unpredictable events and has a factor of chance or luck. This factor of chance or luck is introduced by either dice or cards. These are random, and each action response is not fixed. Such games are also called as stochastic games. Example: Backgammon, Monopoly, Poker, etc.

Applications of Game theory

The game theory is widely applied to study human as well as animal behaviors. It is utilized in economics to understand the economic behaviors, such as behaviors of consumers, markets and firms. Game theory has been commonly used in social sciences as well. It is applied in the study of sociological, political and psychological behaviors. The use of analysis based on game theory is seen in biology too. In addition to behavioral prediction, game theory utilized in the development of theories of normative or ethical behavior.

✓ The following are just a few examples of game theory applications:
- Stock trades and the investors' reactions and decisions against stock market developments and the behaviors and decisions of other investors
- OPEC member countries' decision to change the amount of oil extraction and sale and their compliance or non-compliance with quota arrangements
- Corporate behavior regarding product pricing in monopoly or multilateral competition markets
- Animal interaction with one another in social life (hunting or sharing achievements or supporting each other)

Elements of Game Theory

✓ A game can be defined as a type of search in AI which can be formalized of the following elements:
- **Initial state:** It specifies how the game is set up at the start.
- **Player(s):** It specifies which player has moved in the state space.
- **Action(s):** It returns the set of legal moves in state space.

- **Result(s, a):** It is the transition model, which specifies the result of moves in the state space.
- **Terminal-Test(s):** Terminal test is true if the game is over, else it is false at any case. The state where the game ends is called terminal states.
- **Utility(s, p):** A utility function gives the final numeric value for a game that ends in terminal states s for player p. It is also called payoff function. For Chess, the outcomes are a win, loss, or draw and its payoff values are +1, 0, ½. And for tic-tac-toe, utility values are +1, -1, and 0.

Game tree :

✓ A game tree is a tree where nodes of the tree are the game states and Edges of the tree are the moves by players.

✓ Game tree involves initial state, actions function, and result Function.

Example: Tic-Tac-Toe game tree :

✓ The following figure is showing part of the game-tree for tic-tac-toe game. Following are some key points of the game:

- There are two players MAX and MIN.
- Players have an alternate turn and start with MAX.
- MAX maximizes the result of the game tree
- MIN minimizes the result.

Example Explanation:

- From the initial state, MAX has 9 possible moves as he starts first. MAX place x and MIN place o, and both player plays alternatively until we reach a leaf node where one player has three in a row or all squares are filled.
- Both players will compute each node, minimax, the minimax value which is the best achievable utility against an optimal adversary.
- Suppose both the players are well aware of the tic-tac-toe and playing the best play. Each player is doing his best to prevent another one from winning. MIN is acting against Max in the game.
- So in the game tree, we have a layer of Max, a layer of MIN, and each layer is called as **Ply**. Max place x, then MIN puts o to prevent Max from winning, and this game continues until the terminal node.
- In this either MIN wins, MAX wins, or it's a draw. This game-tree is the whole search space of possibilities that MIN and MAX are playing tic-tac-toe and taking turns alternately.

Hence adversarial Search for the minimax procedure works as follows:

- It aims to find the optimal strategy for MAX to win the game.
- It follows the approach of Depth-first search.
- In the game tree, optimal leaf node could appear at any depth of the tree.
- Propagate the minimax values up to the tree until the terminal node discovered.

✓ In a given game tree, the optimal strategy can be determined from the minimax value of each node, which can be written as MINIMAX(n).

✓ MAX prefer to move to a state of maximum value and MIN prefer to move to a state of minimum value then:

Topic: 2 : OPTIMAL DECISION MAKING IN GAMES

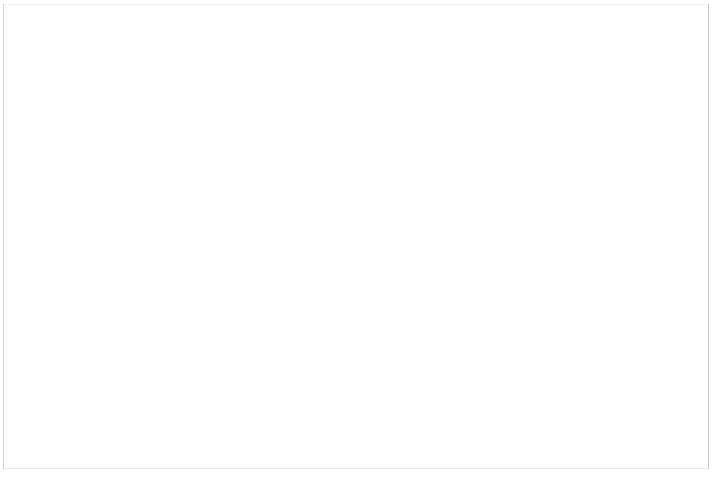
Optimal Decision Making in Games

Let us start with games with two players, whom we'll refer to as MAX and MIN for obvious reasons. MAX is the first to move, and then they take turns until the game is finished. At the conclusion of the game, the victorious player receives points, while the loser receives penalties. A game can be formalized as a type of search problem that has the following elements:

- **S_0:** The initial state of the game, which describes how it is set up at the start.
- **Player (s):** Defines which player in a state has the move.
- **Actions (s):** Returns a state's set of legal moves.
- **Result(s, a):** A transition model that defines a move's outcome.
- **Terminal-Test (s):** A terminal test that returns true if the game is over but false otherwise. Terminal states are those in which the game has come to a conclusion.
- **Utility (s, p):** A utility function (also known as a payout function or objective function) determines the final numeric value for a game that concludes in the terminal state ***s*** for player ***p***. The result in chess is a win, a loss, or a draw, with values of +1, 0, or 1/2. Backgammon's payoffs range from 0 to +192, but certain games have a greater range of possible outcomes. A zero-sum game is defined (confusingly) as one in which the total reward to all players is the same for each game instance. Chess is a zero-sum game because each game has a payoff of 0 + 1, 1 + 0, or 1/2 + 1/2. "Constant-sum" would have been a preferable name, 22 but zero-sum is the usual term and makes sense if each participant is charged 1.

✓ The optimal solution becomes a contingent strategy when specifies MAX(the player on our side)'s move in the initial state, then Max move to the states resulting for every possible response by MIN.

✓ Then MAX's moves in the states resulting from every possible response by MIN to those moves, and so on



✓ From the current state, the minimax method in the Figure above computes the minimax choice.

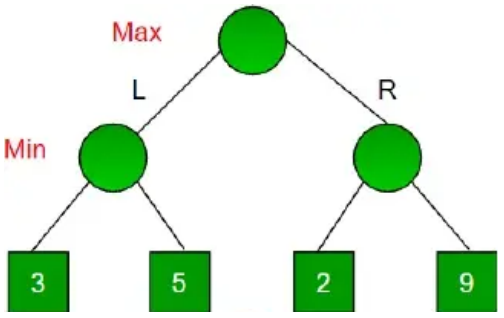
- ✓ It implements the defining equations directly using a simple recursive computation of the minimax values of each successor state. As the recursion unwinds, it progresses all the way down to the tree's leaves, where the minimax values are then backed up through the tree.
- ✓ In the figure below, for example, the algorithm recurses down to the three bottom-left nodes and uses the UTILITY function to determine that their values are 3, 12, and 8, respectively.
- ✓ Then it takes the smallest of these values, 3 in this case, and returns it as node B's backed-up value.
- ✓ The backed-up values of 2 for C and 2 for D are obtained using a similar approach.
- ✓ Finally, we add the maximum of 3, 2, and 2 to get the root node's backed-up value of 3.

MINIMAX ALGORITHM IN GAME THEORY

- ✓ Minimax is a kind of backtracking algorithm that is used in decision making and game theory to find the optimal move for a player, assuming that your opponent also plays optimally.
- ✓ It is widely used in two player turn-based games such as Tic-Tac-Toe, Backgammon, Mancala, Chess, etc.
- ✓ In Minimax the two players are called maximizer and minimizer.
- ✓ The **maximizer** tries to get the highest score possible while the **minimizer** tries to do the opposite and get the lowest score possible.
- ✓ Every board state has a value associated with it. In a given state if the maximizer has upper hand then, the score of the board will tend to be some positive value.
- ✓ If the minimizer has the upper hand in that board state then it will tend to be some negative value.
- ✓ The values of the board are calculated by some heuristics which are unique for every type of game.

Example:

- ✓ Consider a game which has 4 final states and paths to reach final state are from root to 4 leaves of a perfect binary tree as shown below.
- ✓ Assume you are the maximizing player and you get the first chance to move, i.e., you are at the root and your opponent at next level. **Which move you would make as a maximizing player considering that your opponent also plays optimally?**



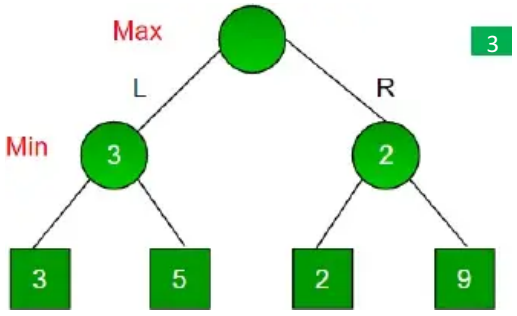
- ✓ Since this is a backtracking based algorithm, it tries all possible moves, then backtracks and makes a decision.

In Minimizer Turn

- ✓ In Left Node, The minimizer now has a choice between 3 and 5. Being the minimizer it will definitely choose the least among both, that is 3.
- ✓ In Right Node, The minimizer now has a choice between 2 and 9. Being the minimizer it will definitely choose the least among both, that is 2.

In Maximizer Turn

- ✓ Being the maximizer you would choose the larger value that is 3. Hence the optimal move for the maximizer is to go LEFT and the optimal value is 3.



- In the above example, there are only two choices for a player.
- In general, there can be more choices.
- In that case, we need to recur for all possible moves and find the maximum/minimum.
- For example, in Tic-Tac-Toe, the first player can make 9 possible moves.
- In the above example, the scores (leaves of Game Tree) are given to us. For a typical game, we need to derive these values

Tic Tac To

- Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory.
- It provides an optimal move for the player assuming that opponent is also playing optimally.
- Mini-Max algorithm uses recursion to search through the game-tree.
- Min-Max algorithm is mostly used for game playing in AI.
- Such as Chess, Checkers, tic-tac-toe, go, and various tow-players game.
- This Algorithm computes the minimax decision for the current state.

Min & MAx

- In this algorithm two players play the game, one is called MAX and other is called MIN.
- Both the players fight it as the opponent player gets the minimum benefit while they get the maximum benefit.
- Both Players of the game are opponent of each other, where MAX will select the maximized value and MIN will select the minimized value.
- The minimax algorithm performs a depth-first search algorithm for the exploration of the complete game tree.

- The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion

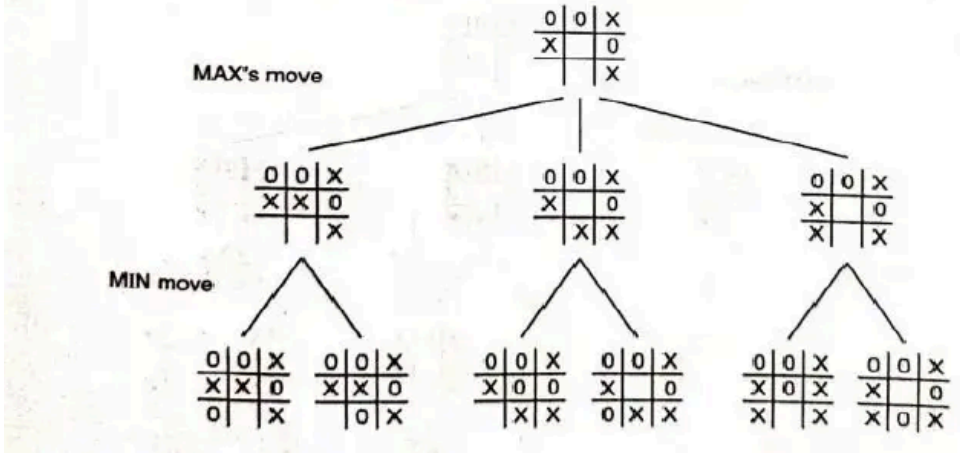
Game Playing Procedure

- Assume the two players named MIN and Max who are playing the game.
- MAX is playing first.
- Initially Max has nine possible moves.
- Play alternate between MAX and MIN until reach leaf node, such that one player has 3 in a row or all squares are filled.
- It's a MAX job to make use of search tree to determine best move.
- To evaluate this, put +1 for a win, 0 for draw.

Consider the tic tac toe game with initial state,

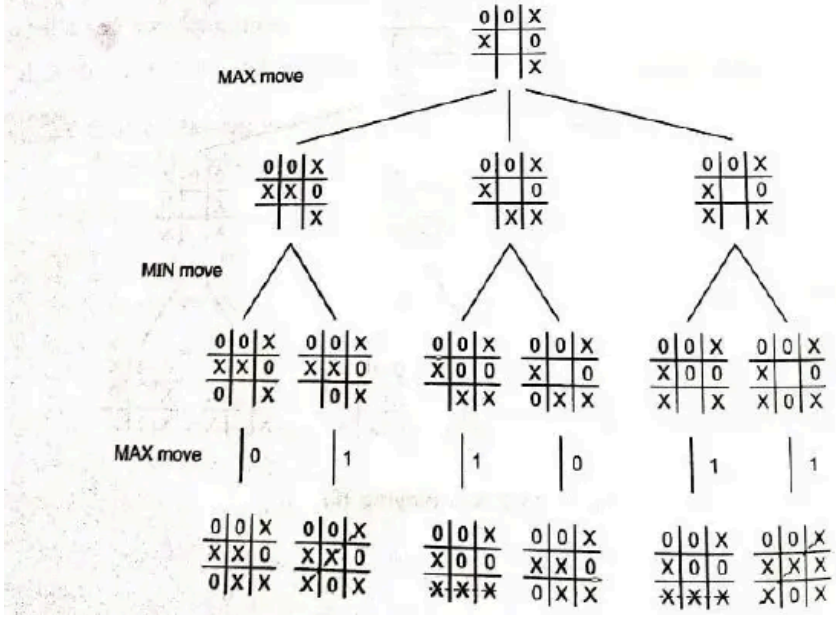
Step 2 :

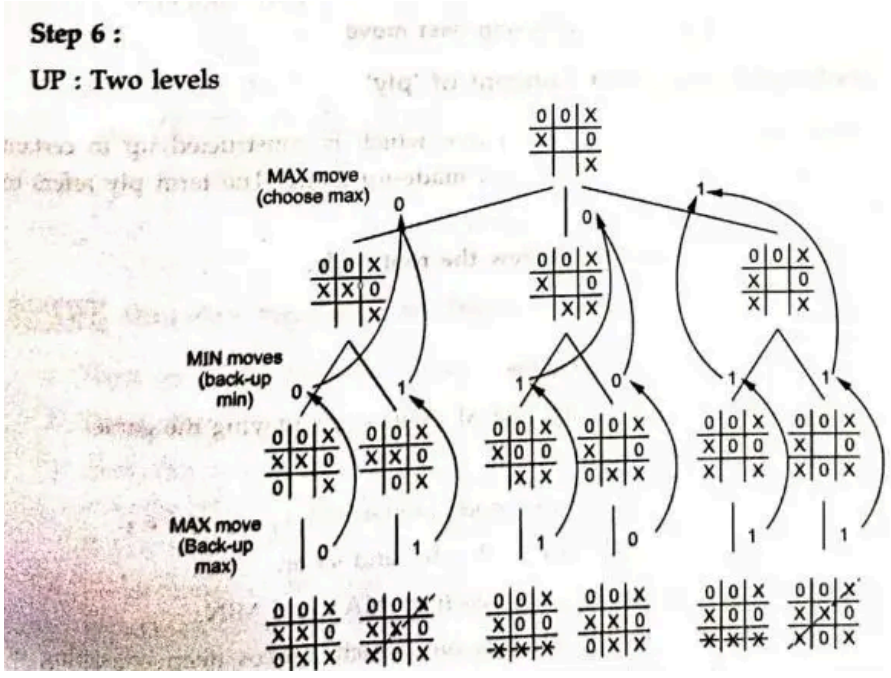
Next : MIN (player 2) moves.



Step 4 :

'+ 1' for a win, '0' for a draw. Criteria '+ 1' for a win, '0' for a draw.





Topic: 3 : Alpha-Beta Search / Alpha-Beta Pruning

- ✓ Alpha-beta pruning is a modified version of the minimax algorithm.
- ✓ It is an optimization technique for the minimax algorithm.
- ✓ As we have seen in the minimax search algorithm that the number of game states it has to examine are exponential in depth of the tree.
- ✓ There is a technique by which without checking each node of the game tree we can compute the correct minimax decision, and this technique is called **pruning**.
- ✓ This involves two threshold parameter Alpha and beta for future expansion, so it is called **alpha-beta pruning**.
- ✓ It is also called as **Alpha-Beta Algorithm**.
- ✓ Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prune the tree leaves but also entire sub-tree.

- ✓ The two-parameter can be defined as:
  Alpha & Beta

Alpha:

- ✓ The best (highest-value) choice we have found so far at any point along the path of Maximizer.
- ✓ The initial value of alpha is -∞.

Beta:

- ✓ The best (lowest-value) choice we have found so far at any point along the path of Minimizer.
- ✓ The initial value of beta is +∞.
  - The Alpha-beta pruning to a standard minimax algorithm returns the same move as the standard algorithm does, but it removes all the nodes which are not really affecting the final decision but making algorithm slow.
  - Hence by pruning these nodes, it makes the algorithm fast.

- ✓ Condition for Alpha-beta pruning:
- ✓ The main condition which required for alpha-beta pruning is:

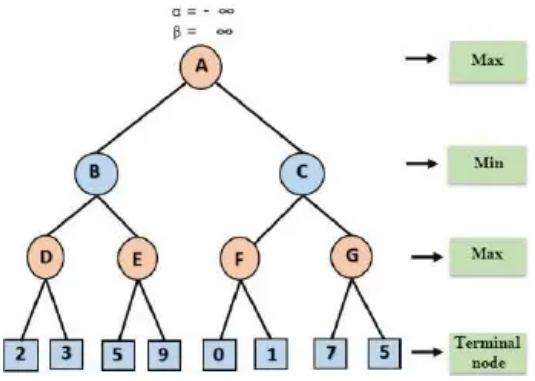
$$\alpha >= \beta$$

- ✓ Key points about alpha-beta pruning:
  - The Max player will only update the value of alpha.
  - The Min player will only update the value of beta.
  - While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta.
  - We will only pass the alpha, beta values to the child nodes.

Working of Alpha-Beta Pruning:

Let's take an example of two-player search tree to understand the working of Alpha-beta pruning

Step 1: At the first step the, Max player will start first move from node A where α= -∞ and β= +∞, these value of alpha and beta passed down to node B where again α= -∞ and β= +∞, and Node B passes the same value to its child D.

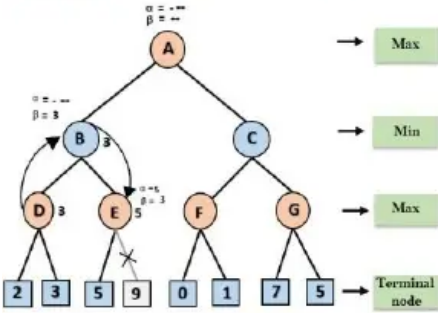


Step 2: At Node D, the value of α will be calculated as its turn for Max. The value of α is compared with firstly 2 and then 3, and the max (2, 3) = 3 will be the value of α at node D and node value will also 3.

Step 3: Now algorithm backtrack to node B, where the value of β will change as this is a turn of Min, Now β= +∞, will compare with the available subsequent nodes value, i.e. min (∞, 3) = 3, hence at node B now α= -∞, and β= 3.

In the next step, algorithm traverse the next successor of Node B which is node E, and the values of α= -∞, and β= 3 will also be passed.

Step 4: At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so max (-∞, 5) = 5, hence at node E α= 5 and β= 3, where α>=β, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5.

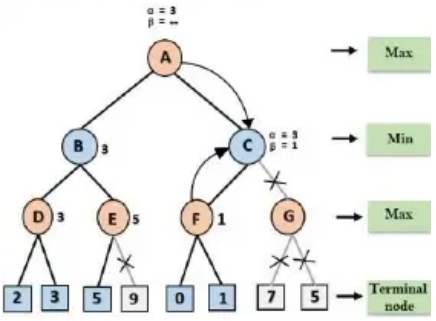


Step 5: At next step, algorithm again backtrack the tree, from node B to node A. At node A, the value of alpha will be changed the maximum available value is 3 as max (-∞, 3)= 3, and β= +∞, these two values now passes to right successor of A which is Node C.

At node C, α=3 and β= +∞, and the same values will be passed on to node F.

Step 6: At node F, again the value of α will be compared with left child which is 0, and max(3,0)= 3, and then compared with right child which is 1, and max(3,1)= 3 still α remains 3, but the node value of F will become 1.

Step 7: Node F returns the node value 1 to node C, at C α= 3 and β= +∞, here the value of beta will be changed, it will compare with 1 so min (∞, 1) = 1. Now at C, α=3 and β= 1, and again it satisfies the condition α>=β, so the next child of C which is G will be pruned, and the algorithm will not compute the entire sub-tree G.



Step 8: C now returns the value of 1 to A here the best value for A is max (3, 1) = 3. Following is the final game tree which is the showing the nodes which are computed and nodes which has never computed. Hence the optimal value for the maximizer is 3 for this example.

Move Ordering in Alpha-Beta pruning:

- ✓ The effectiveness of alpha-beta pruning is highly dependent on the order in which each node is examined. Move order is an important aspect of alpha-beta pruning.
- ✓ It can be of two types:
  - **Worst ordering:** In some cases, alpha-beta pruning algorithm does not prune any of the leaves of the tree, and works exactly as minimax algorithm. In this case, it also consumes more time because of alpha-beta factors, such a move of pruning is called worst ordering. In this case, the best move occurs on the right side of the tree. The time complexity for such an order is $O(b^m)$.
  - **Ideal ordering:** The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and best moves occur at the left side of the tree. We apply DFS hence it first search left of the tree and go deep twice as minimax algorithm in the same amount of time. Complexity in ideal ordering is $O(b^{m/2})$.

Topic: 4 :	Monte Carlo Tree Search (MCTS)

- ✓ The Games like Tic-Tac-Toe, Rubik's Cube, Sudoku, Chess, Go and many others have common property that lead to exponential increase in the number of possible actions that can be played.
- ✓ These possible steps increase exponentially as the game goes forward.
- ✓ Ideally if you can predict every possible move and its result that may occur in the future.
- ✓ You can increase your chance of winning.
- ✓ But since the moves increase exponentially — the computation power that is required to calculate the moves also goes through the roof.
- ✓ Monte Carlo Tree Search is a method usually used in games to predict the path (moves) that should be taken by the policy to reach the final winning solution

Tree Search Algorithm

- ✓ It is a method used to search every possible move that may exist after a turn in the game.
- ✓ For Example: There are many different options for a player in the game of tic-tac-toe, which can be visualized using a tree representation.
- ✓ The move can be further increased for the next turn evolving the diagram into a tree.

- ✓ An exponential increasing tree to find the final solution for a problem by considering every children move/node requires a lot of computational power. resulting in extremely slow output
- ✓ Faster Tree Search can be achieved by making a policy — giving more importance to some nodes from others & allowing their children nodes to be searched first to reach the correct solution.

- ✓ But how to find that node which is most favourable to have the correct solution in their children nodes. Let's find out…

MCTS is an algorithm that figures out the best move out of a set of moves by Selecting → Expanding → Simulating → Updating the nodes in tree to find the final solution. This method is repeated until it reaches the solution and learns the policy of the game.

1. **Selection**
   Selecting good child nodes, starting from the root node R, that represent states leading to better overall outcome (win).
2. **Expansion**
   If *L* is a not a terminal node (i.e. it does not end the game), then create one or more child nodes and select one (*C*).
3. **Simulation (rollout)**
   Run a simulated playout from *C* until a result is achieved.
4. **Backpropagation**
   Update the current move sequence with the simulation result.

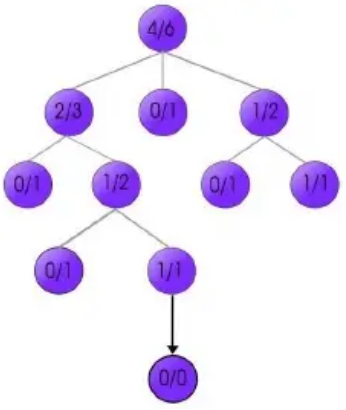
Selection

This process is used to select a node on the tree that has the highest possibility of winning. For Example — Consider the moves with winning possibility 2/3, 0/1 & 1/2 after the first move 4/6, the node **2/3** has the highest possibility of winning.

The node selected is searched from the current state of the tree and selected node is located at the end of the branch. Since the selected node has the highest possibility of winning — that path is also most likely to reach the solution faster than other path in the tree.

Expanding — After selecting the right node. Expanding is used to increase the options further in the game by expanding the selected node and creating many children nodes. We are

using only one children node in this case. These children nodes are the future moves that can be played in the game.

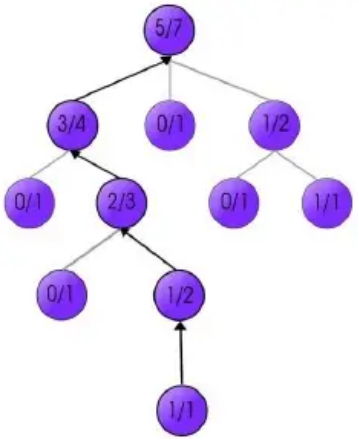


The nodes that are not expanded further for the time being are known are leaves.

Simulating|Exploring Since nobody knows which node is the best children/ leaf from the group. The move which will perform best and lead to the correct answer down the tree. But,
How do we find the best children which will lead us to the correct solution?
We use Reinforcement Learning to make random decisions in the game further down from every children node. Then, reward is given to every children node — by calculating how close the output of their random decision was from the final output that we need to win the game.

The simulation is done for every children node is followed by their individual rewards.
Let's say the simulation of the node gives optimistic results for its future and gets a positive score 1/1.

Updating|Back-propagation — Due to the new nodes and their *positive or negative* scores in the environment. The total scores of their parent nodes must be updated by going back up the tree one-by-one. The new updated scores changes the state of the tree and may also change new future node of the selection process.



After updating all the nodes, the loop again begins by selection the best node in the tree→ *expanding* of the selected node → using RL for simulating exploration → *back-propagating* the updated scores → then finally *selecting* a new node further down the tree that is actual the required final winning result.

Hence, Instead of brute forcing from millions of possible ways to find the right path.

Monte Carlo Tree Search algorithm chooses the best possible move from the current state of Game's Tree with the help of Reinforcement Learning.

Advantages of Monte Carlo Tree Search:

1. MCTS is a simple algorithm to implement.
2. Monte Carlo Tree Search is a heuristic algorithm. MCTS can operate effectively without any knowledge in the particular domain, apart from the rules and end conditions, and can find its own moves and learn from them by playing random playouts.
3. The MCTS can be saved in any intermediate state and that state can be used in future use cases whenever required.
4. MCTS supports asymmetric expansion of the search tree based on the circumstances in which it is operating.

Disadvantages of Monte Carlo Tree Search:

1. As the tree growth becomes rapid after a few iterations, it requires a huge amount of memory.
2. There is a bit of a reliability issue with Monte Carlo Tree Search. In certain scenarios, there might be a single branch or path, that might lead to loss against the opposition

when implemented for those turn-based games. This is mainly due to the vast amount of combinations and each of the nodes might not be visited enough number of times to understand its result or outcome in the long run.

3. MCTS algorithm needs a huge number of iterations to be able to effectively decide the most efficient path. So, there is a bit of a speed issue there.

Topic: 5 :	STOCHASTIC GAMES

- ✓ In real life, many unpredictable external events can put us into unforeseen situations.
- ✓ Many games mirror this unpredictability by including a random element, such as the throwing of dice.
- ✓ We call these **stochastic games**.
- ✓ Backgammon is a typical game that combines luck and skill.
- ✓ Dice are rolled at the beginning of a player's turn to determine the legal moves.
- ✓ In the backgammon position of Figure , for example, White has rolled a 6–5 and has four possible moves.

A typical backgammon position. The goal of the game is to move all one's pieces off the board. White moves clockwise toward 25, and Black moves counter clockwise toward 0. A piece can move to any position unless multiple opponent pieces are there;

if there is one opponent, it is captured and must start over. In the position shown, White has rolled 6–5 and must choose among four legal moves: (5–10,5–11), (5–11,19–24), (5–10,10–16), and (5–11,11–16), where the notation (5–11,11–16) means move one piece from position 5 to 11, and then move a piece from 11 to 16 .

Although White knows what his or her own legal moves are, White does not know what Black is going to roll and thus does not know what Black's legal moves will be. That means White cannot construct a standard game tree of the sort we saw in chess and tic-tac-toe. A game tree in backgammon must include **chance nodes** in addition to MAX and MIN nodes.

Chance nodes are shown as circles in Figure 5.11. The branches leading from each chance node denote the possible dice rolls; each branch is labeled with the roll and its probability. There are 36 ways to roll two dice, each equally likely; but because a 6–5 is the same as a 5–6, there are only 21 distinct rolls. The six doubles (1 –1 through 6–6) each have a probability of 1/36, so we say P(1 –1)=1/36. The other 15 distinct rolls each have a 1/18 probability.

Schematic game tree for a backgammon position.

The next step is to understand how to make correct decisions. Obviously, we still want to pick the move that leads to the best position. However, positions do not have definite minimax values. Instead, we can only calculate the **expected value** of a position: the average over all possible outcomes of the chance nodes.

This leads us to generalize the minimax value for deterministic games to an expectation minimax value for games with chance nodes. Terminal nodes and MAX and MIN nodes (for which the dice roll is known) work exactly the same way as before. For chance nodes we compute the expected value, which is the sum of the value over all outcomes, weighted by the probability of each chance action:

$$\text{EXPECTIMINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if TERMINAL-TEST}(s) \\ \max_a \text{EXPECTIMINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MAX} \\ \min_a \text{EXPECTIMINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MIN} \\ \sum_r P(r)\text{EXPECTIMINIMAX}(\text{RESULT}(s,r)) & \text{if PLAYER}(s) = \text{CHANCE} \end{cases}$$

where r represents a possible dice roll (or other chance event) and RESULT(s, r) is the same state as s, with the additional fact that the result of the dice roll is r.

Topic: 6 : PARTIALLY OBSERVABLE GAMES

Chess has often been described as war in miniature, but it lacks at least one major characteristic of real wars, namely, **partial observability**. In the "fog of war," the existence and disposition of enemy units is often unknown until revealed by direct contact. As a result, warfare includes the use of scouts and spies to gather information and the use of concealment and bluff to confuse the enemy. Partially observable games share these characteristics and are thus qualitatively different from the games described in the preceding sections.

Kriegspiel: Partially observable chess

In deterministic partially observable games, uncertainty about the state of the board arises entirely from lack of access to the choices made by the opponent. This class includes children's games such as Battleships (where each player's ships are placed in

locations hidden from the opponent but do not move) and Stratego (where piece locations are known but piece types are hidden). We will examine the game of **Kriegspiel**, a partially observable variant of chess in which pieces can move but are completely invisible to the opponent.

The rules of Kriegspiel are as follows:

- White and Black each see a board containing only their own pieces.
- A referee, who can see all the pieces, adjudicates the game and periodically makes announcements that are heard by both players.
- On his turn, White proposes to the referee any move that would be legal if there were no black pieces.
- If the move is in fact not legal (because of the black pieces), the referee announces "illegal."
- In this case, White may keep proposing moves until a legal one is found—and learns more about the location of Black's pieces in the process.

Once a legal move is proposed, the referee announces one or more of the following: "Capture on square *X*" if there is a capture, and "Check by *D*"ifthe black king is in check, where *D* is the direction of the check, and can be one of "Knight," "Rank," "File," "Long diagonal," or "Short diagonal." (In case of discovered check, the referee may make two "Check" announcements.) If Black is checkmated or stalemated, the referee says so; otherwise, it is Black's turn to move.

For Kriegspiel, a winning strategy, or **guaranteed checkmate**, is one that, for each possible percept sequence, leads to an actual checkmate for every possible board state in the current belief state, regardless of how the opponent moves. With this definition, the opponent's belief state is irrelevant—the strategy has to work even if the opponent can see all the pieces. This greatly simplifies the computation. Figure 5.13 shows part of a guaranteed checkmate for the KRK (king and rook against king) endgame. In this case, Black has just one piece (the king), so a belief state for White can be shown in a single board by marking each possible position of the Black king.

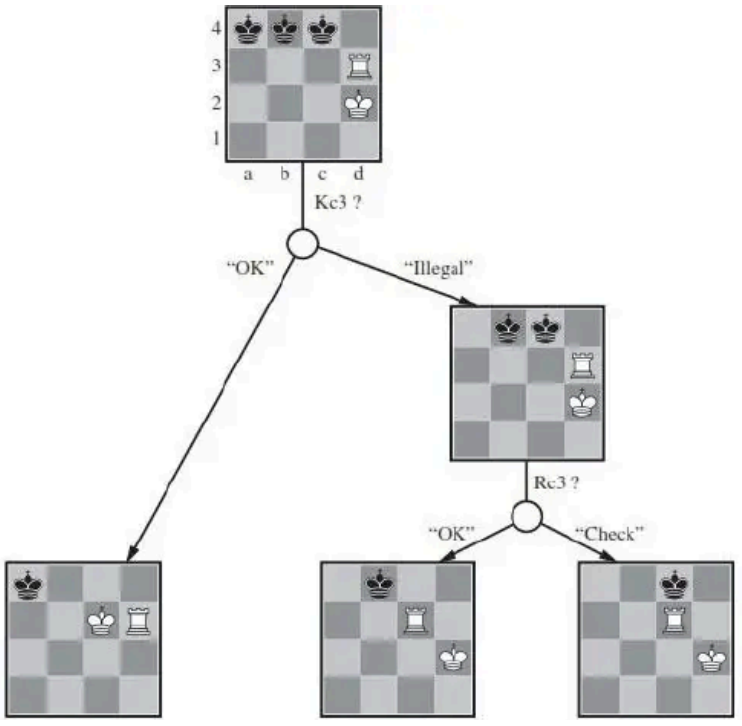


Figure 5.13 Part of a guaranteed checkmate in the KRK endgame, shown on a reduced board. In the initial belief state, Black's king is in one of three possible locations. By a combination of probing moves, the strategy narrows this down to one. Completion of the checkmate is left as an exercise.

Card games

Card games provide many examples of *stochastic* partial observability, where the missing information is generated randomly. For example, in many games, cards are dealt randomly at the beginning of the game, with each player receiving a hand that is not visible to the other players. Such games include bridge, whist, hearts, and some forms of poker.

At first sight, it might seem that these card games are just like dice games: the cards are dealt randomly and determine the moves available to each player, but all the "dice" are rolled at the beginning! Even though this analogy turns out to be incorrect, it suggests an effective algorithm: consider all possible deals of the invisible cards; solve each one as if it were a fully observable game; and then choose the move that

has the best outcome averaged over all the deals. Suppose that each deal s occurs with probability P(s); then the move we want is

Topic: 7 :	CONSTRAINT SATISFACTION PROBLEMS

We have seen so many techniques like Local search, Adversarial search to solve different problems. The objective of every problem-solving technique is one, i.e., to find a solution to reach the goal. Although, in adversarial search and local search, there were no constraints on the agents while solving the problems and reaching to its solutions.

we will discuss another type of problem-solving technique known as Constraint satisfaction technique. By the name, it is understood that constraint satisfaction means *solving a problem under certain constraints or rules.*

Constraint satisfaction is a technique where a problem is solved when its values satisfy certain constraints or rules of the problem. Such type of technique leads to a deeper understanding of the problem structure as well as its complexity.

Constraint satisfaction depends on three components, namely:

- **X:** It is a set of variables.
- **D:** It is a set of domains where the variables reside. There is a specific domain for each variable.
- **C:** It is a set of constraints which are followed by the set of variables.

Types of Domains in CSP

There are following two types of domains which are used by the variables :

- **Discrete Domain:** It is an infinite domain which can have one state for multiple variables. **For example,** a start state can be allocated infinite times for each variable.
- **Finite Domain:** It is a finite domain which can have continuous states describing one domain for one specific variable. It is also called a continuous domain.

Constraint Types in CSP

With respect to the variables, basically there are following types of constraints:

- **Unary Constraints:** It is the simplest type of constraints that restricts the value of a single variable.
- **Binary Constraints:** It is the constraint type which relates two variables. A value $\mathbf{x_2}$ will contain a value which lies between **x1** and **x3**.
- **Global Constraints:** It is the constraint type which involves an arbitrary number of variables.

Example problem: Map coloring

The problem where the constraint is that no adjacent sides can have the same color.

Suppose that, having tired of Romania, we are looking at a map of Australia showing each of its states and territories (Figure 6.1(a)). We are given the task of coloring each region either red, green, or blue in such a way that no neighboring regions have the same color. To formulate this as a CSP, we define the variables to be the regions

X = {WA, NT,Q,NSW,V, SA, T} .

The domain of each variable is the set Di= {red , green, blue}. The constraints require neighboring regions to have distinct colors. Since there are nine places where regions border, there are nine constraints:

Here we are using abbreviations; $SA \neq WA$ is a shortcut for $\langle (SA, WA), SA \neq WA \rangle$, where $SA \neq WA$ can be fully enumerated in turn as

$$\{(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)\} .$$

There are many possible solutions to this problem, such as

$$\{ WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = red \}.$$

It can be helpful to visualize a CSP as a **constraint graph**, as shown in Figure 6.1(b). The nodes of the graph correspond to variables of the problem, and a link connects any two variables that participate in a constraint.

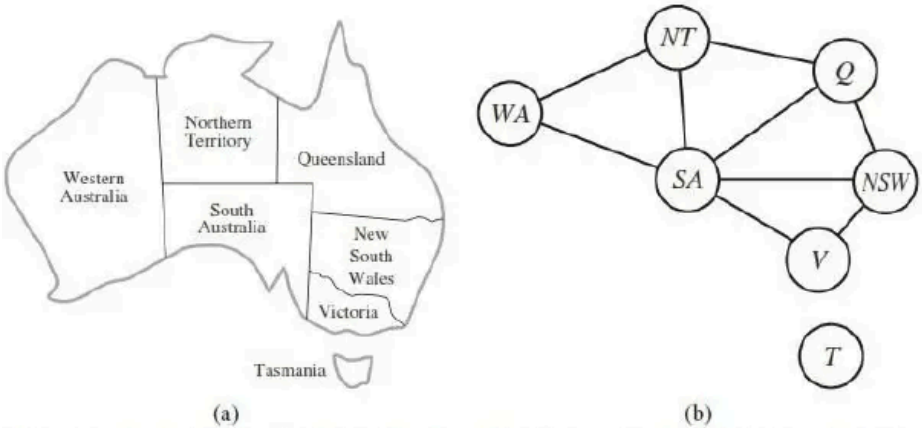


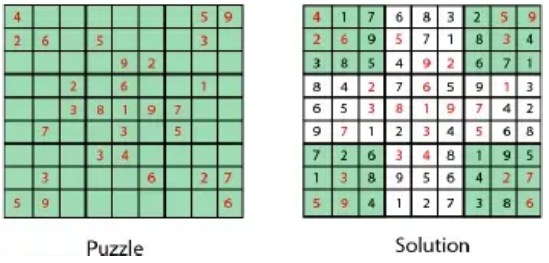
Figure 6.1 (a) The principal states and territories of Australia. Coloring this map can be viewed as a constraint satisfaction problem (CSP). The goal is to assign colors to each region so that no neighboring regions have the same color. (b) The map-coloring problem represented as a constraint graph.

In addition, CSP solvers can be faster than state-space searchers because the CSP solver can quickly eliminate large swatches of the search space. For example, once we have chosen {SA =blue} in the Australia problem, we can conclude that none of the five neighboring variables can take on the value blue. Without taking advantage of constraint propagation, a search procedure would have to consider 3^5 =243 assignments for the five neighboring variables; with constraint propagation we never have to consider blue as a value, so we have only 2^5 =32 assignments to look at, a reduction of 87%.

Another CSP are

- **Sudoku Playing:** The gameplay where the constraint is that no number from 0-9 can be repeated in the same row or column.

SUDOKU



Puzzle Solution

- **n-queen problem:** In n-queen problem, the constraint is that no queen should be placed either diagonally, in the same row or column.

Note: The n-queen problem is already discussed in Problem-solving in AI section.

- **Crossword:** In crossword problem, the constraint is that there should be the correct formation of the words, and it should be meaningful.

- **Cryptarithmetic Problem:** This problem has one most important constraint that is, we cannot assign a different digit to the same character. All digits should contain a unique alphabet.

$$\begin{array}{cccccc} & S & E & N & D \\ + & M & O & R & E \\ \hline M & O & N & E & Y \end{array}$$

Crypt arithmetic Problem in AI

Cryptarithmetic Problem is a type of constraint satisfaction problem where the game is about digits and its unique replacement either with alphabets or other symbols. In cryptarithmetic problem, the digits (0-9) get substituted by some possible alphabets or symbols. The task in cryptarithmetic problem is to substitute each digit with an alphabet to get the result arithmetically correct.

The rules or constraints on a cryptarithmetic problem are as follows:

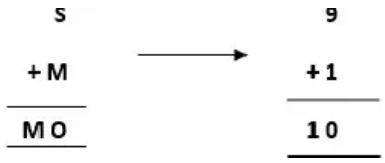
- There should be a unique digit to be replaced with a unique alphabet.
- The result should satisfy the predefined arithmetic rules, i.e., 2+2 =4, nothing else.
- Digits should be from **0-9** only.
- There should be only one carry forward, while performing the addition operation on a problem.
- The problem can be solved from both sides, i.e., **lefthand side (L.H.S), or righthand side (R.H.S)**

Let's understand the cryptarithmetic problem as well its constraints better with the help of an example:

- Given a cryptarithmetic problem, i.e., **S E N D + M O R E = M O N E Y**

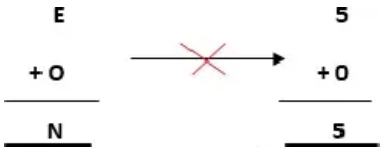
Follow the below steps to understand the given problem by breaking it into its subparts:

- Starting from the left hand side (L.H.S) , the terms are **S** and **M**. Assign a digit which could give a satisfactory result. Let's assign **S->9** and **M->1**.



Hence, we get a satisfactory result by adding up the terms and got an assignment for **O** as **O->0** as well.

- Now, move ahead to the next terms **E** and **O** to get **N** as its output.



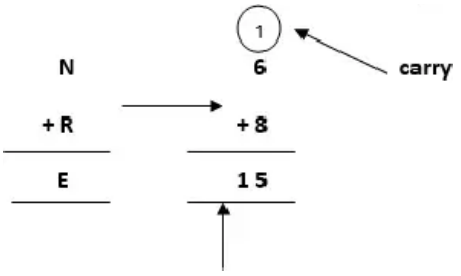
Adding E and O, which means 5+0=0, which is not possible because according to cryptarithmetic constraints, we cannot assign the same digit to two letters. So, we need to think more and assign some other value.

Note: When we will solve further, we will get one carry, so after applying it, the answer will be satisfied.

- Further, adding the next two terms **N** and **R** we get,

But, we have already assigned **E->5**. Thus, the above result does not satisfy the values because we are getting a different value for **E.** So, we need to think more.

Again, after solving the whole problem, we will get a carryover on this term, so our answer will be satisfied.



where 1 will be carry forward to the above term

Let's move ahead.

- Again, on adding the last two terms, i.e., the rightmost terms **D** and **E**, we get **Y** as its result.

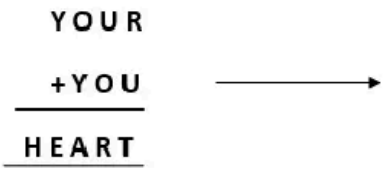
where 1 will be carry forward to the above term

- Keeping all the constraints in mind, the final resultant is as follows:

$$\begin{array}{cccccc} & S & E & N & D \\ + & M & O & R & E \\ \hline M & O & N & E & Y \end{array}$$

S	9
E	5
N	6
D	7
M	1
O	0
R	8
y	2

2.



Y	9
O	4
U	2
R	6
H	1
E	0
A	3
T	8

Topic: 8 :	CONSTRAINT PROPAGATION

Constraint propagation is the process of communicating the domain reduction of a decision variable to all of the constraints that are stated over this variable.

The basic step in constraint propagation is called **filtering a constraint**. As input, filtering takes a constraint and the current domains for its variables. It then tries to remove all the values in the domains that do not appear in any solution to that constraint. That is, it tries to reduce the domains so that *all the solutions are preserved*. Thus "constraint filtering" in CSPs corresponds to a "unit clause propagation" step in CNF SAT solvers, but it is usually more complex because the domains are larger and the constraints are more complex. Filtering on the global constraint level can also be more effective than unit clause propagation in pruning the search space.

Constraint propagation then means the process of applying filtering to the constraints in the CSP instance at hand. As filtering one constraint can reduce the domains of its variables, it can trigger further reduction when filtering another constraint that also involves the reduced-domain variables. Thus a constraint can be filtered multiple times during the constraint propagation process. Usually the process is run to the end, meaning until no more reduction happens in filtering any of the constraints. Or if this takes too long, then until some resource usage limit is exceeded.

In regular state-space search, an algorithm can do only one thing: search. In CSPs there is a choice: an algorithm can search (choose a new variable assignment from several possibilities) or do a specific type of **inference** called **constraint propagation**: using the constraints to reduce the number of legal values for a variable, which in turn can reduce the legal values for another variable, and so on. Constraint propagation may be intertwined with search, or it may be done as a preprocessing step, before search starts. Sometimes this preprocessing can solve the whole problem, so no search is required at all.

The key idea is **local consistency**. If we treat each variable as a node in a graph (see Figure 6.1(b)) and each binary constraint as an arc, then the process of enforcing local consistency in each part of the graph causes inconsistent values to be eliminated throughout the graph.

There are different types of local consistency, which we now cover in turn.

Node consistency

A single variable (corresponding to a node in the CSP network) is node-consistent if all the values in the variable's domain satisfy the variable's unary constraints. For example, in the variant of the Australia map-coloring problem (Figure 6.1) where South Australians dislike green, the variable SA starts with domain {red , green, blue}, and we can make it node consistent by eliminating green, leaving SA with the reduced domain

{red , blue}.We say that a network is node-consistent if every variable in the network is node-consistent.

6.2.2 Arc consistency

A variable in a CSP is **arc-consistent** if every value in its domain satisfies the variable's binary constraints. More formally, X_i is arc-consistent with respect to another variable X_j if for every value in the current domain D_i there is some value in the domain D_j that satisfies the binary constraint on the arc (X_i, X_j). A network is arc-consistent if every variable is arc consistent with every other variable. For example, consider the constraint $Y = X^2$ where the domain of both X and Y is the set of digits. We can write this constraint explicitly as

$$\langle (X, Y), \{(0,0), (1,1), (2,4), (3,9)\}\rangle .$$

6.2.3 Path consistency

Arc consistency can go a long way toward reducing the domains of variables, sometimes finding a solution (by reducing every domain to size 1) and sometimes finding that the CSP cannot be solved (by reducing some domain to size 0). But for other networks, arc consistency fails to make enough inferences. Consider the map-coloring problem on Australia, but with only two colors allowed, red and blue. Arc consistency can do nothing because every variable is already arc consistent: each can be red with blue at the other end of the arc (or vice versa). But clearly there is no solution to the problem: because Western Australia, Northern Territory and South Australia all touch each other, we need at least three colors for them alone.

Topic: 9 : BACKTRACKING SEARCH FOR CSPS

Sudoku problems are designed to be solved by inference over constraints. But many other CSPs cannot be solved by inference alone; there comes a time when we must search for a solution. In this section we look at backtracking search algorithms that work on partial assignments; in the next section we look at local search algorithms over complete assignments.

```
function BACKTRACKING-SEARCH(csp) returns a solution, or failure
  return BACKTRACK({}, csp)

function BACKTRACK(assignment, csp) returns a solution, or failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
      if value is consistent with assignment then
          add {var = value} to assignment
          inferences ← INFERENCE(csp, var, value)
          if inferences ≠ failure then
            add inferences to assignment
            result ← BACKTRACK(assignment, csp)
            if result ≠ failure then
              return result
      remove {var = value} and inferences from assignment
  return failure
```

Figure 6.5 A simple backtracking algorithm for constraint satisfaction problems. The algorithm is modeled on the recursive depth-first search of Chapter 3. By varying the functions SELECT-UNASSIGNED-VARIABLE and ORDER-DOMAIN-VALUES, we can implement the general-purpose heuristics discussed in the text. The function INFERENCE can optionally be used to impose arc-, path-, or k-consistency, as desired. If a value choice leads to failure (noticed either by INFERENCE or by BACKTRACK), then value assignments (including those made by INFERENCE) are removed from the current assignment and a new value is tried.

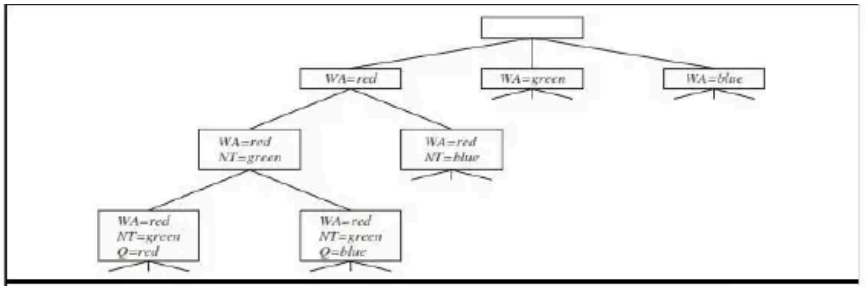


Figure 6.6 Part of the search tree for the map-coloring problem in Figure 6.1.

2. What inferences should be performed at each step in the search (INFERENCE)?
3. When the search arrives at an assignment that violates a constraint, can the search avoid repeating this failure?

Topic: 10 :	**LOCAL SEARCH FOR CSP**

Local search algorithms (see Section 4.1) turn out to be effective in solving many CSPs. They use a complete-state formulation: the initial state assigns a value to every variable, and the search changes the value of one variable at a time. For example, in the 8-queens problem (see Figure 4.3), the initial state might be a random configuration of 8 queens in 8 columns, and each step moves a single queen to a new position in its column. Typically, the initial guess violates several constraints. The point of local search is to eliminate the violated constraints. In choosing a new value for a variable, the most obvious heuristic is to select the value that results in the minimum number of conflicts with other variables—the min-conflicts heuristic. The algorithm is shown in Figure 6.8 and its application to an 8-queens problem is diagrammed in Figure 6.9.

```
function MIN-CONFLICTS(csp, max_steps) returns a solution or failure
  inputs: csp, a constraint satisfaction problem
          max_steps, the number of steps allowed before giving up

  current ← an initial complete assignment for csp
  for i = 1 to max_steps do
      if current is a solution for csp then return current
      var ← a randomly chosen conflicted variable from csp.VARIABLES
      value ← the value v for var that minimizes CONFLICTS(var, v, current, csp)
      set var = value in current
  return failure
```

Figure 6.8 The MIN-CONFLICTS algorithm for solving CSPs by local search. The initial state may be chosen randomly or by a greedy assignment process that chooses a minimal-conflict value for each variable in turn. The CONFLICTS function counts the number of constraints violated by a particular value, given the rest of the current assignment.

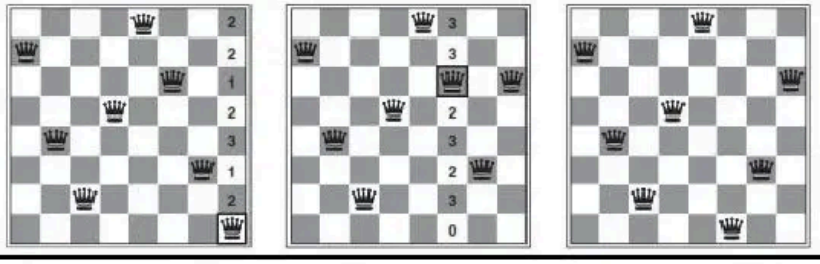


Figure 6.9 A two-step solution using min-conflicts for an 8-queens problem. At each stage, a queen is chosen for reassignment in its column. The number of conflicts (in this case, the number of attacking queens) is shown in each square. The algorithm moves the queen to the min-conflicts square, breaking ties randomly.

Topic: 11 :	THE STRUCTURE OF PROBLEMS

In this section, we examine ways in which the *structure* of the problem, as represented by the constraint graph, can be used to find solutions quickly. Most of the approaches here also apply to other problems besides CSPs, such as probabilistic reasoning. After all, the only way we can possibly hope to deal with the real world is to decompose it into many subproblems. Looking again at the constraint graph for Australia (Figure 6.1(b), repeated as Figure 6.12(a)), one fact stands out: Tasmania is not connected to the mainland.[3] Intuitively, it is obvious that coloring Tasmania and coloring the mainland are **independent subproblems**—any solution for the mainland combined with any solution for Tasmania yields a solution for the whole map. Independence can be ascertained simply by finding **connected components** of the constraint graph. Each component corresponds to a subproblem CSP_i. If assignment S_i is a solution of CSP_i, then $\bigcup_i S_i$ is a solution of $\bigcup_i CSP_i$. Why is this important? Consider the following: suppose each CSP_i has c variables from the total of n variables, where c is a constant. Then there are n/c subproblems, each of which takes at most d^c work to solve,

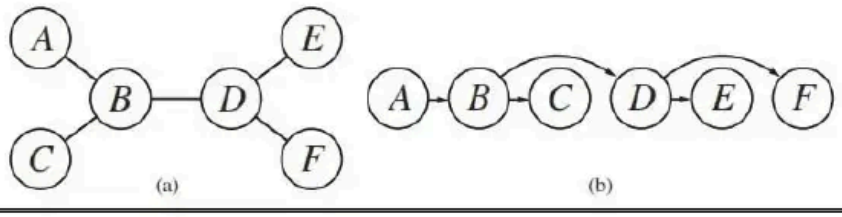


Figure 6.10 (a) The constraint graph of a tree-structured CSP. (b) A linear ordering of the variables consistent with the tree with A as the root. This is known as a **topological sort** of the variables.

function TREE-CSP-SOLVER(csp) **returns** a solution, or failure
inputs: csp, a CSP with components X, D, C

$n \leftarrow$ number of variables in X
$assignment \leftarrow$ an empty assignment
$root \leftarrow$ any variable in X
$X \leftarrow$ TOPOLOGICALSORT(X, $root$)
for $j = n$ **down to** 2 **do**
  MAKE-ARC-CONSISTENT(PARENT(X_j), X_j)
  if it cannot be made consistent **then return** *failure*
for $i = 1$ **to** n **do**
  $assignment[X_i] \leftarrow$ any consistent value from D_i
  if there is no consistent value **then return** *failure*
return $assignment$

Figure 6.11 The TREE-CSP-SOLVER algorithm for solving tree-structured CSPs. If the CSP has a solution, we will find it in linear time; if not, we will detect a contradiction.

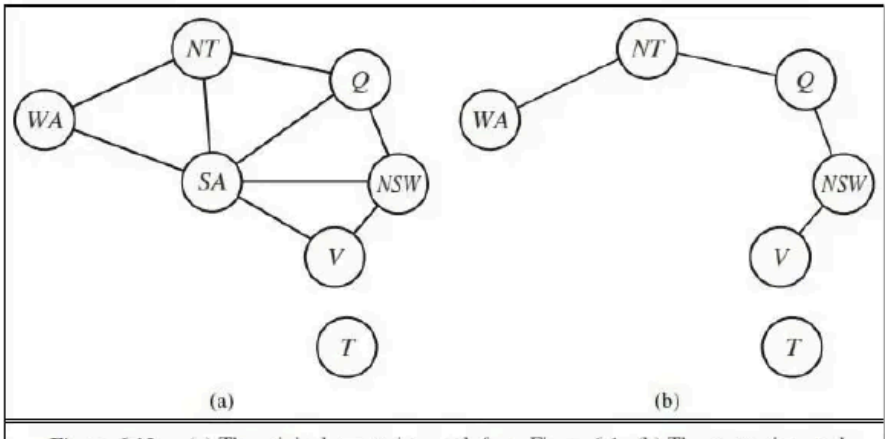


Figure 6.12 (a) The original constraint graph from Figure 6.1. (b) The constraint graph after the removal of SA.

The second approach is based on constructing a tree decomposition of the constraint graph into a set of connected subproblems. Each subproblem is solved independently, and the resulting solutions are then combined. Like most divide-and-conquer algorithms, this works well if no subproblem is too large. Figure 6.13 shows a tree decomposition of the mapcoloring problem into five subproblems.

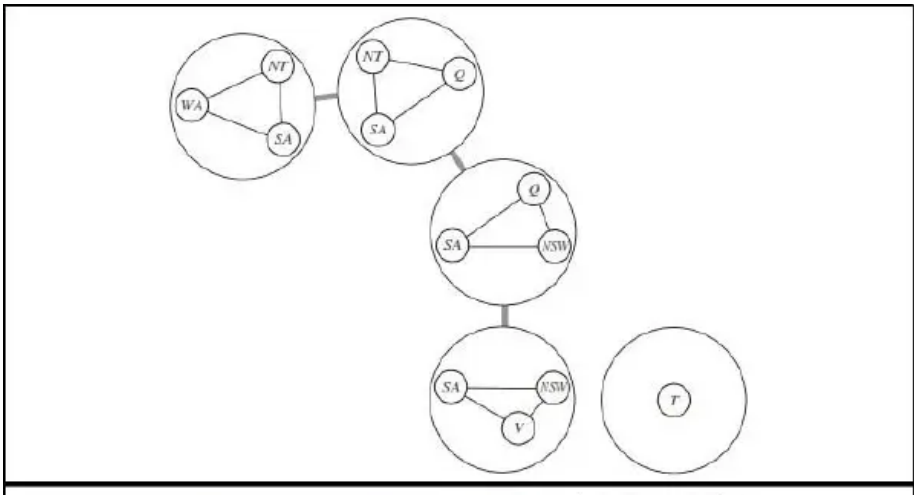


Figure 6.13 A tree decomposition of the constraint graph in Figure 6.12(a).

A tree decomposition must satisfy the following three requirements:

• Every variable in the original problem appears in at least one of the subproblems.
• If two variables are connected by a constraint in the original problem, they must appear together (along with the constraint) in at least one of the subproblems.
• If a variable appears in two subproblems in the tree, it must appear in every subproblem along the path connecting those subproblems.

The first two conditions ensure that all the variables and constraints are represented in the decomposition. The third condition seems rather technical, but simply reflects the constraint that any given variable must have the same value in every subproblem in which it appears; the links joining subproblems in the tree enforce this constraint. For example, SA appears in all four of the connected subproblems in Figure 6.13. You can verify from Figure 6.12 that this decomposition makes sense.

UNIT IV
LOGICAL REASONING

SYLLABUS

Knowledge-based agents – propositional logic – propositional theorem proving – propositional model checking – agents based on propositional logic. First-order logic – syntax and semantics – knowledge representation and engineering – inferences in first-order logic – forward chaining – backward chaining – resolution.

INTRODUCTION

Reasoning:

- ✓ The reasoning is the mental process of deriving logical conclusion and making predictions from available knowledge, facts, and beliefs. Or we can say, "**Reasoning is a way to infer facts from existing data**."
- ✓ It is a general process of thinking rationally, to find valid conclusions.
- ✓ In artificial intelligence, the reasoning is essential so that the machine can also think rationally as a human brain, and can perform like a human.
- ✓ When a system is required to do something, that it has not been explicitly told how to do,, it must figure out what it needs to know from what it already knows.

  Fact 1 : Robins are Birds
  Fact 2 : All birds have wings
  Question : Do Robins have wings?

- ✓ Hence Reasoning system, must find out, what it needs to know from what it already knows.
- ✓ Logic is a language of reasoning. It is a collection of rules called logic arguments, we use when doing logic reasoning.
- ✓ Logical Reasoning is a process of drawing conclusions from premises using rule of inference.

Topic: 1 :	KNOWLEDGE-BASED AGENTS

- An intelligent agent needs **knowledge** about the real world for taking decisions and **reasoning** to act efficiently.
- Knowledge-based agents are those agents who have the capability of **maintaining an internal state of knowledge, reason over that knowledge, update their knowledge after observations and take actions. These agents can represent the world with some formal representation and act intelligently**.
- Knowledge-based agents are composed of two main parts:
  - **Knowledge-base and**
  - **Inference system**.

A knowledge-based agent must able to do the following:

- An agent should be able to represent states, actions, etc.
- An agent Should be able to incorporate new percepts
- An agent can update the internal representation of the world
- An agent can deduce the internal representation of the world
- An agent can deduce appropriate actions.

The architecture of knowledge-based agent:

- ✓ The above diagram is representing a generalized architecture for a knowledge-based agent.
- ✓ The knowledge-based agent (KBA) take input from the environment by perceiving the environment.
- ✓ The input is taken by the inference engine of the agent and which also communicate with KB to decide as per the knowledge store in KB.

- ✓ The learning element of KBA regularly updates the KB by learning new knowledge.

Knowledge base:

- ✓ Knowledge-base is a central component of a knowledge-based agent, it is also known as KB.
- ✓ It is a collection of sentences (here 'sentence' is a technical term and it is not identical to sentence in English).
- ✓ These sentences are expressed in a language which is called a knowledge representation language.
- ✓ The Knowledge-base of KBA stores fact about the world.

Why use a knowledge base?

- ✓ Knowledge-base is required for updating knowledge for an agent to learn with experiences and take action as per the knowledge.

Inference system

- ✓ Inference means deriving new sentences from old.
- ✓ Inference system allows us to add a new sentence to the knowledge base.
- ✓ A sentence is a proposition about the world. Inference system applies logical rules to the KB to deduce new information.
- ✓ Inference system generates new facts so that an agent can update the KB. An inference system works mainly in two rules which are given as:
  - **Forward chaining**
  - **Backward chaining**

Operations Performed by KBA

Following are three operations which are performed by KBA in order to show the intelligent behavior:

1. **TELL:** This operation tells the knowledge base what it perceives from the environment.
2. **ASK:** This operation asks the knowledge base what action it should perform.
3. **Perform:** It performs the selected action.

Various levels of knowledge-based agent:

✓ A knowledge-based agent can be viewed at different levels which are given below:

1. Knowledge level

✓ Knowledge level is the first level of knowledge-based agent, and in this level, we need to specify what the agent knows, and what the agent goals are.

✓ With these specifications, we can fix its behavior.

✓ For example, suppose an automated taxi agent needs to go from a station A to station B, and he knows the way from A to B, so this comes at the knowledge level.

2. Logical level:

✓ At this level, we understand that how the knowledge representation of knowledge is stored.

✓ At this level, sentences are encoded into different logics.

✓ At the logical level, an encoding of knowledge into logical sentences occurs.

✓ At the logical level we can expect to the automated taxi agent to reach to the destination B.

3. Implementation level:

✓ This is the physical representation of logic and knowledge.

✓ At the implementation level agent perform actions as per logical and knowledge level.

✓ At this level, an automated taxi agent actually implements his knowledge and logic so that he can reach to the destination.

Approaches to designing a knowledge-based agent:

✓ There are mainly two approaches to build a knowledge-based agent:

1. Declarative approach:

✓ We can create a knowledge-based agent by initializing with an empty knowledge base and telling the agent all the sentences with which we want to start with.

✓ This approach is called Declarative approach.

2. Procedural approach:

- ✓ In the procedural approach, we directly encode desired behavior as a program code.
- ✓ Which means we just need to write a program that already encodes the desired behavior or agent.
- ✓ However, in the real world, a successful agent can be built by combining both declarative and procedural approaches, and declarative knowledge can often be compiled into more efficient procedural code.

Topic: 2 :	PROPOSITIONAL LOGIC

- ✓ Propositional logic (PL) is the simplest form of logic where all the statements are made by propositions.
- ✓ A proposition is a declarative statement which is either true or false.
- ✓ It is a technique of knowledge representation in logical and mathematical form

Example:

a) It is Sunday.
b) The Sun rises from West (False proposition)
c) 3+3= 7(False proposition)
d) 5 is a prime number.

Following are some basic facts about propositional logic:

- ✓ Propositional logic is also called Boolean logic as it works on 0 and 1.
- ✓ In propositional logic, we use symbolic variables to represent the logic, and we can use any symbol for a representing a proposition, such A, B, C, P, Q, R, etc.
- ✓ Propositions can be either true or false, but it cannot be both.
- ✓ Propositional logic consists of an object, relations or function, and **logical connectives**.
- ✓ These connectives are also called logical operators.
- ✓ The propositions and connectives are the basic elements of the propositional logic.
- ✓ Connectives can be said as a logical operator which connects two sentences.
- ✓ A proposition formula which is always true is called **tautology**, and it is also called a valid sentence.
- ✓ A proposition formula which is always false is called **Contradiction**.

- ✓ Statements which are questions, commands, or opinions are not propositions such as "**Where is Rohini**", "**How are you**", "**What is your name**", are not propositions.

Syntax of propositional logic:

- ✓ The syntax of propositional logic defines the allowable sentences for the knowledge representation.
- ✓ There are two types of Propositions:
  - **Atomic Propositions**
  - **Compound propositions**

Atomic Proposition:

- ✓ Atomic propositions are the simple propositions.
- ✓ It consists of a single proposition symbol.
- ✓ These are the sentences which must be either true or false.

Example:

a) 2+2 is 4, it is an atomic proposition as it is a **true** fact.

b) "The Sun is cold" is also a proposition as it is a **false** fact.

Compound proposition:

- Compound propositions are constructed by combining simpler or atomic propositions, using parenthesis and logical connectives.

Example:

a) "It is raining today, and street is wet."

b) "Ankit is a doctor, and his clinic is in Mumbai."

Logical Connectives:

- Logical connectives are used to connect two simpler propositions or representing a sentence logically.
- We can create compound propositions with the help of logical connectives.
- There are mainly five connectives, which are given as follows:

1. **Negation:** A sentence such as ¬ P is called negation of P. A literal can be either Positive literal or negative literal.
2. **Conjunction:** A sentence which has ∧ connective such as, **P ∧ Q** is called a conjunction.
   Example: Rohan is intelligent and hardworking. It can be written as,
   P= Rohan is intelligent,
   Q= Rohan is hardworking. → P∧ Q.
3. **Disjunction:** A sentence which has ∨ connective, such as **P ∨ Q.** is called disjunction, where P and Q are the propositions.
   Example: "Ritika is a doctor or Engineer",
   Here P= Ritika is Doctor. Q= Ritika is Doctor, so we can write it as **P ∨ Q.**
4. **Implication:** A sentence such as P → Q, is called an implication. Implications are also known as if-then rules. It can be represented as
   If it is raining, then the street is wet.
   Let P= It is raining, and Q= Street is wet, so it is represented as P → Q
5. **Biconditional:** A sentence such as **P⇔ Q is a Biconditional sentence, example**
   If I am breathing, then I am alive
   P= I am breathing, Q= I am alive, it can be represented as P ⇔ Q.

✓ Following is the summarized table for Propositional Logic Connectives:

Truth Table:

✓ In propositional logic, we need to know the truth values of propositions in all possible scenarios.

✓ We can combine all the possible combination with logical connectives, and the representation of these combinations in a tabular format is called **Truth table**.

✓ Following are the truth table for all logical connectives:

For Negation:

P	¬ P
True	**False**
False	**True**

For Conjunction:

P	Q	P∧ Q
True	True	**True**
True	False	**False**
False	True	**False**
False	False	**False**

For disjunction:

P	Q	P ∨ Q.
True	True	**True**
False	True	**True**
True	False	**True**
False	False	**False**

For Implication:

P	Q	P→ Q
True	True	**True**
True	False	**False**
False	True	**True**
False	False	**True**

Topic: 3 :	PROPOSITIONAL THEOREM PROVING

So far, we have shown how to determine entailment by *model checking*: enumerating models and showing that the sentence must hold in all models. In this section, we show how entailment can be done by **theorem proving**—applying rules of inference directly to the sentences in our knowledge base to construct a proof of the desired sentence without consulting models. If the number of models is large but the length of the proof is short, then theorem proving can be more efficient than model checking.

Before we plunge into the details of theorem-proving algorithms, we will need some additional concepts related to entailment. The first concept is **logical equivalence**: two sentences α and β are logically equivalent if they are true in the same set of models. We write this as $\alpha \equiv \beta$. For example, we can easily show (using truth tables) that $P \wedge Q$ and $Q \wedge P$ are logically equivalent; other equivalences are shown in Figure 7.11. These equivalences play much the same role in logic as arithmetic identities do in ordinary mathematics. An alternative definition of equivalence is as follows: any two sentences α and β are equivalent only if each of them entails the other:

$$\alpha \equiv \beta \quad \text{if and only if} \quad \alpha \models \beta \text{ and } \beta \models \alpha \,.$$

Logical equivalence:

- ✓ Logical equivalence is one of the features of propositional logic.
- ✓ Two propositions are said to be logically equivalent if and only if the columns in the truth table are identical to each other.
- ✓ Let's take two propositions A and B, so for logical equivalence, we can write it as A⇔B.

✓ In below truth table we can see that column for ¬A∨ B and A→B, are identical hence A is Equivalent to B

A	B	¬A	¬A∨ B	A→B
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Properties of Operators:

- **Commutativity:**
  - PA Q= Q A P, or
  - P ∨ Q = Q ∨ P.
- **Associativity:**
  - (P A Q) A R= P A (Q A R),
  - (P ∨ Q) ∨ R= P ∨ (Q ∨ R)
- **Identity element:**
  - P A True = P,
  - P ∨ True= True.
- **Distributive:**
  - PA (Q ∨ R) = (P A Q) ∨ (P A R).
  - P ∨ (Q A R) = (P ∨ Q) A (P ∨ R).
- **DE Morgan's Law:**
  - ¬ (P A Q) = (¬P) ∨ (¬Q)
  - ¬ (P ∨ Q) = (¬ P) A (¬Q).
- **Double-negation elimination:**
  - ¬ (¬P) = P.

Topic: 4 :	PROPOSITIONAL MODEL CHECKING

In this section, we describe two families of efficient algorithms for general propositional inference based on model checking: One approach based on backtracking search, and one on local hill-climbing search. These algorithms are part of the "technology" of propositional logic. This section can be skimmed on a first reading of the chapter

7.6.1 A complete backtracking algorithm

The first algorithm we consider is often called the **Davis–Putnam algorithm**, after the seminal paper by Martin Davis and Hilary Putnam (1960). The algorithm is in fact the version described by Davis, Logemann, and Loveland (1962), so we will call it DPLL after the initials of all four authors. DPLL takes as input a sentence in conjunctive normal form—a set of clauses. Like BACKTRACKING-SEARCH and TT-ENTAILS?, it is essentially a recursive, depth-first enumeration of possible models. It embodies three improvements over the simple scheme of TT-ENTAILS?:

- *Early termination*: The algorithm detects whether the sentence must be true or false, even with a partially completed model. A clause is true if *any* literal is true, even if the other literals do not yet have truth values; hence, the sentence as a whole could be judged true even before the model is complete. For example, the sentence $(A \vee B) \wedge (A \vee C)$ is true if A is true, regardless of the values of B and C. Similarly, a sentence is false if *any* clause is false, which occurs when each of its literals is false. Again, this can occur long before the model is complete. Early termination avoids examination of entire subtrees in the search space.
- *Pure symbol heuristic*: A **pure symbol** is a symbol that always appears with the same "sign" in all clauses. For example, in the three clauses $(A \vee \neg B)$, $(\neg B \vee \neg C)$, and $(C \vee A)$, the symbol A is pure because only the positive literal appears, B is pure because only the negative literal appears, and C is impure. It is easy to see that if a sentence has a model, then it has a model with the pure symbols assigned so as to make their literals *true*, because doing so can never make a clause false. Note that, in determining the purity of a symbol, the algorithm can ignore clauses that are already known to be true in the model constructed so far. For example, if the model contains $B = false$, then the clause $(\neg B \vee \neg C)$ is already true, and in the remaining clauses C appears only as a positive literal; therefore C becomes pure.

7.6.2 Local search algorithms

We have seen several local search algorithms so far in this book, including HILL-CLIMBING (page 122) and SIMULATED-ANNEALING (page 126). These algorithms can be applied directly to satisfiability problems, provided that we choose the right evaluation function. Because the goal is to find an assignment that satisfies every clause, an evaluation function that counts the number of unsatisfied clauses will do the job. In fact, this is exactly the measure used by the MIN-CONFLICTS algorithm for CSPs (page 221). All these algorithms take steps in the space of complete assignments, flipping the truth value of one symbol at a time. The space usually contains many local minima, to escape from which various forms of randomness are required. In recent years, there has been a great deal of experimentation to find a good balance between greediness and randomness.

One of the simplest and most effective algorithms to emerge from all this work is called WALKSAT (Figure 7.18). On every iteration, the algorithm picks an unsatisfied clause and picks a symbol in the clause to flip. It chooses randomly between two ways to pick which symbol to flip: (1) a "min-conflicts" step that minimizes the number of unsatisfied clauses in the new state and (2) a "random walk" step that picks the symbol randomly.

When WALKSAT returns a model, the input sentence is indeed satisfiable, but when it returns *failure*, there are two possible causes: either the sentence is unsatisfiable or we need to give the algorithm more time. If we set $max_flips = \infty$ and $p > 0$, WALKSAT will eventually return a model (if one exists), because the random-walk steps will eventually hit

Topic: 5 : AGENTS BASED ON PROPOSITIONAL LOGIC

The agent also knows that there is exactly one wumpus. This is expressed in two parts. First, we have to say that there is *at least one* wumpus:

$$W_{1,1} \vee W_{1,2} \vee \cdots \vee W_{4,3} \vee W_{4,4} .$$

Then, we have to say that there is *at most one* wumpus. For each pair of locations, we add a sentence saying that at least one of them must be wumpus-free:

$$\begin{aligned} &\neg W_{1,1} \vee \neg W_{1,2} \\ &\neg W_{1,1} \vee \neg W_{1,3} \\ &\cdots \\ &\neg W_{4,3} \vee \neg W_{4,4} . \end{aligned}$$

So far, so good. Now let's consider the agent's percepts. If there is currently a stench, one might suppose that a proposition $Stench$ should be added to the knowledge base. This is not quite right, however: if there was no stench at the previous time step, then $\neg Stench$ would already be asserted, and the new assertion would simply result in a contradiction. The problem is solved when we realize that a percept asserts something *only about the current time*. Thus, if the time step (as supplied to MAKE-PERCEPT-SENTENCE in Figure 7.1) is 4, then we add

7.7.2 A hybrid agent

as the successor-state axioms. (The next section explains why the agent doesn't need axioms for *future* time steps.) Then, the agent uses logical inference, by ASKing questions of the knowledge base, to work out which squares are safe and which have yet to be visited.

function HYBRID-WUMPUS-AGENT(*percept*) **returns** an *action*
  inputs: *percept*, a list, [*stench*,*breeze*,*glitter*,*bump*,*scream*]
  persistent: KB, a knowledge base, initially the atemporal "wumpus physics"
    t, a counter, initially 0, indicating time
    plan, an action sequence, initially empty

  TELL(KB, MAKE-PERCEPT-SENTENCE(*percept*, t))
  TELL the KB the temporal "physics" sentences for time t
  $safe \leftarrow \{[x, y] : \text{ASK}(KB, OK^t_{x,y}) = true\}$
  if ASK(KB, $Glitter^t$) $= true$ **then**
    $plan \leftarrow [Grab] + \text{PLAN-ROUTE}(current, \{[1,1]\}, safe) + [Climb]$
  if *plan* is empty **then**
    $unvisited \leftarrow \{[x, y] : \text{ASK}(KB, L^{t'}_{x,y}) = false \text{ for all } t' \leq t\}$
    $plan \leftarrow \text{PLAN-ROUTE}(current, unvisited \cap safe, safe)$
  if *plan* is empty and ASK(KB, $HaveArrow^t$) $= true$ **then**
    $possible_wumpus \leftarrow \{[x, y] : \text{ASK}(KB, \neg\, W_{x,y}) = false\}$
    $plan \leftarrow \text{PLAN-SHOT}(current, possible_wumpus, safe)$
  if *plan* is empty **then** // no choice but to take a risk
    $not_unsafe \leftarrow \{[x, y] : \text{ASK}(KB, \neg\, OK^t_{x,y}) = false\}$
    $plan \leftarrow \text{PLAN-ROUTE}(current, unvisited \cap not_unsafe, safe)$
  if *plan* is empty **then**
    $plan \leftarrow \text{PLAN-ROUTE}(current, \{[1, 1]\}, safe) + [Climb]$
  $action \leftarrow \text{POP}(plan)$
  TELL(KB, MAKE-ACTION-SENTENCE($action, t$))
  $t \leftarrow t + 1$
  return *action*

---

function PLAN-ROUTE(*current*,*goals*,*allowed*) **returns** an action sequence
  inputs: *current*, the agent's current position
    goals, a set of squares; try to plan a route to one of them
    allowed, a set of squares that can form part of the route

  problem $\leftarrow$ ROUTE-PROBLEM(*current*, *goals*,*allowed*)
  return A*-GRAPH-SEARCH(*problem*)

Figure 7.20 A hybrid agent program for the wumpus world. It uses a propositional knowledge base to infer the state of the world, and a combination of problem-solving search and domain-specific code to decide what actions to take.

by a unique binary number, we would need numbers with $\log_2(2^{2^n}) = 2^n$ bits to label the current belief state. That is, exact state estimation may require logical formulas whose size is exponential in the number of symbols.

One very common and natural scheme for *approximate* state estimation is to represent belief states as conjunctions of literals, that is, 1-CNF formulas. To do this, the agent program simply tries to prove X^t and $\neg X^t$ for each symbol X^t (as well as each atemporal symbol whose truth value is not yet known), given the belief state at $t-1$. The conjunction of

Topic: 6 :	FIRST-ORDER LOGIC

- ✓ First-order logic is symbolized reasoning in which each sentence, or statement, is broken down into a subject and a predicate.
- ✓ First-order logic can be useful in the creation of computer programs. It is also of interest to researchers in artificial intelligence.

Features of First-order Predicate Logic

- More expressive and powerful representation
- Allows us to represent almost any English sentences.
- Generalization of propositional logic.

First-Order logic:

- ✓ First-order logic is another way of knowledge representation in artificial intelligence. It is an extension to propositional logic.
- ✓ FOL is sufficiently expressive to represent the natural language statements in a concise way.
- ✓ First-order logic is also known as **Predicate logic or First-order predicate logic**. First-order logic is a powerful language that develops information about the objects in a more easy way and can also express the relationship between those objects.
- ✓ First-order logic (like natural language) does not only assume that the world contains facts like propositional logic but also assumes the following things in the world:
    - **Objects:** A, B, people, numbers, colors, wars, theories, squares, pits, wumpus, ...

- **Relations: It can be unary relation such as:** red, round, is adjacent, **or n-any relation such as:** the sister of, brother of, has color, comes between
- **Function:** Father of, best friend, third inning of, end of, ...

✓ As a natural language, first-order logic also has two main parts:

- **Syntax**
- **Semantics**

Syntax & Semantics

The **syntax** of propositional logic defines the allowable sentences. The **atomic sentences** consist of a single **proposition symbol**. Each such symbol stands for a proposition that can be true or false. We use symbols that start with an uppercase letter and may contain other letters or subscripts, for example: P, Q, R, $W_{1,3}$ and $North$. The names are arbitrary but are often chosen to have some mnemonic value—we use $W_{1,3}$ to stand for the proposition that the wumpus is in [1,3]. (Remember that symbols such as $W_{1,3}$ are *atomic*, i.e., W, 1, and 3 are not meaningful parts of the symbol.) There are two proposition symbols with fixed meanings: $True$ is the always-true proposition and $False$ is the always-false proposition. **Complex sentences** are constructed from simpler sentences, using parentheses and **logical connectives**. There are five connectives in common use:

$$
\begin{aligned}
\textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
\textit{AtomicSentence} &\rightarrow \textit{True} \mid \textit{False} \mid P \mid Q \mid R \mid \ldots \\
\textit{ComplexSentence} &\rightarrow (\textit{Sentence}) \mid [\textit{Sentence}] \\
&\mid \neg \textit{Sentence} \\
&\mid \textit{Sentence} \wedge \textit{Sentence} \\
&\mid \textit{Sentence} \vee \textit{Sentence} \\
&\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\
&\mid \textit{Sentence} \Leftrightarrow \textit{Sentence} \\
\text{OPERATOR PRECEDENCE} &: \neg, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{aligned}
$$

Figure 7.7 A BNF (Backus–Naur Form) grammar of sentences in propositional logic, along with operator precedences, from highest to lowest.

- *True* is true in every model and *False* is false in every model.
- The truth value of every other proposition symbol must be specified directly in the model. For example, in the model m_1 given earlier, $P_{1,2}$ is false.

For complex sentences, we have five rules, which hold for any subsentences P and Q in any model m (here "iff" means "if and only if"):

- $\neg P$ is true iff P is false in m.
- $P \wedge Q$ is true iff both P and Q are true in m.
- $P \vee Q$ is true iff either P or Q is true in m.
- $P \Rightarrow Q$ is true unless P is true and Q is false in m.
- $P \Leftrightarrow Q$ is true iff P and Q are both true or both false in m.

The rules can also be expressed with **truth tables** that specify the truth value of a complex sentence for each possible assignment of truth values to its components. Truth tables for the five connectives are given in Figure 7.8. From these tables, the truth value of any sentence s can be computed with respect to any model m by a simple recursive evaluation. For example,

Topic: 7 : KNOWLEDGE REPRESENTATION AND ENGINEERING

The preceding section illustrated the use of first-order logic to represent knowledge in three simple domains. This section describes the general process of knowledge-base construction— a process called **knowledge engineering**. A knowledge engineer is someone who investigates a particular domain, learns what concepts are important in that domain, and creates a formal representation of the objects and relations in the domain. We illustrate the knowledge engineering process in an electronic circuit domain that should already be fairly familiar, so that we can concentrate on the representational issues involved. The approach we take is suitable for developing *special-purpose* knowledge bases whose domain is carefully circumscribed and whose

range of queries is known in advance. General-purpose knowledge bases, which cover a broad range of human knowledge and are intended to support tasks such as natural language understanding

The knowledge-engineering process

Knowledge engineering projects vary widely in content, scope, and difficulty, but all such projects include the following steps:

1. *Identify the task.* The knowledge engineer must delineate the range of questions that the knowledge base will support and the kinds of facts that will be available for each specific problem instance. For example, does the wumpus knowledge base need to be able to choose actions or is it required to answer questions only about the contents of the environment? Will the sensor facts include the current location? The task will determine what knowledge must be represented in order to connect problem instances to answers. This step is analogous to the PEAS process for designing agents in Chapter 2.
2. *Assemble the relevant knowledge.* The knowledge engineer might already be an expert in the domain, or might need to work with real experts to extract what they know—a process called **knowledge acquisition**. At this stage, the knowledge is not represented formally. The idea is to understand the scope of the knowledge base, as determined by the task, and to understand how the domain actually works.

   For the wumpus world, which is defined by an artificial set of rules, the relevant knowledge is easy to identify. (Notice, however, that the definition of adjacency was not supplied explicitly in the wumpus-world rules.) For real domains, the issue of relevance can be quite difficult—for example, a system for simulating VLSI designs might or might not need to take into account stray capacitances and skin effects.

5. *Encode a description of the specific problem instance.* If the ontology is well thought out, this step will be easy. It will involve writing simple atomic sentences about instances of concepts that are already part of the ontology. For a logical agent, problem instances are supplied by the sensors, whereas a "disembodied" knowledge base is supplied with additional sentences in the same way that traditional programs are supplied with input data.
6. *Pose queries to the inference procedure and get answers.* This is where the reward is: we can let the inference procedure operate on the axioms and problem-specific facts to derive the facts we are interested in knowing. Thus, we avoid the need for writing an application-specific solution algorithm.
7. *Debug the knowledge base.* Alas, the answers to queries will seldom be correct on the first try. More precisely, the answers will be correct *for the knowledge base as written*, assuming that the inference procedure is sound, but they will not be the ones that the user is expecting. For example, if an axiom is missing, some queries will not be answerable from the knowledge base. A considerable debugging process could ensue. Missing axioms or axioms that are too weak can be easily identified by noticing places where the chain of reasoning stops unexpectedly. For example, if the knowledge base includes a diagnostic rule (see Exercise 8.13) for finding the wumpus,

$$\forall s \;\; Smelly(s) \;\Rightarrow\; Adjacent(Home(Wumpus), s)\,,$$

instead of the biconditional, then the agent will never be able to prove the *absence* of wumpuses. Incorrect axioms can be identified because they are false statements about the world. For example, the sentence

$$\forall x \;\; NumOfLegs(x, 4) \;\Rightarrow\; Mammal(x)$$

is false for reptiles, amphibians, and, more importantly, tables. *The falsehood of this sentence can be determined independently of the rest of the knowledge base.* In contrast,

Topic: 8 :	INFERENCES IN FIRST-ORDER LOGIC

Inference in First-Order Logic

- ✓ Inference in First-Order Logic is used to deduce new facts or sentences from existing sentences.
- ✓ Before understanding the FOL inference rule, let's understand some basic terminologies used in FOL.

Substitution:

- ✓ Substitution is a fundamental operation performed on terms and formulas.
- ✓ It occurs in all inference systems in first-order logic.
- ✓ The substitution is complex in the presence of quantifiers in FOL.
- ✓ If we write **F[a/x]**, so it refers to substitute a constant "**a**" in place of variable "**x**".

Equality:

- ✓ First-Order logic does not only use predicate and terms for making atomic sentences but also uses another way, which is equality in FOL.
- ✓ For this, we can use **equality symbols** which specify that the two terms refer to the same object.

Example: Brother (John) = Smith.

As in the above example, the object referred by the **Brother (John)** is similar to the object referred by **Smith**. The equality symbol can also be used with negation to represent that two terms are not the same objects.

Example: ¬(x=y) which is equivalent to x ≠y.

FOL inference rules for quantifier:

- ✓ As propositional logic we also have inference rules in first-order logic, so following are some basic inference rules in FOL:
  - **Universal Generalization**
  - **Universal Instantiation**
  - **Existential Instantiation**
  - **Existential introduction**

1. Universal Generalization:

- Universal generalization is a valid inference rule which states that if premise P(c) is true for any arbitrary element c in the universe of discourse, then we can have a conclusion as ∀ x P(x).
- It can be represented as: $\frac{P(c)}{\forall x\, P(x)}$.
- This rule can be used if we want to show that every element has a similar property.
- In this rule, x must not appear as a free variable.

Example: Let's represent, P(c): "**A byte contains 8 bits**", so for ∀ **x P(x)** "**All bytes contain 8 bits**.", it will also be true.

2. Universal Instantiation:

- Universal instantiation is also called as universal elimination or UI is a valid inference rule. It can be applied multiple times to add new sentences.
- The new KB is logically equivalent to the previous KB.
- As per UI, **we can infer any sentence obtained by substituting a ground term for the variable**.
- The UI rule state that we can infer any sentence P(c) by substituting a ground term c (a constant within domain x) from ∀ **x P(x) for any object in the universe of discourse**.
- It can be represented as: _____.
- **Example:1.**
- IF "Every person like ice-cream"=> ∀x P(x) so we can infer that "John likes ice-cream" => P(c)
- **Example: 2.**
- Let's take a famous example,
- "All kings who are greedy are Evil." So let our knowledge base contains this detail as in the form of FOL:

∀x king(x) A greedy (x) → Evil (x),

So from this information, we can infer any of the following statements using Universal Instantiation:

- **King(John) A Greedy (John) → Evil (John),**
- **King(Richard) A Greedy (Richard) → Evil (Richard),**

- **King(Father(John)) ∧ Greedy (Father(John)) → Evil (Father(John)),**

3. Existential Instantiation:

- Existential instantiation is also called as Existential Elimination, which is a valid inference rule in first-order logic.
- It can be applied only once to replace the existential sentence.
- The new KB is not logically equivalent to old KB, but it will be satisfiable if old KB was satisfiable.
- This rule states that one can infer P(c) from the formula given in the form of ∃x P(x) for a new constant symbol c.
- The restriction with this rule is that c used in the rule must be a new term for which P(c) is true.
- It can be represented as: $\frac{\exists x\, P(x)}{P(c)}$

Example:

From the given sentence: **∃x Crown(x) ∧ OnHead(x, John),**

So we can infer: **Crown(K) ∧ OnHead(K, John),** as long as K does not appear in the knowledge base.

- The above used K is a constant symbol, which is called **Skolem constant**.
- The Existential instantiation is a special case of **Skolemization process**.

4. Existential introduction

- An existential introduction is also known as an existential generalization, which is a valid inference rule in first-order logic.
- This rule states that if there is some element c in the universe of discourse which has a property P, then we can infer that there exists something in the universe which has the property P.
- It can be represented as:
- **Example: Let's say that,**
  "Priyanka got good marks in English."
  "Therefore, someone got good marks in English."

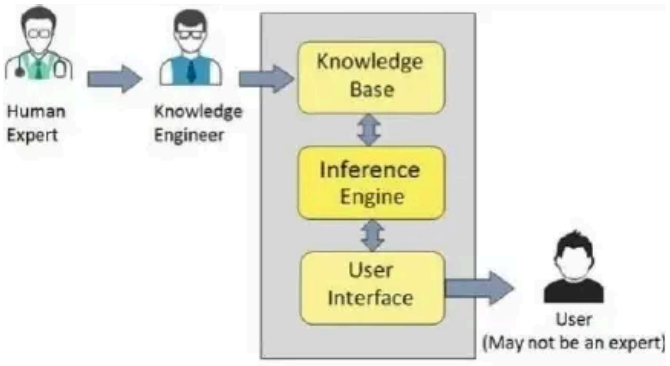
Topic: 9 :	FORWARD AND BACKWARD CHAINING

Forward and Backward Chaining in Artificial Intelligence

- ✓ Backward and forward chaining are methods of reasoning that exist in the Expert System Domain of artificial intelligence.
- ✓ These techniques are used in expert systems such as MYCIN and DENDRAL to generate solutions to real life problems.
- ✓ This article provides an overview of these techniques, and how they work. By the end of the article, readers will have learned real life examples of how backward and forward chaining are applied in artificial intelligence.

Introduction to the Expert System

- ✓ A brief overview of an expert system can help us gain more insights on the origin of backward and forward chaining in artificial intelligence.
- ✓ An expert system is a computer application that uses rules, approaches, and facts to provide solutions to complex problems.
- ✓ Examples of expert systems include MYCIN and DENDRAL.
- ✓ MYCIN uses the backward chaining technique to diagnose bacterial infections.
- ✓ DENDRAL employs forward chaining to establish the structure of chemicals.
- ✓ There are three components in an expert system:
    - user interface,
    - inference engine, and
    - knowledge base.
- ✓ The user interface enables users of the system to interact with the expert system.
- ✓ High-quality and domain-specific knowledge is stored in the knowledge base.
- ✓ Backward and forward chaining stem from the inference engine component.
- ✓ This is a component in which logical rules are applied to the knowledge base to get new information or make a decision.
- ✓ The backward and forward chaining techniques are used by the inference engine as strategies for proposing solutions or deducing information in the expert system.



Forward chaining

Forward chaining is a method of reasoning in artificial intelligence in which inference rules are applied to existing data to extract additional data until an endpoint (goal) is achieved.

In this type of chaining, the inference engine starts by evaluating existing facts, derivations, and conditions before deducing new information. An endpoint (goal) is achieved through the manipulation of knowledge that exists in the knowledge base.

forward chaining can be used in planning, monitoring, controlling, and interpreting applications.

Properties of forward chaining

- The process uses a down-up approach (bottom to top).
- It starts from an initial state and uses facts to make a conclusion.
- This approach is data-driven.
- It's employed in expert systems and production rule system.

Examples of forward chaining

A simple example of forward chaining can be explained in the following sequence.

A

A->B

B

A is the starting point. A->B represents a fact.

This fact is used to achieve a decision B.

A practical example will go as follows;

Tom is running (A)

If a person is running, he will sweat (A->B)

Therefore, Tom is sweating. (B)

A DENDRAL expert system is a good example of how forward chaining is used in artificial intelligence.

- ✓ DENDRAL is used in the prediction of the molecular structure of substances.
- ✓ Deducing the chemical structure starts by finding the number of atoms in every molecule.
- ✓ The mass spectrum of the sample is then used to establish the arrangement of the atoms.
- ✓ We can summarize these steps as follows.
  - The chemical formula is determined (the number of atoms in every molecule).
  - The spectrum machine is used to form mass spectrums of the sample.
  - The isomer and structure of the chemical are identified.
- ✓ In this example, the identification of the chemical structure is the endpoint.
- ✓ In the DENDRAL expert system, a generate and test technique is employed.
- ✓ There are two elements in the generator:
  - a synthesizer and
  - structural enumerator.
- ✓ The synthesizer plays the role of producing the mass spectrum.
- ✓ The structural enumerator identifies the structure of substances and prevents redundancy in the generator.

Advantages

- It can be used to draw multiple conclusions.

- It provides a good basis for arriving at conclusions.
- It's more flexible than backward chaining because it does not have a limitation on the data derived from it.

Disadvantages

- The process of forward chaining may be time-consuming. It may take a lot of time to eliminate and synchronize available data.
- Unlike backward chaining, the explanation of facts or observations for this type of chaining is not very clear. The former uses a goal-driven method that arrives at conclusions efficiently.

Backward chaining

✓ Backward chaining is a concept in artificial intelligence that involves backtracking from the endpoint or goal to steps that led to the endpoint.

✓ This type of chaining starts from the goal and moves backward to comprehend the steps that were taken to attain this goal.

✓ The backtracking process can also enable a person establish logical steps that can be used to find other important solutions.

✓ Backward chaining can be used in debugging, diagnostics, and prescription applications.

Properties of backward chaining

- The process uses an up-down approach (top to bottom).
- It's a goal-driven method of reasoning.
- The endpoint (goal) is subdivided into sub-goals to prove the truth of facts.
- A backward chaining algorithm is employed in inference engines, game theories, and complex database systems.
- The modus ponens inference rule is used as the basis for the backward chaining process. This rule states that if both the conditional statement (p->q) and the antecedent (p) are true, then we can infer the subsequent (q).

Example of backward chaining

The information provided in the previous example (forward chaining) can be used to provide a simple explanation of backward chaining. Backward chaining can be explained in the following sequence.

B

A->B

A

- ✓ B is the goal or endpoint, that is used as the starting point for backward tracking.
- ✓ A is the initial state. A->B is a fact that must be asserted to arrive at the endpoint B.
- ✓ A practical example of backward chaining will go as follows:

  Tom is sweating (B).

  If a person is running, he will sweat (A->B).

  Tom is running (A).

- ✓ The MYCIN expert system is a real life example of how backward chaining works.
- ✓ This is a system that's used in the diagnosis of bacterial infections.
- ✓ It also recommends suitable treatments for this type of infections.
- ✓ The knowledge base of a MYCIN comprises many antecedent-consequent rules, that enable the system to recognize various causes of (bacterial) infections.
- ✓ This system is suitable for patients who have a bacterial infection, but don't know the specific infection.
- ✓ The system will gather information relating to symptoms and history of the patient. It will then analyze this information to establish the bacterial infection.
- ✓ A suitable sequence can be as follows:
  - The patient has a bacterial infection.
  - The patient is vomiting.

- He/she is also experiencing diarrhea and severe stomach upset.
- Therefore, the patient has typhoid (salmonella bacterial infection).

✓ The MYCIN expert system uses the information collected from the patient to recommend suitable treatment.

✓ The recommended treatment corresponds to the identified bacterial infection. In the case above, the system may recommend the use of ciprofloxacin.

Advantages

- The result is already known, which makes it easy to deduce inferences.
- It's a quicker method of reasoning than forward chaining because the endpoint is available.
- In this type of chaining, correct solutions can be derived effectively if pre-determined rules are met by the inference engine.

Disadvantages

- The process of reasoning can only start if the endpoint is known.
- It doesn't deduce multiple solutions or answers.
- It only derives data that is needed, which makes it less flexible than forward chaining.

Thus,

✓ Backward and forward chaining are important methods of reasoning in artificial intelligence.

✓ These concepts differ mainly in terms of approach, strategy, technique, speed, and operational direction.

✓ Forward chaining is important to developers that want to use data-driven algorithms to develop effective computer-based systems.

✓ Backward chaining is important to developers that are interested in using goal-driven algorithms to design effective solutions in complex database systems.

Topic: 10 :	RESOLUTION

- ✓ Resolution is a theorem proving technique that proceeds by building refutation proofs, i.e., proofs by contradictions.
- ✓ It was invented by a Mathematician John Alan Robinson in the year 1965.
- ✓ Resolution is used, if there are various statements are given, and we need to prove a conclusion of those statements.
- ✓ Unification is a key concept in proofs by resolutions. Resolution is a single inference rule which can efficiently operate on the **conjunctive normal form or clausal form**.

Clause: Disjunction of literals (an atomic sentence) is called a **clause**. It is also known as a unit clause.

Conjunctive Normal Form: A sentence represented as a conjunction of clauses is said to be **conjunctive normal form** or **CNF**.

Steps for Resolution:

1. Conversion of facts into first-order logic.
2. Convert FOL statements into CNF
3. Negate the statement which needs to prove (proof by contradiction)
4. Draw resolution graph (unification).

To better understand all the above steps, we will take an example in which we will apply resolution.

Example:

- John likes all kind of food.
- Apple and vegetable are food
- Anything anyone eats and not killed is food.
- Anil eats peanuts and still alive
- Harry eats everything that Anil eats.

Prove by resolution that:

- John likes peanuts.

Step-1: Conversion of Facts into FOL

In the first step we will convert all the given statements into its first order logic.

a. ∀x: food(x) → likes(John, x)

b. food(Apple) ∧ food(vegetables)

c. ∀x ∀y: eats(x, y) ∧ ¬ killed(x) → food(y)

d. eats (Anil, Peanuts) ∧ alive(Anil).

e. ∀x : eats(Anil, x) → eats(Harry, x)

f. ∀x: ¬ killed(x) → alive(x) } **added predicates.**

g. ∀x: alive(x) →¬ killed(x) }

h. likes(John, Peanuts)

Step-2: Conversion of FOL into CNF

In First order logic resolution, it is required to convert the FOL into CNF as CNF form makes easier for resolution proofs.

Eliminate all implication (→) and rewrite

a. ∀x ¬ food(x) V likes(John, x)
b. food(Apple) Λ food(vegetables)
c. ∀x ∀y ¬ [eats(x, y) Λ ¬ killed(x)] V food(y)
d. eats (Anil, Peanuts) Λ alive(Anil)
e. ∀x ¬ eats(Anil, x) V eats(Harry, x)
f. ∀x¬ [¬ killed(x)] V alive(x)
g. ∀x ¬ alive(x) V ¬ killed(x)
h. likes(John, Peanuts).

Move negation (¬)inwards and rewrite

- ∀x ¬ food(x) V likes(John, x)
- food(Apple) Λ food(vegetables)
- ∀x ∀y ¬ eats(x, y) V killed(x) V food(y)
- eats (Anil, Peanuts) Λ alive(Anil)
- ∀x ¬ eats(Anil, x) V eats(Harry, x)
- ∀x ¬killed(x)] V alive(x)
- ∀x ¬ alive(x) V ¬ killed(x)
- likes(John, Peanuts).

Rename variables or standardize variables

- ∀x ¬ food(x) V likes(John, x)
- food(Apple) Λ food(vegetables)
- ∀y ∀z ¬ eats(y, z) V killed(y) V food(z)
- eats (Anil, Peanuts) Λ alive(Anil)
- ∀w¬ eats(Anil, w) V eats(Harry, w)
- ∀g ¬killed(g)] V alive(g)
- ∀k ¬ alive(k) V ¬ killed(k)
- likes(John, Peanuts).

Step-3: Negate the statement to be proved

In this statement, we will apply negation to the conclusion statements, which will be written as ¬likes(John, Peanuts)

Step-4: Draw Resolution graph:

Now in this step, we will solve the problem by resolution tree using substitution. For the above problem, it will be given as follows:

Hence the negation of the conclusion has been proved as a complete contradiction with the given set of statements.

UNIT V
PROBABILISTIC REASONING

SYLLABUS

Acting under uncertainty – Bayesian inference – naïve Bayes models.Probabilistic reasoning –Bayesian networks – exact inference in BN – approximate inference in BN – causal networks.

INTRODUCTION

- ✓ In Artificial Intelligence and Cognitive Science, probabilistic approaches are critical for generating, reasoning, and making decisions (simple and complex).
- ✓ Probabilistic reasoning is a form of knowledge representation in which the concept of probability is used to indicate the degree of uncertainty in knowledge.
- ✓ In AI, probabilistic models are used to examine data using statistical codes.
- ✓ It was one of the first machine learning methods.
- ✓ To this day, it's still widely used. The Naive Bayes algorithm is one of the most well-known algorithms in this group.

How does it work?

- ✓ Probabilistic modeling provides a framework for accepting the concept of learning.
- ✓ The probabilistic framework specifies how to express and deploy model reservations.
- ✓ In scientific data analysis, predictions play a significant role. Machine learning, automation, cognitive computing, and artificial intelligence all rely heavily on them.
- ✓ AI makes use of probabilistic reasoning:
- ✓ When we are uncertain about the premises
- ✓ When the number of possible predicates becomes unmanageable
- ✓ When it is known that an experiment contains an error

How does knowledge support reasoning?

- ✓ Simplifying any realistic domain requires some simplifications.
- ✓ The act of preparing knowledge to support reasoning necessitates the omission of numerous facts, their omission or their crude summarization.

- ✓ Rather than ignoring or enumerating exceptions, an alternative is to summaries them, i.e., to provide some warning signs indicating which areas of the minefield are more dangerous than others.
- ✓ Summarization is critical for striking a reasonable balance between safety and movement speed.
- ✓ One way to summaries exceptions is to assign a numerical measure of uncertainty to each proposition and then combine these measures using uniform syntactic principles, similar to how truth values are combined in logic.
- ✓ AI is a computational representation of intelligent behavior and common sense reasoning.
- ✓ Probability theory provides a logical explanation for how belief should change in the presence of incomplete or uncertain information. .
- ✓ AI systems are not unfamiliar with network representations.
- ✓ Thus, we need uncertain reasoning or probabilistic reasoning to represent uncertain knowledge.

Causes of uncertainty:

The following are some of the most common sources of uncertainty in the real world:

- The information came from unreliable sources.
- Errors in Experimental Design
- Equipment failure
- Temperature variations
- Climate change

Why Is Probabilistic Reasoning Necessary in AI?

- when the outcome is unpredictable.
- when the specification or set of possible predicates becomes unmanageable,
- when an experiment encounters an unknown error.

- ✓ In probabilistic reasoning, there are two ways to solve problems involving uncertain knowledge:
  - Bayes' rule
  - Bayesian Statistics

- ✓ Numerous problems in AI (reasoning, planning, learning, perception, and robotics) necessitate that the agent operates with incomplete or uncertain information.

- ✓ Using methods from probability theory and economics, AI researchers have developed a number of powerful tools for resolving these problems.
- ✓ Bayesian networks are a very versatile tool that can be used to solve a variety of problems, including reasoning (via the Bayesian inference algorithm), learning (via the expectation maximization algorithm), planning (via decision networks), and perception (using dynamic Bayesian networks).
- ✓ Probabilistic algorithms can also be used to filter, predict, smooth, and find explanations for data streams, assisting perception systems in analyzing time-dependent processes (e.g., hidden Markov models or Kalman filters).

Topic: 1 :	ACTING UNDER UNCERTAINTY

Uncertainty:

- ✓ Till now, we have learned knowledge representation using first-order logic and propositional logic with certainty, which means we were sure about the predicates.
- ✓ With this knowledge representation, we might write A→B, which means if A is true then B is true, but consider a situation where we are not sure about whether A is true or not then we cannot express this statement, this situation is called uncertainty.
- ✓ So to represent uncertain knowledge, where we are not sure about the predicates, we need uncertain reasoning or probabilistic reasoning.

Probabilistic reasoning:

- ✓ Probabilistic reasoning is a way of knowledge representation where we apply the concept of probability to indicate the uncertainty in knowledge.
- ✓ In probabilistic reasoning, we combine probability theory with logic to handle the uncertainty.
- ✓ We use probability in probabilistic reasoning because it provides a way to handle the uncertainty that is the result of someone's laziness and ignorance.
- ✓ Agents may need to handle **uncertainty**, whether due to partial observability, non determinism, or a combination of the two.
- ✓ An agent may never know for certain what state it's in or where it will end up after a sequence of actions.
- ✓ Problem-solving agents and Logical agents designed to handle uncertainty by keeping track of a **belief state**—a representation of the set of all possible world

states that it might be in—and generating a contingency plan that handles every possible eventuality that its sensors may report during execution.

- ✓ Despite its many virtues, however, this approach has significant drawbacks when taken literally as a recipe for creating agent programs:

  - When interpreting partial sensor information, a logical agent must consider *every logically possible* explanation for the observations, no matter how unlikely. This leads to impossible large and complex belief-state representations.
  - A correct contingent plan that handles every eventuality can grow arbitrarily large and must consider arbitrarily unlikely contingencies.
  - Sometimes there is no plan that is guaranteed to achieve the goal—yet the agent must act. It must have some way to compare the merits of plans that are not guaranteed.

Suppose, for example, that an automated taxi, automated has the goal of delivering a passenger, that involves leaving home 90 minutes before the flight departs and driving at a reasonable speed. Even though the airport is only about 5 miles away, a logical taxi agent will not be able to conclude with certaintythat "Plan A_{90} get us to the airport on time. Instead, it reaches the weaker conclusion. "Plan A will get us to the airport in time, as long as the car doesn't break down or run out of gas, and I don't get into an accident, and there are no accidents on the bridge, and the plane doesn't leave early, and no meteorite hits the car, and ... ." None of these conditions can be deduced for sure, so the plan's success cannot be inferred. This is the **qualification problem,** for which we so far have seen no real solution.

- ✓ Let's consider an example of uncertain reasoning: diagnosing a dental patient's toothache. Diagnosis—whether for medicine, automobile repair, or whatever—almost always involves uncertainty.
- ✓ Let us try to write rules for dental diagnosis using propositional logic, so that we can see how the logical approach breaks down.
- ✓ Consider the following simple rule:

Toothache ⇒Cavity .

- ✓ The problem is that this rule is wrong. Not all patients with toothaches have cavities; some of them have gum disease, an abscess, or one of several other problems:

Toothache ⇒ Cavity ∨GumProblem∨Abscess ...

- ✓ Unfortunately, in order to make the rule true, we have to add an almost unlimited list of possible problems. We could try turning the rule into a causal rule:

Cavity ⇒Toothache .

- ✓ But this rule is not right either; not all cavities cause pain.
- ✓ The only way to fix the rule is to make it logically exhaustive: to augment the left-hand side with all the qualifications required for a cavity to cause a toothache.
- ✓ Trying to use logic to cope with a domain like medical diagnosis thus fails for three main reasons:

Laziness:

- It is too much work to list the complete set of antecedents or consequents needed to ensure an exception less rule and too hard to use such rules.

Theoretical ignorance:

- Medical science has no complete theory for the domain.

Practical ignorance:

- Even if we know all the rules, we might be uncertain about particular patient because not all the necessary tests have been or can be run.

- ✓ The connection between toothaches and cavities is just not a logical consequence in either direction.
- ✓ This is typical of the medical domain, as well as most other judgmental domains:
- ✓ law, business, design, automobile repair, gardening, dating, and so on.
- ✓ The agent's knowledge can at best provide only a **degree of belief** in the relevant sentences.
- ✓ Our main tool for dealing with degrees of belief is **probability theory**.
- ✓ Probability provides a way of **summarizing** the uncertainty that comes from our laziness and ignorance, thereby solving the qualification problem.

- ✓ We might not know for sure what afflicts a particular patient, but we believe that there is, say, an 80% chance—that is, a probability of 0.8—that the patient who has a toothache has a cavity.
- ✓ That is, we expect that out of all the situations that are indistinguishable from the current situation as far as our knowledge goes, the patient will have a cavity in 80% of them.
- ✓ This belief could be derived from statistical data—80% of the toothache patients seen so far have had cavities—or from some general dental knowledge, or from a combination of evidence sources.

- ✓ Consider again the A plan for getting to the airport. Suppose it gives us a 97% chance of catching our flight.
- ✓ Does this mean it is a rational choice? Not necessarily: there might be other plans, such as A_{90}, with higher probabilities.
- ✓ If it is vital not to miss the flight, then it is worth risking the longer wait at the airport.
- ✓ What about A_{1440} , a plan that involves leaving home 24 hours in advance?
- ✓ In most circumstances, this is not a good choice, because although it almost guarantees getting there on time, it involves an intolerable wait—not to mention a possibly unpleasant diet of airport food.
- ✓ To make such choices, an agent must first have **preferences** between the different possible **outcomes** of the various plans.
- ✓ An outcome is a completely specified state, including such factors as whether the agent arrives on time and the length of the wait at the airport.
- ✓ Preferences, as expressed by utilities, are combined with probabilities in the generaltheory of rational decisions called **decision theory**:

Decision theory = probability theory + utility theory .

- ✓ The fundamental idea of decision theory is that an agent is rational if and only if it choosesthe action that yields the highest expected utility, averaged over all the possible outcomesof the action.
- ✓ This is called the principle of **maximum expected utility** (MEU).

Topic: 2 :	BAYESIAN INFERENCE

- ✓ Bayes' theorem is also known as **Bayes' rule, Bayes' law**, or **Bayesian reasoning**, which determines the probability of an event with uncertain knowledge.
- ✓ In probability theory, it relates the conditional probability and marginal probabilities of two random events.
- ✓ Bayes' theorem was named after the British mathematician **Thomas Bayes**. The **Bayesian inference** is an application of Bayes' theorem, which is fundamental to Bayesian statistics.
- ✓ It is a way to calculate the value of P(B|A) with the knowledge of P(A|B).

- ✓ Bayes' theorem allows updating the probability prediction of an event by observing new information of the real world.
- ✓ **Example**: If cancer corresponds to one's age then by using Bayes' theorem, we can determine the probability of cancer more accurately with the help of age.
- ✓ Bayes' theorem can be derived using product rule and conditional probability of event A with known event B:
- ✓ As from product rule we can write:
  1. $P(A \wedge B) = P(A|B)\ P(B)$ or
- ✓ Similarly, the probability of event B with known event A:

$$P(A \wedge B) = P(B|A)\ P(A)$$

- ✓ Equating right hand side of both the equations, we will get:
- ✓ The above equation (a) is called as **Bayes' rule** or **Bayes' theorem**. This equation is basic of most modern AI systems for **probabilistic inference**.
- ✓ It shows the simple relationship between joint and conditional probabilities.
- ✓ Here, P(A|B) is known as **posterior**, which we need to calculate, and it will be read as Probability of hypothesis A when we have occurred an evidence B.
- ✓ P(B|A) is called the likelihood, in which we consider that hypothesis is true, then we calculate the probability of evidence.
- ✓ P(A) is called the **prior probability**, probability of hypothesis before considering the evidence
- ✓ P(B) is called **marginal probability**, pure probability of an evidence.

- ✓ In the equation (a), in general, we can write P (B) = P(A)*P(B|Ai), hence the Bayes' rule can be written as:

$$P(A_i|B) = \frac{P(A_i)*P(B|A_i)}{\sum_{i=1}^{k} P(A_i)*P(B|A_i)}$$

Where A_1, A_2, A_3, ..., A_n is a set of mutually exclusive and exhaustive events.

Applying Bayes' rule:

- ✓ Bayes' rule allows us to compute the single term P(B|A) in terms of P(A|B), P(*B*), and P(A).
- ✓ This is very useful in cases where we have a good probability of these three terms and want to determine the fourth one.
- ✓ Suppose we want to perceive the effect of some unknown cause, and want to compute that cause, then the Bayes' rule becomes:

Application of Bayes' theorem in Artificial intelligence:

Following are some applications of Bayes' theorem:

- It is used to calculate the next step of the robot when the already executed step is given.
- Bayes' theorem is helpful in weather forecasting.
- It can solve the Monty Hall problem.

Topic: 3 : NAÏVE BAYES MODEL

The Naïve Bayes algorithm is comprised of two words Naïve and Bayes, Which can be described as:

- **Naïve**: It is called Naïve because it assumes that the occurrence of a certain feature is independent of the occurrence of other features. Such as if the fruit is identified on the bases of color, shape, and taste, then red, spherical, and sweet fruit is recognized as an apple. Hence each feature individually contributes to identify that it is an apple without depending on each other.
- **Bayes**: It is called Bayes because it depends on the principle of Bayes' Theorem.

Bayes' Theorem:

- Bayes' theorem is also known as **Bayes' Rule** or **Bayes' law**, which is used to determine the probability of a hypothesis with prior knowledge.
- It depends on the conditional probability.
- The formula for Bayes' theorem is given as:

$$P(A|B)=\frac{P(B|A)P(A)}{P(B)}$$

What is Naive Bayes algorithm?

- ✓ It is a classification technique based on Bayes' Theorem with an assumption of independence among predictors. In simple terms, a Naive Bayes classifier assumes that the presence of a particular feature in a class is unrelated to the presence of any other feature.
- ✓ For example, a fruit may be considered to be an apple if it is red, round, and about 3 inches in diameter.
- ✓ Even if these features depend on each other or upon the existence of the other features, all of these properties independently contribute to the probability that this fruit is an apple and that is why it is known as 'Naive'.
- ✓ Naive Bayes model is easy to build and particularly useful for very large data sets. Along with simplicity, Naive Bayes is known to outperform even highly sophisticated classification methods.
- ✓ Bayes theorem provides a way of calculating posterior probability P(c|x) from P(c), P(x) and P(x|c). Look at the equation below:

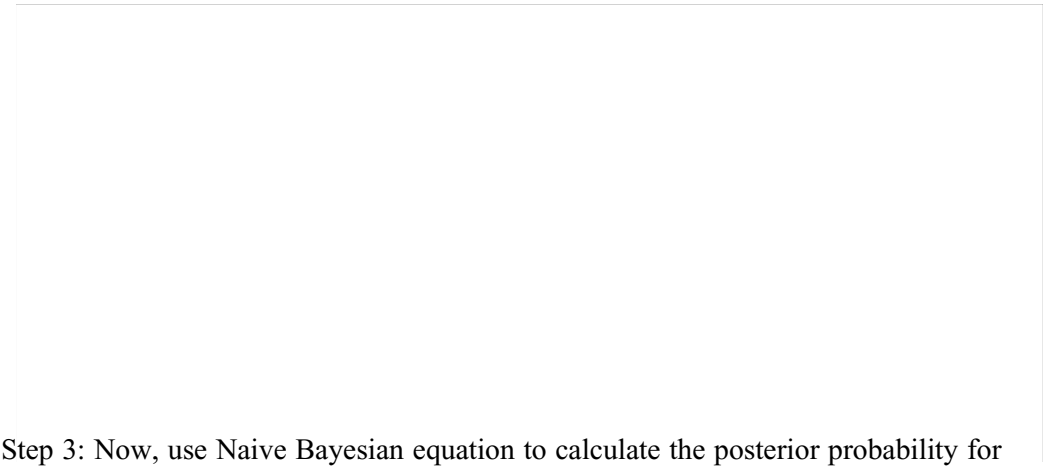
Above,

- $P(c|x)$ is the posterior probability of *class* (c, *target*) given *predictor* (x, *attributes*).
- $P(c)$ is the prior probability of *class*.

- $P(x|c)$ is the likelihood which is the probability of *predictor* given *class*.
- $P(x)$ is the prior probability of *predictor*.

How Naive Bayes algorithm works?

- ✓ Let's understand it using an example. Below I have a training data set of weather and corresponding target variable 'Play' (suggesting possibilities of playing).
- ✓ Now, we need to classify whether players will play or not based on weather condition. Let's follow the below steps to perform it.
- ✓ Step 1: Convert the data set into a frequency table
- ✓ Step 2: Create Likelihood table by finding the probabilities like Overcast probability = 0.29 and probability of playing is 0.64



Step 3: Now, use Naive Bayesian equation to calculate the posterior probability for each class. The class with the highest posterior probability is the outcome of prediction.

Problem: Players will play if weather is sunny. Is this statement is correct?
We can solve it using above discussed method of posterior probability.

P(Yes | Sunny) = P(Sunny | Yes) * P(Yes) / P (Sunny)
Here we have **P(Sunny |Yes)** = 3/9 = 0.33,
P(Sunny) = 5/14 = 0.36,
P(Yes)= 9/14 = 0.64
Now,
P (Yes | Sunny) = 0.33 * 0.64 / 0.36 = 0.60,

which has higher probability.

Naive Bayes uses a similar method to predict the probability of different class based on various attributes. This algorithm is mostly used in text classification and with problems having multiple classes.

What are the Pros and Cons of Naive Bayes?

Pros:

- It is easy and fast to predict class of test data set. It also perform well in multi class prediction
- When assumption of independence holds, a Naive Bayes classifier performs better compare to other models like logistic regression and you need less training data.
- It perform well in case of categorical input variables compared to numerical variable(s). For numerical variable, normal distribution is assumed (bell curve, which is a strong assumption).

Cons:

- If categorical variable has a category (in test data set), which was not observed in training data set, then model will assign a 0 (zero) probability and will be unable to make a prediction. This is often known as "Zero Frequency". To solve this, we can use the smoothing technique. One of the simplest smoothing techniques is called Laplace estimation.
- On the other side naive Bayes is also known as a bad estimator, so the probability outputs from predict_proba are not to be taken too seriously.
- Another limitation of Naive Bayes is the assumption of independent predictors. In real life, it is almost impossible that we get a set of predictors which are completely independent.

4 Applications of Naive Bayes Algorithms

- **Real time Prediction:** Naive Bayes is an eager learning classifier and it is sure fast. Thus, it could be used for making predictions in real time.
- **Multi class Prediction:** This algorithm is also well known for multi class prediction feature. Here we can predict the probability of multiple classes of target variable.
- **Text classification/ Spam Filtering/ Sentiment Analysis:** Naive Bayes classifiers mostly used in text classification (due to better result in multi class

problems and independence rule) have higher success rate as compared to other algorithms. As a result, it is widely used in Spam filtering (identify spam e-mail) and Sentiment Analysis (in social media analysis, to identify positive and negative customer sentiments)

- **Recommendation System:** Naive Bayes Classifier and Collaborative Filtering together builds a Recommendation System that uses machine learning and data mining techniques to filter unseen information and predict whether a user would like a given resource or not

Topic: 4 :	BAYESIAN NETWORK

✓ Bayesian belief network is key computer technology for dealing with probabilistic events and to solve a problem which has uncertainty. We can define a Bayesian network as:

"A Bayesian network is a probabilistic graphical model which represents a set of variables and their conditional dependencies using a directed acyclic graph."

✓ It is also called a **Bayes network, belief network, decision network**, or **Bayesian model**.

Bayesian networks are probabilistic, because these networks are built from a **probability distribution**, and also use probability theory for prediction and anomaly detection.

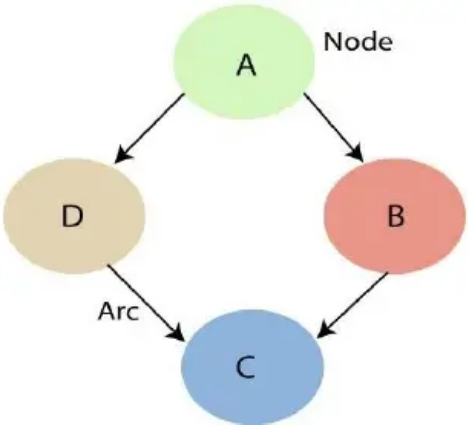
✓ Real world applications are probabilistic in nature, and to represent the relationship between multiple events, we need a Bayesian network.

✓ It can also be used in various tasks including **prediction, anomaly detection, diagnostics, automated insight, reasoning, time series prediction**, and **decision making under uncertainty**.

✓ Bayesian Network can be used for building models from data and experts opinions, and it consists of two parts:

- **Directed Acyclic Graph**
- **Table of conditional probabilities.**

- ✓ The generalized form of Bayesian network that represents and solve decision problems under uncertain knowledge is known as an **Influence diagram**.
- ✓ A Bayesian network graph is made up of nodes and Arcs (directed links), where:



- ✓ Each **node** corresponds to the random variables, and a variable can be **continuous** or **discrete**.
- ✓ **Arc or directed arrows** represent the causal relationship or conditional probabilities between random variables. These directed links or arrows connect the pair of nodes in the graph.
- ✓ These links represent that one node directly influence the other node, and if there is no directed link that means that nodes are independent with each other
  - In the above diagram, A, B, C, and D are random variables represented by the nodes of the network graph.
  - If we are considering node B, which is connected with node A by a directed arrow, then node A is called the parent of Node B.
  - Node C is independent of node A.

The Bayesian network has mainly two components:

- **Causal Component**
- **Actual numbers**

- ✓ Each node in the Bayesian network has condition probability distribution $\mathbf{P(X_i \mid Parent(X_i))}$, which determines the effect of the parent on that node.
- ✓ Bayesian network is based on Joint probability distribution and conditional probability.
- ✓ So let's first understand the joint probability distribution:

Joint Probability – It is the measure of two events happening at the same time. It can be written as P(A∩B)

Conditional Probability– It is the measure of the probability of an event occurring given that another event has occurred. In other terms, the conditional probability of an event X is the probability that the event will occur given that event Y has already occurred.

P(X|Y): Probability of event X occurring given that event Y already occurred.

If X and Y are dependent events, then P(X|Y) = P(X∩Y)/P(Y)

If X and Y are independent events, then P(X∩Y) = 0

So, P(X|Y) = P(X)

Let's understand the Bayesian network by an example.

Example Problem:

Calculate the probability that alarm has sounded, but there is neither a burglary, nor an earthquake occurred, and David and Sophia both called the Harry.

Solution:

- The Bayesian network for the above problem is given below. The network structure is showing that burglary and earthquake is the parent node of the alarm and directly affecting the probability of alarm's going off, but David and Sophia's calls depend on alarm probability.
- The network is representing that our assumptions do not directly perceive the burglary and also do not notice the minor earthquake, and they also not confer before calling.
- The conditional distributions for each node are given as conditional probabilities table or CPT.
- Each row in the CPT must be sum to 1 because all the entries in the table represent an exhaustive set of cases for the variable.
- In CPT, a boolean variable with k boolean parents contains 2^K probabilities. Hence, if there are two parents, then CPT will contain 4 probability values

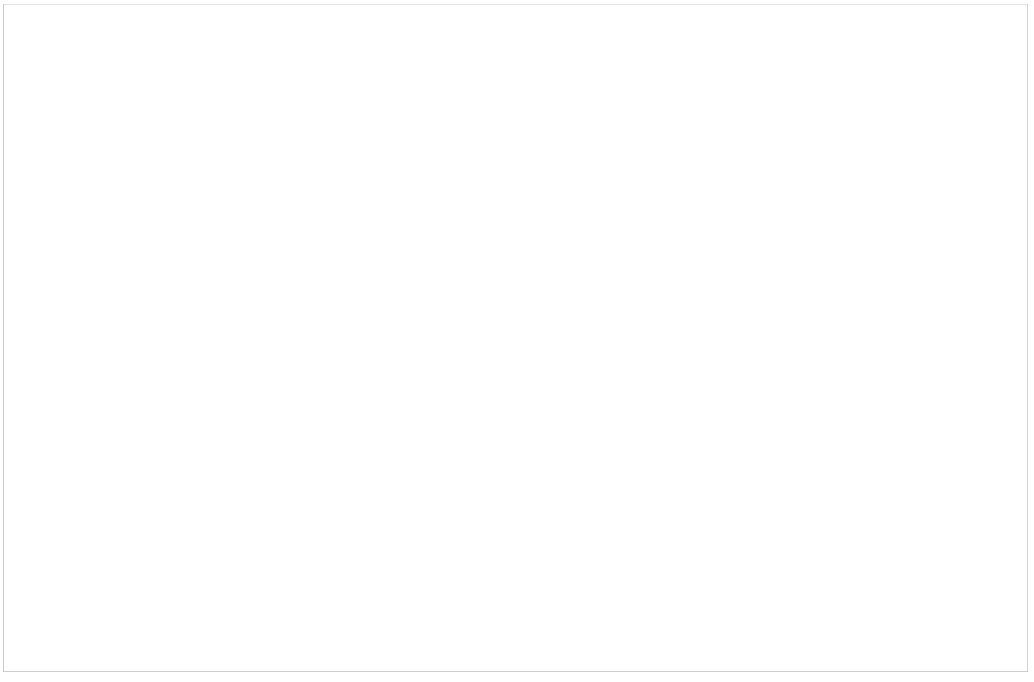
List of all events occurring in this network:

- **Burglary (B)**
- **Earthquake(E)**

- **Alarm(A)**
- **David Calls(D)**
- **Sophia calls(S)**

We can write the events of problem statement in the form of probability: **P[D, S, A, B, E]**, can rewrite the above probability statement using joint probability distribution:

P[D, S, A, B, E]= P[D | S, A, B, E]. P[S, A, B, E]
=P[D | S, A, B, E]. P[S | A, B, E]. P[A, B, E]
= P [D| A]. P [S| A, B, E]. P[A, B, E]
= P[D | A]. P[S | A]. P[A| B, E]. P[B, E]
= P[D | A]. P[S | A]. P[A| B, E]. P[B |E]. P[E]



Let's take the observed probability for the Burglary and earthquake component:

P(B= True) = 0.002, which is the probability of burglary.
P(B= False)= 0.998, which is the probability of no burglary.
P(E= True)= 0.001, which is the probability of a minor earthquake

P(E= False)= 0.999, Which is the probability that an earthquake not occurred.
We can provide the conditional probabilities as per the below tables:

Conditional probability table for Alarm A:
The Conditional probability of Alarm A depends on Burglar and earthquake:

B	E	P(A= True)	P(A= False)
True	True	0.94	0.06
True	False	0.95	0.04
False	True	0.31	0.69
False	False	0.001	0.999

Conditional probability table for David Calls:
The Conditional probability of David that he will call depends on the probability of Alarm.

A	P(D= True)	P(D= False)
True	0.91	0.09
False	0.05	0.95

Conditional probability table for Sophia Calls:
The Conditional probability of Sophia that she calls is depending on its Parent Node "Alarm."

A	P(S= True)	P(S= False)
True	0.75	0.25
False	0.02	0.98

From the formula of joint distribution, we can write the problem statement in the form of probability distribution:

P(S, D, A, ¬B, ¬E) = P (S|A) *P (D|A)*P (A|¬B ^ ¬E) *P (¬B) *P (¬E).
= 0.75* 0.91* 0.001* 0.998*0.999
= 0.00068045.

Hence, a Bayesian network can answer any query about the domain by using Joint distribution.

Application of Bayesian Network

Healthcare Industry: The Bayesian network is used in the healthcare industry for the detection and prevention of diseases. Based on models built, we can find out the likely symptoms and predict whether a person will be diseased or not. For instance, if a person has cholesterol, then there are high chances that the person gets a heart problem. With this information, a person can take preventive measures.

Web Search: Bayesian Network modes can be used for search accuracy based on user intent. Based on the user's intent, these models show things that are relevant to the person. For instance, when we search for Python functions most of the time, then the web search model activates our intent and it makes sure to show relevant things to the user.

Mail Spam Filtering: Gmail is using Bayesian Models to filter the mails by reading or understanding the context of mail. For instance, we may have observed spam emails in the spam folder in Gmail. So, how are these emails classified as spams? Using the Bayesian model, which observes the mail and based on the previous experience and the probability it classifies mail as spam or not.

Biomonitoring: Bayesian Models are used to quantify the concentration of chemicals in blood and human tissues. These use indicators to measure blood and urine. To find the level of ECCs, one can conduct biometric studies.

Information Retrieval: Models can be used for retrieving information from the database. During this process, we refine our problem multiple times. It is used to reduce information overload.

Topic: 5 :	INFERENCES

- Inference is the process of calculating a probability distribution of interest .
- The terms inference and queries are used interchangeably.

Types

1. Exact Inference
2. Approximate Inference

Exact Inference

- It is the term used when inference is performed exactly (subject to standard numerical rounding errors).
- Exact inference is applicable to a large range of problems, but may not be possible when combinations/paths get large.

Approximate inference

- Wider class of problems
- Non deterministic
- No guarantee of correct answer

14.4.1 Inference by enumeration

Chapter 13 explained that any conditional probability can be computed by summing terms from the full joint distribution. More specifically, a query $\mathbf{P}(X \mid \mathbf{e})$ can be answered using Equation (13.9), which we repeat here for convenience:

$$\mathbf{P}(X \mid \mathbf{e}) = \alpha \mathbf{P}(X, \mathbf{e}) = \alpha \sum_{\mathbf{y}} \mathbf{P}(X, \mathbf{e}, \mathbf{y}) .$$

Now, a Bayesian network gives a complete representation of the full joint distribution. More specifically, Equation (14.2) on page 513 shows that the terms $P(x, \mathbf{e}, \mathbf{y})$ in the joint distribution can be written as products of conditional probabilities from the network. Therefore, *a query can be answered using a Bayesian network by computing sums of products of conditional probabilities from the network.*

Consider the query $\mathbf{P}(Burglary \mid JohnCalls = true, MaryCalls = true)$. The hidden variables for this query are $Earthquake$ and $Alarm$. From Equation (13.9), using initial letters for the variables to shorten the expressions, we have[4]

$$\mathbf{P}(B \mid j, m) = \alpha \mathbf{P}(B, j, m) = \alpha \sum_{e} \sum_{a} \mathbf{P}(B, j, m, e, a,) .$$

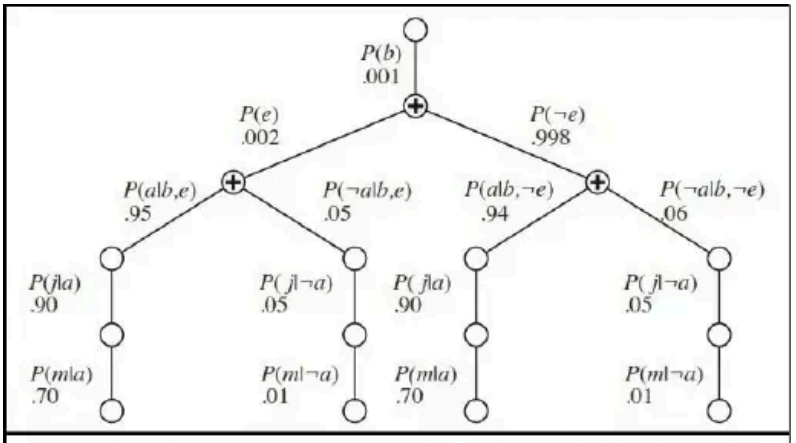


Figure 14.8 The structure of the expression shown in Equation (14.4). The evaluation proceeds top down, multiplying values along each path and summing at the "+" nodes. Notice the repetition of the paths for j and m.

Topic: 6 : CASUAL NETWORK

What is causal Network?

Causal reasoning is a crucial element to how humans understand, explain, and make decisions about the world. Causal AI means automating causal reasoning with machine learning. Today's learning machines have superhuman prediction ability but aren't particularly good at causal reasoning, even when we train them on obscenely large amounts of data. In this book, you will learn how to write algorithms that capture causal reasoning in the context of machine learning and automated data science.

Though humans rely heavily on causal reasoning to navigate the world, our cognitive biases make our causal inferences highly error-prone. We developed empiricism, the scientific method, and experimental statistics to address our tendencies to make errors in causal inference tasks such as finding and validating causal relations, distinguishing causality from mere correlation, and predicting the consequences of actions, decisions, and policies. Yet even empiricism still requires humans to interpret and explain observational data (data we observe in passing). The way we interpret causality from those different types of data is also error-prone. Causal AI attempts to use statistics,

probability, and computer science to help us surpass these errors in our reasoning.

The difficulty of answering causal questions has motivated the work of millennia of philosophers, centuries of scientists, and decades of statisticians. But now, a convergence of statistical and computational advances has shifted the focus from discourse to algorithms that we can train on data and deploy to software. It is now a fascinating time to learn how to build causal AI.

Simulated experiments and causal effect inference

Causal effect inference – quantifying how much a cause (e.g., a promotion) affects an effect (e.g., sales) is the most common goal of applied data science. The gold standard for causal effect inference is randomized experiments, such as an A/B test. The concepts of causal inference explain why randomized experiments work so well; randomization eliminates non-causal sources of statistical correlation.

More importantly, causal inference enables data scientists to simulate experiments and estimate causal effects from *observational data*. Most data in the world is observational data because most data is not the result of an experiment. When people say "big data," they almost always mean observational data. When a tech company boasts of training a natural language model on petabytes of Internet data, it is observational data. When we can't run a randomized experiment because of infeasibility, cost, or ethics, causal inference enables data scientists to turn to observational data to estimate causal effects.

A common belief is that in the era of big data, it is easy to run a virtually unlimited amount of experiments. If you can run unlimited experiments, who needs causal inference? But even when you can run experiments at little actual cost, there are often opportunity costs to running experiments. For example, suppose a data scientist at an e-commerce company has a choice of one thousand experiments to run and running each one would take time and potentially sacrifice some sales or clicks in the process. Causal modeling allows that data scientist to use results from a past experiment that answers one question to simulate the results of a new experiment that would answer a slightly different question. That means she could use past data to simulate the results for each of these one thousand experiments and then prioritize the experiments by those predicted to have the most impactful results in simulation. She avoids wasting clicks and sales on less insightful experiments.

Training large machine learning artifacts is often more art than science, relying on the intuition and experience of individual machine learning engineers. When they leave the organization to move on to their next opportunity, they take all of that intuition and experience with them. In contrast, by virtue of being decomposable and explainable, it is easier for your team to understand individual components of the model. Further, by forcing the engineer to encode their causal knowledge about a problem domain into the structure of the artifact, more of the fruits of their knowledge and experience stays with the organization as valuable intellectual property after that employee moves on.